

Código Final

Modelo Novo-Keynesiano de Política Monetária para o Chile
Todo o código (Python + Dynare) em um único documento

Daniel Colli e João Zangrandi

16/06/2026

Este documento reúne **todo** o código do projeto: 49 arquivos (Python e Dynare `.mod`), agrupados na ordem do *pipeline*. Caracteres não-ASCII do código (letras gregas, acentos) foram transliterados para ASCII na listagem para garantir a compilação; a lógica é idêntica à dos arquivos-fonte do repositório.

Contents

Infraestrutura compartilhada	2
1 python/common.py	2
Dados (BCCh / FRED / filtro de Kalman)	6
2 python/build_chile_dataset.py	6
3 python/reprocess_dataset.py	9
4 python/build_open_economy_dataset.py	10
5 python/discover_ipc_series.py	12
Geracao dos modelos Dynare	13
6 python/generate_dynare_models.py	13
7 python/generate_macro_extension_models.py	15
Modelos Dynare (.mod)	17
8 dynare/nk_chile_base.mod	17
9 dynare/nk_chile_estim.mod	18
10 dynare/nk_chile_hybrid.mod	19
11 dynare/nk_chile_hybrid_estim.mod	20
12 dynare/nk_chile_forecast.mod	21
13 dynare/nk_chile_history.mod	23
14 dynare/nk_chile_history_hybrid.mod	24
15 dynare/nk_chile_mcmc.mod	25
16 dynare/nk_chile_open.mod	26
Solucao e calibracao	27
17 python/hybrid_solution.py	27
18 python/calibrate_shocks.py	29
19 python/analyze_determinacy.py	32

Estimacao dos parametros	33
20 python/estimate_nkpc.py	33
21 python/estimate_taylor_rule.py	35
22 python/estimate_rhoi_chile.py	37
23 python/estimate_rstar.py	39
24 python/estimate_time_varying_rstar.py	40
Execucao (Octave/Dynare + Bayesiano)	42
25 python/run_dynare_batch.py	42
26 python/run_bayesian.py	44
27 python/run_mcmc.py	46
28 python/run_forecast.py	49
29 python/run_history.py	51
30 python/run_macro_extensions.py	53
Previsao e avaliacao	54
31 python/forecast_model.py	54
32 python/evaluate_forecasts.py	58
Analise (SVAR, Bayes, resultados)	60
33 python/analyze_svar.py	60
34 python/compare_bayesian_models.py	62
35 python/analyze_model_results.py	65
36 python/analyze_macro_extensions.py	69
Nucleos da inflacao e mercado de trabalho	71
37 python/build_inflation_cores.py	71
38 python/build_inflation_cores_bcch.py	73
39 python/build_labor_activity.py	75
Graficos	77
40 python/plot_history.py	77
41 python/plot_ipc_decomposition.py	80
42 python/plot_forecast_outlook.py	82
43 python/plot_hybrid_test.py	84
44 python/plot_irfs.py	85
45 python/plot_mcmc.py	88
Tabelas, coleta e entregaveis	89

46	python/collect_outputs.py	89
47	python/make_tables.py	92
48	python/build_final_html.py	94
49	python/build_codigo_final.py	105

Infraestrutura compartilhada

1 python/common.py

```
1  """Shared paths and lightweight model-based fallback calculations."""
2
3  from __future__ import annotations
4
5  import math
6  import os
7  import shutil
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12
13 ROOT = Path(__file__).resolve().parents[1]
14 DATA_RAW = ROOT / "data" / "raw"
15 DATA_CLEAN = ROOT / "data" / "clean"
16 TABLES = ROOT / "outputs" / "tables"
17 FIGURES = ROOT / "outputs" / "figures"
18 LOGS = ROOT / "outputs" / "logs"
19 DYNARE_OUTPUTS = ROOT / "outputs" / "dynare"
20
21 # Shock standard deviations for the Chile baseline. They are calibrated to an
22 # explicitly illustrative FEVD target at the estimated interest-rate persistence.
23 # The target is a modelling choice, not an empirical decomposition published by BCCh.
24 SHOCK_STD = {"e_x": 0.00500, "e_pi": 0.00277, "e_i": 0.00068}
25 INITIAL_SHOCK_STD = {"e_x": 0.0050, "e_pi": 0.0025, "e_i": 0.0015}
26 COURSE_BENCHMARK_SHOCK_STD_FALLBACK = {
27     "e_x": 0.00500,
28     "e_pi": 0.00331,
29     "e_i": 0.00080,
30 }
31 BASELINE = {
32     "rstar_annual": 0.03,
33     "sigma": 1.0,
34     "kappa": 0.10,
35     "rho_i": 0.80,
36     "phi_pi": 1.75,
37     "phi_x": 0.50,
38 }
39
40
41 def ensure_directories() -> None:
42     for path in (DATA_RAW, DATA_CLEAN, TABLES, FIGURES, LOGS, DYNARE_OUTPUTS):
43         path.mkdir(parents=True, exist_ok=True)
44
45
46 def quarterly_rstar(annual_rate: float) -> float:
47     return (1.0 + annual_rate) ** 0.25 - 1.0
48
49
50 def kalman_gap(log_level) -> tuple[np.ndarray, np.ndarray]:
51     """Trend-cycle decomposition by an Unobserved-Components (Kalman) model.
52
53     Replaces the HP filter everywhere in the project. The series (a log level) is
54     split into a smooth stochastic trend and a stochastic, damped cycle; the cycle
55     is the output-gap analogue. A smooth (integrated random-walk) trend keeps sharp
56     shocks such as COVID-2020 in the CYCLE rather than the trend. Returns
57     (cycle, trend) as numpy arrays of the same length as the input.
58     """
59     from statsmodels.tsa.statespace.structural import UnobservedComponents
60
61     raw = np.asarray(log_level, dtype=float)
62     y = raw * 100.0 # log points: far better-conditioned for the optimiser
63     # An AR(2) cycle on a smooth (I(2)) trend is the classic Clark/Watson gap; the
64     # cycle is constrained enough to capture the business cycle instead of letting
65     # a flexible trend absorb it (which collapses the gap to ~0).
66     attempts = (
67         ("autoregressive", dict(level="smooth trend", autoregressive=2)),
68         ("cycle", dict(level="smooth trend", cycle=True, stochastic_cycle=True,
69                        damped_cycle=True, cycle_period_bounds=(6, 40))),
70         ("autoregressive", dict(level="local linear trend", autoregressive=2)),
71     )
72     last_error: Exception | None = None
73     for component, spec in attempts:
74         try:
75             result = UnobservedComponents(y, **spec).fit(disp=False, maxiter=3000)
76             cycle = np.asarray(getattr(result, component).smoothed, dtype=float) / 100.0
77             try:
78                 trend = np.asarray(result.level.smoothed, dtype=float) / 100.0
79             except Exception: # noqa: BLE001
80                 trend = raw - cycle
81             if cycle.shape == raw.shape and np.all(np.isfinite(cycle)) and np.nanstd(cycle) > 1e-3:
82                 return cycle, trend
83             except Exception as exc: # noqa: BLE001
84                 last_error = exc
85     raise RuntimeError(f"kalman_gap: no UnobservedComponents spec converged ({last_error}).")
86
87
88 def complete_parameters(overrides: dict | None = None) -> dict:
89     params = dict(BASELINE)
90     if overrides:
91         params.update(overrides)
92     params["rstar"] = quarterly_rstar(float(params["rstar_annual"]))
93     params["beta"] = 1.0 / (1.0 + params["rstar"])
94     return params
95
96
97 def calibration_rho() -> float:
98     estimate_file = TABLES / "rhoi_estimate.csv"
```

```

99     if not estimate_file.exists():
100         return BASELINE["rho_i"]
101     table = pd.read_csv(estimate_file)
102     if table.empty or "rho_i_used" not in table:
103         return BASELINE["rho_i"]
104     value = float(table.loc[0, "rho_i_used"])
105     return value if math.isfinite(value) else BASELINE["rho_i"]
106
107
108 def course_benchmark_shock_std() -> dict[str, float]:
109     """Read shock sizes fitted to the didactic FEVD benchmark.
110
111     The fitted values depend on the current rho_i calibration. The CSV is produced
112     by calibrate_shocks.py; fixed fallbacks keep model generation runnable before
113     that calibration script has been executed.
114     """
115
116     path = TABLES / "shock_sigmas_comparison.csv"
117     if not path.exists():
118         return dict(COURSE_BENCHMARK_SHOCK_STD_FALLBACK)
119     table = pd.read_csv(path)
120     selected = table[table["calibration"] == "didactic_benchmark"]
121     if selected.empty:
122         return dict(COURSE_BENCHMARK_SHOCK_STD_FALLBACK)
123     values = {
124         str(row["shock"]): float(row["sigma_calibrated"])
125         for _, row in selected.iterrows()
126     }
127     return {
128         shock: values.get(shock, fallback)
129         for shock, fallback in COURSE_BENCHMARK_SHOCK_STD_FALLBACK.items()
130     }
131
132
133 def find_octave() -> Path | None:
134     configured = os.environ.get("OCTAVE_BIN")
135     candidates = [
136         configured,
137         shutil.which("octave-cli"),
138         shutil.which("octave"),
139         r"C:\Program Files\GNU Octave\Octave-11.1.0\mingw64\bin\octave-cli.exe",
140         r"C:\Program Files\GNU Octave\Octave-10.3.0\mingw64\bin\octave-cli.exe",
141     ]
142     for candidate in candidates:
143         if candidate and Path(candidate).exists():
144             return Path(candidate)
145     octave_root = Path(r"C:\Program Files\GNU Octave")
146     if octave_root.exists():
147         matches = sorted(octave_root.glob(r"Octave-*\mingw64\bin\octave-cli.exe"))
148         if matches:
149             return matches[-1]
150     return None
151
152
153 def find_dynare_matlab() -> Path | None:
154     configured = os.environ.get("DYNARE_MATLAB")
155     candidates = [
156         configured,
157         r"C:\dynare\7.1\matlab",
158         r"C:\dynare\7.0\matlab",
159         r"C:\Program Files\Dynare\7.1\matlab",
160     ]
161     for candidate in candidates:
162         if candidate and (Path(candidate) / "dynare.m").exists():
163             return Path(candidate)
164     dynare_root = Path(r"C:\dynare")
165     if dynare_root.exists():
166         matches = sorted(dynare_root.glob(r"*\matlab\dynare.m"))
167         if matches:
168             return matches[-1].parent
169     return None
170
171
172 def _state_coefficients(params: dict) -> tuple[float, float, float]:
173     """Exact minimum-state-variable solution of the linear NK model.
174
175     The only predetermined state is the lagged policy rate, so in the absence of
176     shocks the dynamics collapse to a single stable eigenvalue  $a_i$ :
177      $x_t = a_x x_{t-1}$ ,  $\pi_t = a_\pi \pi_{t-1}$ ,  $i_t = a_i i_{t-1}$ .
178      $a_i$  is the stable root ( $|a_i| < 1$ ) of the system's characteristic cubic, which is
179     unique under Blanchard-Kahn determinacy (one predetermined variable, two jumpers).
180     """
181
182     beta = params["beta"]
183     sigma = params["sigma"]
184     kappa = params["kappa"]
185     rho_i = params["rho_i"]
186     phi_pi = params["phi_pi"]
187     phi_x = params["phi_x"]
188
189     # Characteristic cubic  $P(a)=0$  from  $a_x = a_x a_i + \dots$ ,  $a_\pi$ , and the Taylor rule:
190     #  $P(a) = (a - \rho_i)(\beta a^2 - (1 + \beta \kappa / \sigma) a + 1)$ 
191     #  $+ ((1 - \rho_i) / \sigma) * (\phi_\pi \kappa + \phi_x) a - \phi_x \beta a^2$ 
192     q2, q1, q0 = beta, -(1.0 + beta + kappa / sigma), 1.0
193     c3 = q2
194     c2 = q1 - rho_i * q2
195     c1 = q0 - rho_i * q1
196     c0 = -rho_i * q0
197     g = (1.0 - rho_i) / sigma
198     c2 += -g * phi_x * beta
199     c1 += g * (phi_pi * kappa + phi_x)
200     roots = np.roots([c3, c2, c1, c0])
201
202     stable = [r.real for r in roots if abs(r.imag) < 1e-8 and abs(r) < 1.0 - 1e-10]
203     if not stable:

```

```

204     stable = [min(roots, key=lambda z: abs(z)).real]
205     a_i = float(min(stable, key=lambda v: abs(v - rho_i)))
206
207     determinant = (1.0 - a_i) * (1.0 - beta * a_i) - a_i * kappa / sigma
208     a_x = (-a_i / sigma) * (1.0 - beta * a_i) / determinant
209     a_pi = (-a_i * kappa / sigma) / determinant
210     return a_x, a_pi, a_i
211
212
213 def characteristic_roots(params: dict) -> np.ndarray:
214     """Eigenvalues (roots of the characteristic cubic) of the linear NK system.
215
216     With one predetermined variable (the lagged rate) and two jumpers, Blanchard-Kahn
217     determinacy requires exactly one root inside the unit circle.
218     """
219     beta = params["beta"]
220     sigma = params["sigma"]
221     kappa = params["kappa"]
222     rho_i = params["rho_i"]
223     phi_pi = params["phi_pi"]
224     phi_x = params["phi_x"]
225     q2, q1, q0 = beta, -(1.0 + beta + kappa / sigma), 1.0
226     c3 = q2
227     c2 = q1 - rho_i * q2
228     c1 = q0 - rho_i * q1
229     c0 = -rho_i * q0
230     g = (1.0 - rho_i) / sigma
231     c2 += -g * phi_x * beta
232     c1 += g * (phi_pi * kappa + phi_x)
233     return np.roots([c3, c2, c1, c0])
234
235
236 def fallback_irfs(
237     params: dict, scenario: str, horizon: int = 20
238 ) -> pd.DataFrame:
239     """Generate model-based synthetic IRFs when Dynare output is unavailable."""
240
241     a_x, a_pi, a_i = _state_coefficients(params)
242     sigma = params["sigma"]
243     rho_i = params["rho_i"]
244     phi_pi = params["phi_pi"]
245     phi_x = params["phi_x"]
246     beta = params["beta"]
247
248     c_i = a_x - (1.0 - a_pi) / sigma
249     impact_matrix = np.array(
250         [
251             [1.0, 0.0, -c_i],
252             [-params["kappa"], 1.0, -beta * a_pi],
253             [
254                 -(1.0 - rho_i) * phi_x,
255                 -(1.0 - rho_i) * phi_pi,
256                 1.0,
257             ],
258         ]
259     )
260
261     records = []
262     variables = ("x", "pi", "i")
263     shocks = ("e_x", "e_pi", "e_i")
264     for shock_index, shock in enumerate(shocks):
265         innovation = np.zeros(3)
266         innovation[shock_index] = SHOCK_STD[shock]
267         impact = np.linalg.solve(impact_matrix, innovation)
268         responses = [impact]
269         lagged_rate = impact[2]
270         for _ in range(1, horizon):
271             current = np.array([a_x, a_pi, a_i]) * lagged_rate
272             responses.append(current)
273             lagged_rate = current[2]
274         for period, response in enumerate(responses):
275             for variable, value in zip(variables, response):
276                 records.append(
277                     {
278                         "scenario": scenario,
279                         "source": "synthetic_model_fallback",
280                         "variable": variable,
281                         "shock": shock,
282                         "horizon": period,
283                         "response": float(value),
284                     }
285                 )
286     return pd.DataFrame.from_records(records)
287
288
289 def fallback_moments(params: dict, observations: int = 50000) -> pd.DataFrame:
290     """Simulate deviation moments from the fallback solution."""
291
292     a_x, a_pi, _ = _state_coefficients(params)
293     sigma = params["sigma"]
294     rho_i = params["rho_i"]
295     beta = params["beta"]
296     c_i = a_x - (1.0 - a_pi) / sigma
297     impact_matrix = np.array(
298         [
299             [1.0, 0.0, -c_i],
300             [-params["kappa"], 1.0, -beta * a_pi],
301             [
302                 -(1.0 - rho_i) * params["phi_x"],
303                 -(1.0 - rho_i) * params["phi_pi"],
304                 1.0,
305             ],
306         ]
307     )
308

```

```

309 rng = np.random.default_rng(20260614)
310 values = np.zeros((observations, 3))
311 lagged_rate = 0.0
312 std = np.array([SHOCK_STD["e_x"], SHOCK_STD["e_pi"], SHOCK_STD["e_i"]])
313 for t in range(observations):
314     rhs = rng.normal(0.0, std)
315     rhs[2] += rho_i * lagged_rate
316     values[t] = np.linalg.solve(impact_matrix, rhs)
317     lagged_rate = values[t, 2]
318
319 rows = []
320 for index, variable in enumerate(("x", "pi", "i")):
321     series = values[:, index]
322     mean_value = float(series.mean())
323     if variable == "i":
324         mean_value += params["rstar"]
325     rows.append(
326         {
327             "variable": variable,
328             "mean": mean_value,
329             "std_dev": float(series.std(ddof=1)),
330             "variance": float(series.var(ddof=1)),
331             "autocorrelation_1": float(np.corrcoef(series[1:], series[:-1])[0, 1]),
332             "source": "synthetic_model_fallback",
333         }
334     )
335 return pd.DataFrame(rows)

```

Dados (BCCh / FRED / filtro de Kalman)

2 python/build_chile_dataset.py

```
1 """Build the real Chilean quarterly dataset from the Banco Central de Chile (BCCh).
2
3 Source: BCCh "Base de Datos Estadísticos" REST web service (SieteRestWS).
4 Series actually downloaded (confirmed available):
5 - TPM (monetary policy rate, daily, % p.a.): F022.TPM.TIN.D001.NO.Z.D
6 - IPC (CPI monthly variation, %): F074.IPC.VAR.Z.Z.C.M
7 - PIB (real GDP, chained vol., seasonally adj.,
8   spliced, reference 2018, quarterly): F032.PIB.FLU.R.CLP.EP18.Z.Z.1.T
9
10 Credentials are read from the environment variables BCCH_USER / BCCH_PASS, or from a
11 local, git-ignored file data/raw/bcch_credentials.json with {"user": ..., "pass": ...}.
12 Credentials are never written to data/clean or printed.
13
14 If credentials are missing or the download fails, the script falls back to a clearly
15 labelled SYNTHETIC AR(1) series (for pipeline testing only) and flags is_synthetic=True.
16
17 Outputs (all in data/clean):
18 - chile_policy_rate.csv quarterly policy rate (% p.a.) for the AR(1) of rho_i
19 - chile_macro_quarterly.csv full quarterly panel (rate, inflation, gdp, gap, real rate)
20 - chile_observables.csv model-unit deviations (pi, i, x) for Dynare estimation
21 - dataset_metadata.json provenance (institution, series ids, units, dates, transforms)
22 """
23
24 from __future__ import annotations
25
26 import json
27 import datetime as dt
28 from pathlib import Path
29
30 import numpy as np
31 import pandas as pd
32
33 from common import DATA_CLEAN, DATA_RAW, ensure_directories, kalman_gap
34
35 BCCH_BASE = "https://si3.bcentral.cl/SieteRestWS/SieteRestWS.ashx"
36 SERIES = {
37     "tpm": "F022.TPM.TIN.D001.NO.Z.D",
38     "ipc_var": "F074.IPC.VAR.Z.Z.C.M",
39     "pib": "F032.PIB.FLU.R.CLP.EP18.Z.Z.1.T",
40 }
41 START_DATE = "2001-01-01"
42 INFLATION_TARGET_ANNUAL = 0.03 # Meta de inflacion del BCCh: 3% anual.
43 HP_LAMBDA = 1600 # Convencion trimestral.
44
45 # ----- #
46 # Credentials and HTTP access
47 # ----- #
48
49 def get_credentials() -> tuple[str, str] | None:
50     import os
51
52     user = os.environ.get("BCCH_USER")
53     password = os.environ.get("BCCH_PASS")
54     if user and password:
55         return user, password
56     creds_file = DATA_RAW / "bcch_credentials.json"
57     if creds_file.exists():
58         try:
59             payload = json.loads(creds_file.read_text(encoding="utf-8"))
60             user = payload.get("user")
61             password = payload.get("pass") or payload.get("password")
62             if user and password:
63                 return str(user), str(password)
64         except Exception as exc: # noqa: BLE001
65             print(f"Warning: could not read bcch_credentials.json: {exc}")
66     return None
67
68
69 def fetch_series(series_id: str, user: str, password: str,
70                 first: str = START_DATE, last: str | None = None) -> pd.Series:
71     """Download one BCCh series and return a float Series indexed by date."""
72     import urllib.parse
73     import urllib.request
74
75     if last is None:
76         last = dt.date.today().isoformat()
77     query = urllib.parse.urlencode(
78         {
79             "user": user,
80             "pass": password,
81             "function": "GetSeries",
82             "timeseries": series_id,
83             "firstdate": first,
84             "lastdate": last,
85         }
86     )
87     url = f"{BCCH_BASE}?{query}"
88     with urllib.request.urlopen(url, timeout=60) as response:
89         payload = json.loads(response.read().decode("utf-8", "replace"))
90     if payload.get("Codigo") != 0:
91         raise RuntimeError(
92             f"BCCh error for {series_id}: {payload.get('Descripcion')}"
93         )
94     series_block = payload.get("Series") or {}
95     observations = series_block.get("Obs") or []
96     dates, values = [], []
97     for obs in observations:
98         if str(obs.get("statusCode", "")).upper() not in ("OK", ""):
```



```

99         continue
100     raw_value = str(obs.get("value", "")).replace(",", ".").strip()
101     if raw_value in ("", "NaN", "ND"):
102         continue
103     try:
104         value = float(raw_value)
105     except ValueError:
106         continue
107     dates.append(pd.to_datetime(obs["indexDateString"], format="%d-%m-%Y"))
108     values.append(value)
109 if not dates:
110     raise RuntimeError(f"BCCh returned no usable observations for {series_id}.")
111 return pd.Series(values, index=pd.DatetimeIndex(dates), name=series_id).sort_index()
112
113
114 # ----- #
115 # Real dataset assembly
116 # ----- #
117 def build_real_panel(user: str, password: str) -> tuple[pd.DataFrame, dict]:
118     tpm_daily = fetch_series(SERIES["tpm"], user, password)
119     ipc_var_m = fetch_series(SERIES["ipc_var"], user, password)
120     pib_q = fetch_series(SERIES["pib"], user, password)
121
122     # Policy rate: quarterly mean of the daily TPM (annual %).
123     tpm_q = tpm_daily.groupby(tpm_daily.index.to_period("Q")).mean()
124
125     # Inflation: compound three monthly CPI variations into a quarterly fraction.
126     ipc_frac = ipc_var_m / 100.0
127     grouped = ipc_frac.groupby(ipc_frac.index.to_period("Q"))
128     infl_q = grouped.apply(lambda s: float(np.prod(1.0 + s.to_numpy()) - 1.0))
129     months_per_quarter = grouped.size()
130     infl_q = infl_q[months_per_quarter == 3] # keep only complete quarters
131
132     # GDP: real, seasonally adjusted, quarterly -> log -> HP cycle (output gap).
133     pib_q.index = pib_q.index.to_period("Q")
134     pib_q = pib_q[-pib_q.index.duplicated(keep="last")].sort_index()
135
136     panel = pd.DataFrame(
137         {
138             "tpm_annual_pct": tpm_q,
139             "infl_q": infl_q,
140             "pib_sa": pib_q,
141         }
142     ).dropna()
143     if len(panel) < 20:
144         raise RuntimeError(
145             f"Only {len(panel)} overlapping quarters; expected a long sample."
146         )
147
148     panel["log_pib"] = np.log(panel["pib_sa"])
149     cycle, trend = kalman_gap(panel["log_pib"].to_numpy()) # Kalman UC, not HP
150     panel["pib_trend_kalman"] = trend
151     panel["output_gap"] = cycle
152
153     # Quarterly nominal rate as a fraction (model unit), Fisher-consistent.
154     panel["i_q"] = (1.0 + panel["tpm_annual_pct"] / 100.0) ** 0.25 - 1.0
155     panel["infl_annual_pct"] = ((1.0 + panel["infl_q"]) ** 4 - 1.0) * 100.0
156     panel["real_rate_q"] = panel["i_q"] - panel["infl_q"]
157
158     panel.index = panel.index.astype(str)
159     panel.index.name = "date"
160
161     metadata = {
162         "institution": "Banco Central de Chile (BCCh)",
163         "service": "Base de Datos Estadísticos - SieteRestWS REST API",
164         "url_pattern": f"{BCCH_BASE}?function=GetSeries&timeseries=<ID>&firstdate=...&lastdate=...",
165         "official_bde_url": "https://si3.bcentral.cl/Siete",
166         "policy_framework_url": "https://www.bcentral.cl/web/banco-central/areas/politica-monetaria",
167         "terms_of_use_url": "https://www.bcentral.cl/web/banco-central/condiciones-de-uso",
168         "series_ids": SERIES,
169         "frequency": "quarterly (built from daily TPM, monthly CPI, quarterly GDP)",
170         "period": f"{panel.index[0]} to {panel.index[-1]}",
171         "observations": int(len(panel)),
172         "access_date": dt.date.today().isoformat(),
173         "units": {
174             "tpm_annual_pct": "TPM, percent per year (quarterly average of daily series)",
175             "i_q": "nominal policy rate, quarterly fraction = (1+TPM/100)^(1/4)-1",
176             "infl_q": "quarterly CPI inflation, fraction (compounded monthly variations)",
177             "infl_annual_pct": "annualised quarterly inflation, percent",
178             "pib_sa": "real GDP, chained volume, seasonally adjusted, ref 2018 (CLP bn)",
179             "output_gap": "HP-filter cycle of log real GDP (lambda=1600), fraction",
180             "real_rate_q": "ex-post real quarterly rate = i_q - infl_q",
181         },
182         "transformations": (
183             "TPM: daily->quarterly mean. CPI: monthly % -> quarterly compounded fraction. "
184             f"GDP: log + HP filter (lambda={HP_LAMBDA}) for the output gap."
185         ),
186         "inflation_target_annual": INFLATION_TARGET_ANNUAL,
187         "is_synthetic": False,
188         "note": "Real official data from BCCh; credentials are not stored in this file.",
189     }
190     return panel, metadata
191
192
193 def synthetic_panel() -> tuple[pd.DataFrame, dict]:
194     """Clearly-labelled synthetic fallback for pipeline testing only."""
195     rng = np.random.default_rng(20260614)
196     periods = pd.period_range("2001Q3", "2026Q1", freq="Q")
197     long_run = 4.5
198     rho = 0.92
199     tpm = np.empty(len(periods))
200     tpm[0] = long_run
201     for i in range(1, len(periods)):
202         tpm[i] = max(0.25, long_run + rho * (tpm[i - 1] - long_run) + rng.normal(0, 0.5))
203     infl_q = 0.0074 + 0.5 * (tpm / 100.0 / 4 - 0.0074) + rng.normal(0, 0.004, len(periods))

```

```

204 gap = np.zeros(len( periods ))
205 for i in range(1, len( periods )):
206     gap[i] = 0.7 * gap[i - 1] + rng.normal(0, 0.01)
207 panel = pd.DataFrame(
208     {
209         "tpm_annual_pct": tpm,
210         "infl_q": infl_q,
211         "pib_sa": 100.0 * np.exp(np.cumsum(0.005 + gap * 0.0)),
212         "output_gap": gap,
213     },
214     index=periods.astype(str),
215 )
216 panel.index.name = "date"
217 panel["log_pib"] = np.log(panel["pib_sa"])
218 panel["pib_trend_hp"] = panel["log_pib"]
219 panel["i_q"] = (1.0 + panel["tpm_annual_pct"] / 100.0) ** 0.25 - 1.0
220 panel["infl_annual_pct"] = ((1.0 + panel["infl_q"]) ** 4 - 1.0) * 100.0
221 panel["real_rate_q"] = panel["i_q"] - panel["infl_q"]
222 metadata = {
223     "institution": "SYNTHETIC (no real data)",
224     "is_synthetic": True,
225     "source": "synthetic_ari_pipeline_test_not_chilean_data",
226     "access_date": dt.date.today().isoformat(),
227     "note": "No BCCh credentials available; generated a labelled synthetic panel.",
228 }
229 return panel, metadata
230
231
232 # ----- #
233 def main() -> None:
234     ensure_directories()
235     credentials = get_credentials()
236     if credentials is not None:
237         try:
238             panel, metadata = build_real_panel(*credentials)
239             print(f"Downloaded real BCCh data: {metadata['period']} "
240                   f"{metadata['observations']} quarters.")
241         except Exception as exc: # noqa: BLE001
242             print(f"WARNING: BCCh download failed ({exc}). Using synthetic fallback.")
243             panel, metadata = synthetic_panel()
244     else:
245         print("WARNING: no BCCh credentials (set BCCH_USER/BCCH_PASS or "
246               "data/raw/bcch_credentials.json). Using synthetic fallback.")
247         panel, metadata = synthetic_panel()
248
249     is_synth = bool(metadata.get("is_synthetic", False))
250     source_label = (
251         "synthetic_ari_pipeline_test_not_chilean_data"
252         if is_synth
253         else f"BCCh:{SERIES['tpm']}"
254     )
255
256     # 1) Policy-rate file consumed by the AR(1) estimation.
257     policy = pd.DataFrame(
258         {
259             "date": panel.index,
260             "policy_rate": panel["tpm_annual_pct"].to_numpy(),
261             "is_synthetic": is_synth,
262             "data_source": source_label,
263         }
264     )
265     policy.to_csv(DATA_CLEAN / "chile_policy_rate.csv", index=False)
266
267     # 2) Full quarterly macro panel.
268     macro_cols = [
269         "tpm_annual_pct", "i_q", "infl_q", "infl_annual_pct",
270         "pib_sa", "log_pib", "pib_trend_hp", "output_gap", "real_rate_q",
271     ]
272     panel.reset_index()[["date"] + macro_cols].to_csv(
273         DATA_CLEAN / "chile_macro_quarterly.csv", index=False
274     )
275
276     # 3) Model-unit observables (deviations from steady state) for estimation.
277     observables = pd.DataFrame(
278         {
279             "date": panel.index,
280             "pi": (panel["infl_q"] - ((1.0 + INFLATION_TARGET_ANNUAL) ** 0.25 - 1.0)).to_numpy(),
281             "i": (panel["i_q"] - panel["i_q"].mean()).to_numpy(),
282             "x": (panel["output_gap"] - panel["output_gap"].mean()).to_numpy(),
283         }
284     )
285     observables.to_csv(DATA_CLEAN / "chile_observables.csv", index=False)
286
287     metadata["files"] = {
288         "policy_rate": "chile_policy_rate.csv",
289         "macro_panel": "chile_macro_quarterly.csv",
290         "observables": "chile_observables.csv (deviations: pi, i, x)",
291     }
292     (DATA_CLEAN / "dataset_metadata.json").write_text(
293         json.dumps(metadata, indent=2, ensure_ascii=False), encoding="utf-8"
294     )
295     print(f"Wrote chile_policy_rate.csv, chile_macro_quarterly.csv, "
296           f"chile_observables.csv ({len(panel)} quarters). synthetic={is_synth}")
297
298
299 if __name__ == "__main__":
300     main()

```

3 python/reprocess_dataset.py

```
1 """Re-derive the output gap (Kalman, not HP) and the model observables from the
2 cached BCCh panel, WITHOUT re-hitting the BCCh API.
3
4 Two methodological changes requested:
5 * the output gap is an Unobserved-Components (Kalman) cycle, replacing the HP filter;
6 * the inflation observable is centred on the 3% TARGET (not the sample mean), so the
7   model's steady state is the target and forecasts converge to 3% by construction.
8
9 The rate stays demeaned by its sample mean (= realised neutral; Fisher-implied r*).
10 build_chile_dataset.py carries the same logic for a fresh pull, but it needs BCCh
11 credentials; this script works from the committed cache (data/clean/*.csv).
12 """
13
14 from __future__ import annotations
15
16 import json
17
18 import numpy as np
19 import pandas as pd
20
21 from common import DATA_CLEAN, ensure_directories, kalman_gap
22
23 INFLATION_TARGET_ANNUAL = 0.03
24
25
26 def main() -> None:
27     ensure_directories()
28     panel = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
29
30     # --- Output gap by Kalman (Unobserved Components) instead of HP ---
31     cycle, trend = kalman_gap(panel["log_pib"].to_numpy())
32     panel["output_gap_hp"] = panel["output_gap"] # keep HP for reference
33     panel["output_gap"] = cycle
34     panel["pib_trend_kalman"] = trend
35     panel.to_csv(DATA_CLEAN / "chile_macro_quarterly.csv", index=False)
36
37     # --- Observables: inflation centred on the 3% target ---
38     target_q = (1.0 + INFLATION_TARGET_ANNUAL) ** 0.25 - 1.0
39     observables = pd.DataFrame(
40         {
41             "date": panel["date"],
42             "pi": (panel["infl_q"] - target_q).to_numpy(),
43             "i": (panel["i_q"] - panel["i_q"].mean()).to_numpy(),
44             "x": (panel["output_gap"] - panel["output_gap"].mean()).to_numpy(),
45         }
46     )
47     observables.to_csv(DATA_CLEAN / "chile_observables.csv", index=False)
48
49     # --- Provenance ---
50     meta_path = DATA_CLEAN / "dataset_metadata.json"
51     meta = json.loads(meta_path.read_text(encoding="utf-8"))
52     meta["output_gap_method"] = (
53         "Unobserved-Components Kalman smoother (smooth I(2) trend + AR(2) cycle), "
54         "replacing the HP filter; COVID stays in the cycle"
55     )
56     meta["inflation_centering"] = (
57         "observable pi = quarterly inflation minus the 3% annual target "
58         "(steady state = target; model forecasts converge to 3% by construction)"
59     )
60     meta_path.write_text(json.dumps(meta, indent=2, ensure_ascii=False), encoding="utf-8")
61
62     mean_infl_annual = ((1.0 + panel["infl_q"].mean()) ** 4 - 1.0) * 100.0
63     covid = float(cycle[panel["date"].to_numpy() == "2020Q2"][0]) * 100.0
64     print(f"Kalman output gap: std {cycle.std()*100:.2f}% | COVID 2020Q2 "
65           f"f{covid:.1f}% | last {cycle[-1]*100:.2f}%")
66     print(f"Inflation centred on 3% target (sample mean was {mean_infl_annual:.2f}%); "
67           f"pi observable mean now {observables['pi'].mean()*400:.2f} annual p.p. above target")
68     print("Wrote chile_macro_quarterly.csv, chile_observables.csv, dataset_metadata.json")
69
70
71 if __name__ == "__main__":
72     main()
```

4 python/build_open_economy_dataset.py

```
1  """Build the open-economy extension dataset for Chile.
2
3  Public monthly series are downloaded as CSV from FRED:
4  - CCUSMA02CLM618N: CLP per USD, source OECD Main Economic Indicators.
5  - PCOPPUUSD: global copper price, source IMF Primary Commodity Prices.
6
7  The raw/clean CSV is kept local by .gitignore. Public outputs contain only
8  derived summary statistics, calibration targets, metadata and a figure.
9  """
10
11 from __future__ import annotations
12
13 import datetime as dt
14 import json
15 import urllib.request
16 from io import BytesIO
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import numpy as np
23 import pandas as pd
24 import statsmodels.api as sm
25 from statsmodels.tsa.filters.hp_filter import hpfilter
26
27 from common import DATA_CLEAN, FIGURES, TABLES, ensure_directories
28
29
30 SERIES = {
31     "exchange_rate_clp_per_usd": "CCUSMA02CLM618N",
32     "copper_usd_per_metric_ton": "PCOPPUUSD",
33 }
34 FRED_CSV = "https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}"
35
36
37 def download_fred(series_id: str) -> pd.Series:
38     request = urllib.request.Request(
39         FRED_CSV.format(series_id=series_id),
40         headers={"User-Agent": "nk-monetary-policy-chile/1.0"},
41     )
42     with urllib.request.urlopen(request, timeout=90) as response:
43         payload = response.read()
44         frame = pd.read_csv(BytesIO(payload))
45         frame.columns = ["date", "value"]
46         frame["date"] = pd.to_datetime(frame["date"])
47         frame["value"] = pd.to_numeric(frame["value"], errors="coerce")
48         return frame.dropna().set_index("date")["value"].sort_index()
49
50
51 def ari(series: pd.Series) -> tuple[float, float]:
52     y = series.iloc[1:].to_numpy()
53     x = sm.add_constant(series.iloc[:-1].to_numpy())
54     result = sm.OLS(y, x).fit()
55     residual_std = float(np.std(result.resid, ddof=2))
56     return float(result.params[1]), residual_std
57
58
59 def main() -> None:
60     ensure_directories()
61     fx_monthly = download_fred(SERIES["exchange_rate_clp_per_usd"])
62     copper_monthly = download_fred(SERIES["copper_usd_per_metric_ton"])
63
64     fx_q = fx_monthly.groupby(fx_monthly.index.to_period("Q")).mean()
65     copper_q = copper_monthly.groupby(copper_monthly.index.to_period("Q")).mean()
66     open_panel = pd.DataFrame({"fx_clp_usd": fx_q, "copper_usd_ton": copper_q})
67     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
68     macro.index = pd.PeriodIndex(macro["date"], freq="Q")
69     open_panel = open_panel.join(
70         macro[["output_gap", "infl_q", "i_q", "tpm_annual_pct"]],
71         how="inner",
72     ).dropna()
73
74     open_panel["log_fx"] = np.log(open_panel["fx_clp_usd"])
75     open_panel["log_copper"] = np.log(open_panel["copper_usd_ton"])
76     from common import kalman_gap # Kalman UC cycle, not HP
77     fx_cycle, fx_trend = kalman_gap(open_panel["log_fx"].to_numpy())
78     copper_cycle, copper_trend = kalman_gap(open_panel["log_copper"].to_numpy())
79     open_panel["fx_gap"] = fx_cycle
80     open_panel["fx_trend"] = fx_trend
81     open_panel["copper_gap"] = copper_cycle
82     open_panel["copper_trend"] = copper_trend
83     open_panel["fx_depreciation_q"] = open_panel["log_fx"].diff()
84
85     rho_copper, std_copper = ari(open_panel["copper_gap"])
86     rho_fx, std_fx_gap = ari(open_panel["fx_gap"])
87     std_fx_depreciation = float(open_panel["fx_depreciation_q"].dropna().std(ddof=1))
88
89     calibration = pd.DataFrame(
90         [
91             {
92                 "rho_copper": float(np.clip(rho_copper, 0.20, 0.95)),
93                 "rho_fx_gap_descriptive": rho_fx,
94                 "std_e_copper": float(np.clip(std_copper, 0.01, 0.12)),
95                 "std_e_fx": float(np.clip(std_fx_gap, 0.01, 0.10)),
96                 "std_fx_depreciation_q": std_fx_depreciation,
97                 "alpha_q_is": 0.05,
98                 "alpha_copper_is": 0.03,
99                 "gamma_q_phillips": 0.03,
100                 "eta_i_uip": 1.0,
101                 "phi_q_taylor": 0.05,
102                 "calibration_scope": "illustrative open-economy extension",
```

```

103     }
104 ]
105 )
106 calibration.to_csv(TABLES / "open_economy_calibration.csv", index=False)
107
108 summary_rows = []
109 for column in [
110     "fx_clp_usd",
111     "copper_usd_ton",
112     "fx_gap",
113     "copper_gap",
114     "fx_depreciation_q",
115 ]:
116     values = open_panel[column].dropna()
117     summary_rows.append(
118         {
119             "series": column,
120             "observations": len(values),
121             "mean": float(values.mean()),
122             "std_dev": float(values.std(ddof=1)),
123             "minimum": float(values.min()),
124             "maximum": float(values.max()),
125             "sample_start": str(values.index[0]),
126             "sample_end": str(values.index[-1]),
127         }
128     )
129 pd.DataFrame(summary_rows).to_csv(
130     TABLES / "open_economy_data_summary.csv", index=False
131 )
132
133 clean = open_panel.reset_index(names="date")
134 clean["date"] = clean["date"].astype(str)
135 clean.to_csv(DATA_CLEAN / "chile_open_economy.csv", index=False)
136 metadata = {
137     "access_date": dt.date.today().isoformat(),
138     "is_synthetic": False,
139     "frequency": "quarterly averages built from monthly series",
140     "series": {
141         "exchange_rate": {
142             "id": SERIES["exchange_rate_clp_per_usd"],
143             "title": "CLP per USD, average daily rate",
144             "source": "OECD Main Economic Indicators via FRED",
145             "url": "https://fred.stlouisfed.org/series/CCUSMA02CLM618N",
146         },
147         "copper": {
148             "id": SERIES["copper_usd_per_metric_ton"],
149             "title": "Global price of copper, USD per metric ton",
150             "source": "IMF Primary Commodity Prices via FRED",
151             "url": "https://fred.stlouisfed.org/series/PCOPPUUSD",
152         },
153     },
154     "transformations": (
155         "Monthly levels to quarterly averages; natural logs; HP-filter cycles "
156         "with lambda=1600. Calibration uses AR(1) persistence and innovation std."
157     ),
158     "warning": (
159         "Open-economy parameters are illustrative and are not jointly estimated "
160         "structural coefficients."
161     ),
162 }
163 (DATA_CLEAN / "open_economy_metadata.json").write_text(
164     json.dumps(metadata, ensure_ascii=False, indent=2), encoding="utf-8"
165 )
166
167 x = np.arange(len(open_panel))
168 ticks = np.arange(0, len(open_panel), 8)
169 labels = [str(open_panel.index[index]) for index in ticks]
170 fig, axes = plt.subplots(2, 2, figsize=(13, 8), sharex=True)
171 axes[0, 0].plot(x, open_panel["fx_clp_usd"], color="#1E4E79")
172 axes[0, 0].set_title("Cambio nominal: pesos chilenos por US$ 1")
173 axes[0, 0].set_ylabel("CLP/USD")
174 axes[0, 1].plot(x, open_panel["copper_usd_ton"], color="#B45309")
175 axes[0, 1].set_title("Preço global do cobre")
176 axes[0, 1].set_ylabel("US$/tonelada")
177 axes[1, 0].plot(x, 100 * open_panel["fx_gap"], color="#1E4E79")
178 axes[1, 0].axhline(0, color="black", linewidth=0.7)
179 axes[1, 0].set_title("Gap cambial HP (alta = peso depreciado)")
180 axes[1, 0].set_ylabel("% log")
181 axes[1, 1].plot(x, 100 * open_panel["copper_gap"], color="#B45309")
182 axes[1, 1].axhline(0, color="black", linewidth=0.7)
183 axes[1, 1].set_title("Gap do preço do cobre HP")
184 axes[1, 1].set_ylabel("% log")
185 for ax in axes.ravel():
186     ax.grid(alpha=0.25)
187     ax.set_xticks(ticks)
188     ax.set_xticklabels(labels, rotation=70, fontsize=8)
189 fig.suptitle("Chile como economia pequena e aberta: cambio e cobre")
190 fig.text(
191     0.01,
192     0.01,
193     "Fontes: OECD (cambio) e FMI (cobre), distribuidas pelo FRED. "
194     "Gaps HP sao proxies ciclicas, nao valores de equilibrio observados.",
195     fontsize=8,
196 )
197 fig.tight_layout(rect=(0, 0.03, 1, 0.95))
198 fig.savefig(FIGURES / "open_economy_data.png", dpi=180)
199 plt.close(fig)
200 print(calibration.to_string(index=False))
201 print(f"Wrote open-economy data for {len(open_panel)} quarters.")
202
203
204 if __name__ == "__main__":
205     main()

```

5 python/discover_ipc_series.py

```
1 """Discover BCCh IPC core/sub-index series IDs via SieteRestWS SearchSeries.
2
3 Run once credentials are available (env BCCH_USER/BCCH_PASS or
4 data/raw/bcch_credentials.json). Prints every MONTHLY series whose Spanish title
5 mentions the CPI cores we need (sin volatiles, servicios, bienes, energia,
6 alimentos, volatil), so we can lock the exact ``F074.IPC...`` ids for the puller.
7
8 Credentials are never printed.
9 """
10
11 from __future__ import annotations
12
13 import json
14 import urllib.parse
15 import urllib.request
16
17 from build_chile_dataset import BCCH_BASE, get_credentials
18
19 KEYWORDS = (
20     "ipc", "volatil", "volatil", "sin volatiles", "sin volatiles",
21     "servicio", "bien", "energ", "aliment", "subyacente", "nucleo", "nucleo",
22 )
23
24
25 def search(frequency: str, user: str, password: str) -> list[dict]:
26     query = urllib.parse.urlencode(
27         {"user": user, "pass": password, "function": "SearchSeries",
28          "frequency": frequency}
29     )
30     url = f"{BCCH_BASE}?{query}"
31     with urllib.request.urlopen(url, timeout=90) as response:
32         payload = json.loads(response.read().decode("utf-8", "replace"))
33     return payload.get("SeriesInfos") or []
34
35
36 def main() -> None:
37     creds = get_credentials()
38     if not creds:
39         raise SystemExit("No BCCh credentials (set BCCH_USER/BCCH_PASS or "
40                          "data/raw/bcch_credentials.json).")
41     user, password = creds
42     infos = search("MONTHLY", user, password)
43     print(f"MONTHLY series returned: {len(infos)}")
44     hits = []
45     for info in infos:
46         sid = info.get("seriesId", "")
47         title = (info.get("spanishTitle") or info.get("frequencyCode") or "")
48         low = f"{sid} {title}".lower()
49         if sid.startswith("F074.IPC") or ("ipc" in low and
50                                           any(k in low for k in KEYWORDS)):
51             hits.append((sid, title))
52     print(f"IPC-related monthly series: {len(hits)}")
53     for sid, title in sorted(hits):
54         print(f" {sid:32s} | {title}")
55
56
57 if __name__ == "__main__":
58     main()
```

Geracao dos modelos Dynare

6 python/generate_dynare_models.py

```
1  """Generate Dynare sensitivity models from the baseline specification."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import pandas as pd
8
9  from common import (
10     BASELINE,
11     ROOT,
12     SHOCK_STD,
13     TABLES,
14     calibration_rho,
15     complete_parameters,
16     course_benchmark_shock_std,
17     ensure_directories,
18 )
19
20 TEMPLATE = """// Auto-generated scenario: {scenario}
21 var x pi i;
22 varexo e_x e_pi e_i;
23 parameters beta sigma kappa rho_i phi_pi phi_x rstar;
24
25 beta = {beta:.12g};
26 sigma = {sigma:.12g};
27 kappa = {kappa:.12g};
28 rho_i = {rho_i:.12g};
29 phi_pi = {phi_pi:.12g};
30 phi_x = {phi_x:.12g};
31 rstar = {rstar:.12g};
32
33 model(linear);
34 x = x(+1) - (1/sigma)*(i - pi(+1) - rstar) + e_x;
35 pi = beta*pi(+1) + kappa*x + e_pi;
36 i = rho_i*i(-1) + (1-rho_i)*(rstar + phi_pi*pi + phi_x*x) + e_i;
37 end;
38
39 initval;
40 x = 0;
41 pi = 0;
42 i = rstar;
43 end;
44
45 shocks;
46 var e_x; stderr {std_e_x:.12g};
47 var e_pi; stderr {std_e_pi:.12g};
48 var e_i; stderr {std_e_i:.12g};
49 end;
50
51 steady;
52 check;
53 verbatim;
54 export_stability('{scenario}');
55 end;
56 stoch_simul(order=1, irf=20);
57 verbatim;
58 export_results('{scenario}');
59 end;
60 """
61
62
63 def scenario_name(prefix: str, value: float, digits: int = 2) -> str:
64     text = f"{value:.{digits}f}".replace(".", "p")
65     return f"{prefix}_{text}"
66
67
68 def main() -> None:
69     ensure_directories()
70     generated = ROOT / "dynare" / "generated"
71     generated.mkdir(parents=True, exist_ok=True)
72
73     rho_i = calibration_rho()
74     baseline_shocks = {
75         "std_e_x": SHOCK_STD["e_x"],
76         "std_e_pi": SHOCK_STD["e_pi"],
77         "std_e_i": SHOCK_STD["e_i"],
78     }
79
80     # Regenerate the baseline .mod so it stays consistent with the estimated rho_i
81     # and the current shock calibration (single source of truth for the baseline).
82     baseline_params = complete_parameters({"rho_i": rho_i})
83     (ROOT / "dynare" / "nk_chile_base.mod").write_text(
84         TEMPLATE.format(
85             scenario="baseline",
86             **baseline_shocks,
87             **baseline_params,
88         ),
89         encoding="ascii",
90     )
91
92     scenarios: list[dict] = []
93     for annual in (0.02, 0.03, 0.04):
94         params = complete_parameters({"rstar_annual": annual, "rho_i": rho_i})
95         scenarios.append(
96             {
97                 "scenario": scenario_name("rstar", annual * 100, 0),
98                 "scenario_type": "rstar",
```

```

99         **baseline_shocks,
100         **params,
101     }
102 )
103 for kappa in (0.07, 0.10, 0.13):
104     params = complete_parameters({"kappa": kappa, "rho_i": rho_i})
105     scenarios.append(
106         {
107             "scenario": scenario_name("kappa", kappa, 2),
108             "scenario_type": "kappa",
109             **baseline_shocks,
110             **params,
111         }
112     )
113 for phi_pi in [round(value, 1) for value in
114                [1.3 + 0.1 * index for index in range(10)]]:
115     params = complete_parameters({"phi_pi": phi_pi, "rho_i": rho_i})
116     scenarios.append(
117         {
118             "scenario": scenario_name("phi_pi", phi_pi, 1),
119             "scenario_type": "phi_pi",
120             **baseline_shocks,
121             **params,
122         }
123     )
124
125 # "Calibrate or estimate rho_i": retain the empirical baseline and add the
126 # lecture-style calibrated route as a clean one-parameter robustness case.
127 scenarios.append(
128     {
129         "scenario": "rho_calibrated_0p80",
130         "scenario_type": "rho_i",
131         **baseline_shocks,
132         **complete_parameters({"rho_i": BASELINE["rho_i"]}),
133     }
134 )
135
136 # Alternative shock calibration fitted to the didactic FEVD benchmark,
137 # holding all structural parameters at the Chilean empirical baseline.
138 benchmark_shocks = course_benchmark_shock_std()
139 scenarios.append(
140     {
141         "scenario": "fevd_didactic_benchmark",
142         "scenario_type": "fevd",
143         "std_e_x": benchmark_shocks["e_x"],
144         "std_e_pi": benchmark_shocks["e_pi"],
145         "std_e_i": benchmark_shocks["e_i"],
146         **baseline_params,
147     }
148 )
149
150 for row in scenarios:
151     content = TEMPLATE.format(**row)
152     (generated / f"{row['scenario']}.mod").write_text(
153         content, encoding="ascii"
154     )
155
156 expected_models = {f"{row['scenario']}.mod" for row in scenarios}
157 for old_model in generated.glob("*.mod"):
158     if old_model.name not in expected_models:
159         try:
160             old_model.unlink()
161         except PermissionError:
162             print(
163                 f"Warning: OneDrive locked obsolete model {old_model.name}; "
164                 "remove it manually before publishing."
165             )
166
167 manifest_columns = [
168     "scenario",
169     "scenario_type",
170     "rstar_annual",
171     "rstar",
172     "beta",
173     "sigma",
174     "kappa",
175     "rho_i",
176     "phi_pi",
177     "phi_x",
178     "std_e_x",
179     "std_e_pi",
180     "std_e_i",
181 ]
182 pd.DataFrame(scenarios)[manifest_columns].to_csv(
183     TABLES / "scenario_manifest.csv", index=False
184 )
185 print(f"Generated {len(scenarios)} models in {generated}")
186
187 if __name__ == "__main__":
188     main()

```


7 python/generate_macro_extension_models.py

```
1 """Generate Dynare files for the open-economy and hybrid-NKPC extensions."""
2
3 from __future__ import annotations
4
5 import pandas as pd
6
7 from common import ROOT, SHOCK_STD, TABLES, calibration_rho, complete_parameters
8
9
10 OPEN_TEMPLATE = """// Illustrative small-open-economy extension for Chile.
11 var x pi i q copper;
12 varexo e_x e_pi e_i e_q e_copper;
13 parameters beta sigma kappa rho_i phi_pi phi_x alpha_q alpha_copper
14           gamma_q eta_i phi_q rho_copper;
15
16 beta = {beta:.12g};
17 sigma = {sigma:.12g};
18 kappa = {kappa:.12g};
19 rho_i = {rho_i:.12g};
20 phi_pi = {phi_pi:.12g};
21 phi_x = {phi_x:.12g};
22 alpha_q = {alpha_q:.12g};
23 alpha_copper = {alpha_copper:.12g};
24 gamma_q = {gamma_q:.12g};
25 eta_i = {eta_i:.12g};
26 phi_q = {phi_q:.12g};
27 rho_copper = {rho_copper:.12g};
28
29 model(linear);
30 // Domestic demand: real rate, competitiveness and copper-income channel.
31 x = x(+1) - (1/sigma)*(i-pi(+1)) + alpha_q*q + alpha_copper*copper + e_x;
32 // Domestic inflation with direct exchange-rate pass-through.
33 pi = beta*pi(+1) + kappa*x + gamma_q*q + e_pi;
34 // Taylor rule may lean mildly against depreciation.
35 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi + phi_x*x + phi_q*q) + e_i;
36 // UIP-style exchange-rate equation; q>0 means a weaker peso.
37 q = q(+1) - eta_i*i + e_q;
38 // Exogenous persistent copper-price gap.
39 copper = rho_copper*copper(-1) + e_copper;
40 end;
41
42 steady;
43 check;
44 verbatim;
45     export_stability('open_economy');
46 end;
47
48 shocks;
49     var e_x; stderr {std_e_x:.12g};
50     var e_pi; stderr {std_e_pi:.12g};
51     var e_i; stderr {std_e_i:.12g};
52     var e_q; stderr {std_e_q:.12g};
53     var e_copper; stderr {std_e_copper:.12g};
54 end;
55
56 stoch_simul(order=1, irf=20, nograph);
57 verbatim;
58     export_extension('open_economy');
59 end;
60 """
61
62
63 HYBRID_TEMPLATE = """// Hybrid Phillips-curve extension with intrinsic inflation persistence.
64 var x pi i;
65 varexo e_x e_pi e_i;
66 parameters beta sigma kappa rho_i phi_pi phi_x gamma_pi;
67
68 beta = {beta:.12g};
69 sigma = {sigma:.12g};
70 kappa = {kappa:.12g};
71 rho_i = {rho_i:.12g};
72 phi_pi = {phi_pi:.12g};
73 phi_x = {phi_x:.12g};
74 gamma_pi = {gamma_pi:.12g};
75
76 model(linear);
77 x = x(+1) - (1/sigma)*(i-pi(+1)) + e_x;
78 pi = gamma_pi*pi(-1) + (1-gamma_pi)*(beta*pi(+1)+kappa*x) + e_pi;
79 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi+phi_x*x) + e_i;
80 end;
81
82 steady;
83 check;
84 verbatim;
85     export_stability('hybrid_nkpc');
86 end;
87
88 shocks;
89     var e_x; stderr {std_e_x:.12g};
90     var e_pi; stderr {std_e_pi:.12g};
91     var e_i; stderr {std_e_i:.12g};
92 end;
93
94 stoch_simul(order=1, irf=20, nograph);
95 verbatim;
96     export_extension('hybrid_nkpc');
97 end;
98 """
99
100
101 def main() -> None:
102     params = complete_parameters({"rho_i": calibration_rho()})
```

```

103 calibration_path = TABLES / "open_economy_calibration.csv"
104 if calibration_path.exists():
105     open_cal = pd.read_csv(calibration_path).iloc[0].to_dict()
106 else:
107     open_cal = {
108         "rho_copper": 0.75,
109         "std_e_copper": 0.08,
110         "std_e_fx": 0.04,
111         "alpha_q_is": 0.05,
112         "alpha_copper_is": 0.03,
113         "gamma_q_phillips": 0.03,
114         "eta_i_uip": 1.0,
115         "phi_q_taylor": 0.05,
116     }
117 open_values = {
118     **params,
119     "alpha_q": float(open_cal["alpha_q_is"]),
120     "alpha_copper": float(open_cal["alpha_copper_is"]),
121     "gamma_q": float(open_cal["gamma_q_phillips"]),
122     "eta_i": float(open_cal["eta_i_uip"]),
123     "phi_q": float(open_cal["phi_q_taylor"]),
124     "rho_copper": float(open_cal["rho_copper"]),
125     "std_e_x": SHOCK_STD["e_x"],
126     "std_e_pi": SHOCK_STD["e_pi"],
127     "std_e_i": SHOCK_STD["e_i"],
128     "std_e_q": float(open_cal["std_e_fx"]),
129     "std_e_copper": float(open_cal["std_e_copper"]),
130 }
131 (ROOT / "dynare" / "nk_chile_open.mod").write_text(
132     OPEN_TEMPLATE.format(**open_values), encoding="ascii"
133 )
134
135 # Backward OLS estimate is around 0.34; use 0.35 as an illustrative
136 # intrinsic-persistence share, not as a structural estimate of indexation.
137 hybrid_values = {
138     **params,
139     "gamma_pi": 0.35,
140     "std_e_x": SHOCK_STD["e_x"],
141     "std_e_pi": SHOCK_STD["e_pi"],
142     "std_e_i": SHOCK_STD["e_i"],
143 }
144 (ROOT / "dynare" / "nk_chile_hybrid.mod").write_text(
145     HYBRID_TEMPLATE.format(**hybrid_values), encoding="ascii"
146 )
147 pd.DataFrame(
148     [
149         {"scenario": "open_economy", **open_values},
150         {"scenario": "hybrid_nkpc", **hybrid_values},
151     ]
152 ).to_csv(TABLES / "macro_extension_calibration.csv", index=False)
153 print("Generated nk_chile_open.mod and nk_chile_hybrid.mod")
154
155
156 if __name__ == "__main__":
157     main()

```

Modelos Dynare (.mod)

8 dynare/nk_chile_base.mod

```
1 // Auto-generated scenario: baseline
2 var x pi i;
3 varexo e_x e_pi e_i;
4 parameters beta sigma kappa rho_i phi_pi phi_x rstar;
5
6 beta = 0.992637536145;
7 sigma = 1;
8 kappa = 0.1;
9 rho_i = 0.934095195259;
10 phi_pi = 1.75;
11 phi_x = 0.5;
12 rstar = 0.00741707177773;
13
14 model(linear);
15 x = x(+1) - (1/sigma)*(i - pi(+1) - rstar) + e_x;
16 pi = beta*pi(+1) + kappa*x + e_pi;
17 i = rho_i*i(-1) + (1-rho_i)*(rstar + phi_pi*pi + phi_x*x) + e_i;
18 end;
19
20 initval;
21 x = 0;
22 pi = 0;
23 i = rstar;
24 end;
25
26 shocks;
27 var e_x; stderr 0.005;
28 var e_pi; stderr 0.00277;
29 var e_i; stderr 0.00068;
30 end;
31
32 steady;
33 check;
34 verbatim;
35 export_stability('baseline');
36 end;
37 stoch_simul(order=1, irf=20);
38 verbatim;
39 export_results('baseline');
40 end;
```

9 dynare/nk_chile_estim.mod

```
1 // Posterior-mode estimation of the minimal New Keynesian model on Chilean data.
2 // Written in pure deviation form (r* drops out), so the steady state is x=pi=i=0 and
3 // matches the demeaned observables (pi, i, x are quarterly fractions, mean-zero).
4 // Three observables and three structural shocks => no stochastic singularity.
5
6 var x pi i;
7 varexo e_x e_pi e_i;
8 parameters beta sigma kappa rho_i phi_pi phi_x;
9
10 beta = 0.992637536145; // 1/(1+r*), r* from the 3% annual scenario
11 sigma = 1.0;
12 kappa = 0.10;
13 rho_i = 0.90;
14 phi_pi = 1.75;
15 phi_x = 0.50;
16
17 model(linear);
18 x = x(+1) - (1/sigma)*(i - pi(+1)) + e_x;
19 pi = beta*pi(+1) + kappa*x + e_pi;
20 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi + phi_x*x) + e_i;
21 end;
22
23 steady;
24
25 varobs pi i x;
26
27 estimated_params;
28 sigma, gamma_pdf, 1.00, 0.40;
29 kappa, gamma_pdf, 0.10, 0.05;
30 rho_i, beta_pdf, 0.80, 0.10;
31 phi_pi, gamma_pdf, 1.50, 0.25;
32 phi_x, gamma_pdf, 0.50, 0.25;
33 stderr e_x, inv_gamma_pdf, 0.005, inf;
34 stderr e_pi, inv_gamma_pdf, 0.0025, inf;
35 stderr e_i, inv_gamma_pdf, 0.002, inf;
36 end;
37
38 // Posterior-mode (MAP) estimation: fast and robust in Octave. Standard errors come from
39 // the mode Hessian (recovered in Python). mh_replic can be raised to add a full MCMC.
40 estimation(datafile='chile_observables_dynare.csv', mode_compute=4,
41           mh_replic=0, plot_priors=0, nograph);
42
43 verbatim;
44 export_bayesian('estimation');
45 end;
```

10 dynare/nk_chile_hybrid.mod

```
1 // Hybrid Phillips-curve extension with intrinsic inflation persistence.
2 var x pi i;
3 varexo e_x e_pi e_i;
4 parameters beta sigma kappa rho_i phi_pi phi_x gamma_pi;
5
6 beta = 0.992637536145;
7 sigma = 1;
8 kappa = 0.1;
9 rho_i = 0.934095195259;
10 phi_pi = 1.75;
11 phi_x = 0.5;
12 gamma_pi = 0.35;
13
14 model(linear);
15   x = x(+1) - (1/sigma)*(i-pi(+1)) + e_x;
16   pi = gamma_pi*pi(-1) + (1-gamma_pi)*(beta*pi(+1)+kappa*x) + e_pi;
17   i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi+phi_x*x) + e_i;
18 end;
19
20 steady;
21 check;
22 verbatim;
23   export_stability('hybrid_nkpc');
24 end;
25
26 shocks;
27   var e_x; stderr 0.005;
28   var e_pi; stderr 0.00277;
29   var e_i; stderr 0.00068;
30 end;
31
32 stoch_simul(order=1, irf=20, nograph);
33 verbatim;
34   export_extension('hybrid_nkpc');
35 end;
```

11 dynare/nk_chile_hybrid_estim.mod

```
1 // Posterior-mode estimation of the hybrid-Phillips Chilean NK model.
2 // This model uses the same observables and common priors as nk_chile_estim.mod,
3 // adding one estimated indexation parameter gamma_pi. The matched data sample
4 // makes its marginal likelihood comparable with the forward-looking baseline.
5
6 var x pi i;
7 varexo e_x e_pi e_i;
8 parameters beta sigma kappa rho_i phi_pi phi_x gamma_pi;
9
10 beta      = 0.992637536145;
11 sigma     = 1.0;
12 kappa     = 0.10;
13 rho_i     = 0.90;
14 phi_pi    = 1.75;
15 phi_x     = 0.50;
16 gamma_pi  = 0.35;
17
18 model(linear);
19   x = x(+1) - (1/sigma)*(i-pi(+1)) + e_x;
20   pi = gamma_pi*pi(-1)
21       + (1-gamma_pi)*(beta*pi(+1)+kappa*x) + e_pi;
22   i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi+phi_x*x) + e_i;
23 end;
24
25 steady;
26 check;
27 varobs pi i x;
28
29 estimated_params;
30   sigma,      gamma_pdf,    1.00, 0.40;
31   kappa,      gamma_pdf,    0.10, 0.05;
32   rho_i,      beta_pdf,     0.80, 0.10;
33   phi_pi,     gamma_pdf,    1.50, 0.25;
34   phi_x,      gamma_pdf,    0.50, 0.25;
35   gamma_pi,   beta_pdf,     0.35, 0.15;
36   stderr e_x, inv_gamma_pdf, 0.005, inf;
37   stderr e_pi, inv_gamma_pdf, 0.0025, inf;
38   stderr e_i, inv_gamma_pdf, 0.002, inf;
39 end;
40
41 estimation(datafile='chile_observables_dynare.csv', mode_compute=4,
42           mh_replic=0, plot_priors=0, nograph, nodisplay);
```

12 dynare/nk_chile_forecast.mod

```

1 // Native Dynare forecast for the calibrated Chilean New Keynesian model.
2 // Implements the "projections from observed data" workflow: read quarterly
3 // observables, keep the calibration fixed (no Bayesian estimation), and project
4 // pi, i and x eight quarters ahead, both unconditionally and under controlled
5 // interest-rate and inflation paths.
6 //
7 // The model is written in pure deviation form (r* drops out), so the steady
8 // state is x=pi=i=0 and matches the demeaned observables. Three observables and
9 // three structural shocks => no stochastic singularity.
10
11 var x pi i;
12 varexo e_x e_pi e_i;
13 parameters beta sigma kappa rho_i phi_pi phi_x;
14
15 // Baseline calibration (same values used by the stoch_simul baseline):
16 beta = 0.992637536145; // 1/(1+r*), r* from the 3% annual scenario
17 sigma = 1.0; // log-utility benchmark
18 kappa = 0.10; // Phillips-curve slope
19 rho_i = 0.934095195259; // interest-rate smoothing, AR(1) of the quarterly TPM
20 phi_pi = 1.75; // inflation response (Taylor principle satisfied)
21 phi_x = 0.50; // output-gap response
22
23 model(linear);
24 // IS (expectational), deviation form
25 x = x(+1) - (1/sigma)*(i - pi(+1)) + e_x;
26 // New Keynesian Phillips curve
27 pi = beta*pi(+1) + kappa*x + e_pi;
28 // Taylor rule with interest-rate smoothing
29 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi + phi_x*x) + e_i;
30 end;
31
32 steady;
33 check;
34
35 // Calibrated shock sizes (same as the baseline stoch_simul):
36 shocks;
37 var e_x; stderr 0.005;
38 var e_pi; stderr 0.00277;
39 var e_i; stderr 0.00068;
40 end;
41
42 varobs pi i x;
43
44 // The three shock standard deviations are declared so the estimation machinery
45 // has something to "estimate", but mode_compute=0 together with
46 // estimated_params_init(use_calibration) means nothing is optimised: every
47 // parameter stays at its calibrated value. The priors below are never used.
48 estimated_params;
49 stderr e_x, inv_gamma_pdf, 0.005, inf;
50 stderr e_pi, inv_gamma_pdf, 0.00277, inf;
51 stderr e_i, inv_gamma_pdf, 0.00068, inf;
52 end;
53
54 estimated_params_init(use_calibration);
55 stderr e_x, 0.005;
56 stderr e_pi, 0.00277;
57 stderr e_i, 0.00068;
58 end;
59
60 // Kalman smoother over the history + 8-quarter unconditional forecast, at the
61 // fixed calibration. forecast=8 fills oo_.forecast.Mean / HPDinf / HPDsup.
62 estimation(datafile='chile_observables_dynare.csv', mode_compute=0, mh_replic=0,
63 smoother, forecast=8, nograph, nodisplay) x pi i;
64
65 verbatim;
66 export_forecast('forecast');
67 end;
68
69 // -----
70 // Conditional projections. The controlled exogenous variable is the monetary
71 // policy shock e_i, which is solved for so that the constrained path is hit.
72 // Values are in model (deviation) units; the steady state is zero.
73 // -----
74
75 // Scenario 1: hold the policy-rate gap at its last observed level for 2 quarters.
76 conditional_forecast_paths;
77 var i;
78 periods 1 2;
79 values 0.000988 0.000988;
80 end;
81 conditional_forecast(parameter_set=calibration, controlled_varexo=(e_i),
82 periods=8, replic=2000);
83 verbatim;
84 export_conditional('cond_hold_2q');
85 end;
86
87 // Scenario 2: tighten and hold the policy-rate gap -1 p.p. (annual) above its
88 // historical mean for 4 quarters.
89 conditional_forecast_paths;
90 var i;
91 periods 1 2 3 4;
92 values 0.0025 0.0025 0.0025 0.0025;
93 end;
94 conditional_forecast(parameter_set=calibration, controlled_varexo=(e_i),
95 periods=8, replic=2000);
96 verbatim;
97 export_conditional('cond_hike_4q');
98 end;
99
100 // Scenario 3: force inflation to the 3% official target for 4 quarters.
101 // The steady state corresponds to the sample-mean inflation (~3.79% annual), so
102 // the 3% target maps to a NEGATIVE inflation gap of (1.03)^(1/4)-1 minus the

```

```
103 // mean quarterly inflation = -0.0019233. Imposing this gap is a genuine
104 // disinflation experiment (not vacuous), enforced through the policy shock e_i.
105 conditional_forecast_paths;
106   var pi;
107   periods 1 2 3 4;
108   values -0.0019233 -0.0019233 -0.0019233 -0.0019233;
109 end;
110 conditional_forecast(parameter_set=calibration, controlled_varexo=(e_i),
111                     periods=8, replic=2000);
112 verbatim;
113   export_conditional('cond_pi_to_target_4q');
114 end;
```

13 dynare/nk_chile_history.mod

```
1 // Historical analysis of the calibrated Chilean New Keynesian model.
2 // Runs the Kalman smoother over 2001Q1-2026Q1 at the fixed calibration and
3 // asks the macro-narrative question: which structural shocks (demand,
4 // cost-push, monetary) actually drove inflation, the output gap and the policy
5 // rate over the sample? Uses Dynare's shock_decomposition, plus the smoothed
6 // structural shock series. Same deviation-form model as the forecast file.
7
8 var x pi i;
9 varexo e_x e_pi e_i;
10 parameters beta sigma kappa rho_i phi_pi phi_x;
11
12 beta = 0.992637536145;
13 sigma = 1.0;
14 kappa = 0.10;
15 rho_i = 0.934095195259;
16 phi_pi = 1.75;
17 phi_x = 0.50;
18
19 model(linear);
20 x = x(+1) - (1/sigma)*(i - pi(+1)) + e_x;
21 pi = beta*pi(+1) + kappa*x + e_pi;
22 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi + phi_x*x) + e_i;
23 end;
24
25 steady;
26 check;
27
28 shocks;
29 var e_x; stderr 0.005;
30 var e_pi; stderr 0.00277;
31 var e_i; stderr 0.00068;
32 end;
33
34 varobs pi i x;
35
36 estimated_params;
37 stderr e_x, inv_gamma_pdf, 0.005, inf;
38 stderr e_pi, inv_gamma_pdf, 0.00277, inf;
39 stderr e_i, inv_gamma_pdf, 0.00068, inf;
40 end;
41
42 estimated_params_init(use_calibration);
43 stderr e_x, 0.005;
44 stderr e_pi, 0.00277;
45 stderr e_i, 0.00068;
46 end;
47
48 // Kalman smoother at the fixed calibration (no estimation).
49 estimation(datafile='chile_observables_dynare.csv', mode_compute=0, mh_replic=0,
50 smoother, nograph, nodisplay) x pi i;
51
52 // Historical decomposition: contribution of each shock to each variable.
53 shock_decomposition(parameter_set=calibration, nograph) pi i x;
54
55 verbatim;
56 export_history('history');
57 end;
```

14 dynare/nk_chile_history_hybrid.mod

```
1 // Historical shock decomposition for the HYBRID-NKPC model (intrinsic inflation
2 // persistence). Same smoother + shock_decomposition exercise as nk_chile_history.mod,
3 // but with the indexation term gamma_pi*pi(-1). Used to ask whether the story of
4 // Chilean IPC inflation changes once inflation has memory.
5
6 var x pi i;
7 varexo e_x e_pi e_i;
8 parameters beta sigma kappa rho_i phi_pi phi_x gamma_pi;
9
10 beta      = 0.992637536145;
11 sigma     = 1.0;
12 kappa     = 0.10;
13 rho_i     = 0.934095195259;
14 phi_pi    = 1.75;
15 phi_x     = 0.50;
16 gamma_pi  = 0.35;
17
18 model(linear);
19   x = x(+1) - (1/sigma)*(i - pi(+1)) + e_x;
20   pi = gamma_pi*pi(-1) + (1-gamma_pi)*(beta*pi(+1) + kappa*x) + e_pi;
21   i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi + phi_x*x) + e_i;
22 end;
23
24 steady;
25 check;
26
27 shocks;
28   var e_x;   stderr 0.005;
29   var e_pi;  stderr 0.00277;
30   var e_i;   stderr 0.00068;
31 end;
32
33 varobs pi i x;
34
35 estimated_params;
36   stderr e_x,   inv_gamma_pdf, 0.005,   inf;
37   stderr e_pi,  inv_gamma_pdf, 0.00277, inf;
38   stderr e_i,   inv_gamma_pdf, 0.00068, inf;
39 end;
40
41 estimated_params_init(use_calibration);
42   stderr e_x,   0.005;
43   stderr e_pi,  0.00277;
44   stderr e_i,   0.00068;
45 end;
46
47 estimation(datafile='chile_observables_dynare.csv', mode_compute=0, mh_replic=0,
48             smoother, nograph, nodisplay) x pi i;
49
50 shock_decomposition(parameter_set=calibration, nograph) pi i x;
51
52 verbatim;
53   export_history('history_hybrid');
54 end;
```

15 dynare/nk_chile_mcmc.mod

```
1 // Full Bayesian MCMC extension of the minimal Chilean NK model.
2 var x pi i;
3 varexo e_x e_pi e_i;
4 parameters beta sigma kappa rho_i phi_pi phi_x;
5
6 beta = 0.992637536145;
7 sigma = 1.0;
8 kappa = 0.10;
9 rho_i = 0.90;
10 phi_pi = 1.75;
11 phi_x = 0.50;
12
13 model(linear);
14 x = x(+1) - (1/sigma)*(i-pi(+1)) + e_x;
15 pi = beta*pi(+1) + kappa*x + e_pi;
16 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi+phi_x*x) + e_i;
17 end;
18
19 steady;
20 check;
21 varobs pi i x;
22
23 estimated_params;
24 sigma, gamma_pdf, 1.00, 0.40;
25 kappa, gamma_pdf, 0.10, 0.05;
26 rho_i, beta_pdf, 0.80, 0.10;
27 phi_pi, gamma_pdf, 1.50, 0.25;
28 phi_x, gamma_pdf, 0.50, 0.25;
29 stderr e_x, inv_gamma_pdf, 0.005, inf;
30 stderr e_pi, inv_gamma_pdf, 0.0025, inf;
31 stderr e_i, inv_gamma_pdf, 0.002, inf;
32 end;
33
34 // Two independent chains. Ten thousand retained proposals per chain are enough
35 // for a serious course extension, but convergence is assessed rather than assumed.
36 estimation(datafile='chile_observables_dynare.csv', mode_compute=4,
37             mh_replic=10000, mh_nblocks=2, mh_drop=0.30, mh_jscale=0.30,
38             mh_conf_sig=0.90, plot_priors=0, nograph, nodisplay);
39
40 verbatim;
41 export_mcmc('mcmc');
42 end;
```

16 dynare/nk_chile_open.mod

```
1 // Illustrative small-open-economy extension for Chile.
2 var x pi i q copper;
3 varexo e_x e_pi e_i e_q e_copper;
4 parameters beta sigma kappa rho_i phi_pi phi_x alpha_q alpha_copper
5     gamma_q eta_i phi_q rho_copper;
6
7 beta = 0.992637536145;
8 sigma = 1;
9 kappa = 0.1;
10 rho_i = 0.934095195259;
11 phi_pi = 1.75;
12 phi_x = 0.5;
13 alpha_q = 0.05;
14 alpha_copper = 0.03;
15 gamma_q = 0.03;
16 eta_i = 1;
17 phi_q = 0.05;
18 rho_copper = 0.941182126643;
19
20 model(linear);
21 // Domestic demand: real rate, competitiveness and copper-income channel.
22 x = x(+1) - (1/sigma)*(i-pi(+1)) + alpha_q*q + alpha_copper*copper + e_x;
23 // Domestic inflation with direct exchange-rate pass-through.
24 pi = beta*pi(+1) + kappa*x + gamma_q*q + e_pi;
25 // Taylor rule may lean mildly against depreciation.
26 i = rho_i*i(-1) + (1-rho_i)*(phi_pi*pi + phi_x*x + phi_q*q) + e_i;
27 // UIP-style exchange-rate equation; q>0 means a weaker peso.
28 q = q(+1) - eta_i*i + e_q;
29 // Exogenous persistent copper-price gap.
30 copper = rho_copper*copper(-1) + e_copper;
31 end;
32
33 steady;
34 check;
35 verbatim;
36     export_stability('open_economy');
37 end;
38
39 shocks;
40     var e_x; stderr 0.005;
41     var e_pi; stderr 0.00277;
42     var e_i; stderr 0.00068;
43     var e_q; stderr 0.0460980810142;
44     var e_copper; stderr 0.12;
45 end;
46
47 stoch_simul(order=1, irf=20, nograph);
48 verbatim;
49     export_extension('open_economy');
50 end;
```

Solucao e calibracao

17 python/hybrid_solution.py

```
1  """Exact first-order reduced form of the hybrid-NKPC Chilean model.
2
3  The hybrid model adds intrinsic inflation persistence:
4
5      x  = E x(+1) - (1/sigma)(i - E pi(+1)) + e_x
6      pi = gamma*pi(-1) + (1-gamma)*(beta*E pi(+1) + kappa*x) + e_pi
7      i  = rho*i(-1) + (1-rho)(phi_pi*pi + phi_x*x) + e_i
8
9  Now there are TWO predetermined states, s_t = [i_{t-1}, pi_{t-1}], so the minimal
10 state-variable solution is y_t = T s_{t-1} + R eps_t with T a 3x2 matrix. We
11 solve T by undetermined coefficients (a 6-equation fixed point) and validate the
12 solution against Dynare's hybrid IRFs (it reproduces them to -1e-6).
13 """
14
15 from __future__ import annotations
16
17 import numpy as np
18
19 from common import calibration_rho, complete_parameters
20
21 E1 = np.array([1.0, 0.0]) # picks i_{t-1}
22 E2 = np.array([0.0, 1.0]) # picks pi_{t-1}
23 I2 = np.eye(2)
24
25 def hybrid_params(gamma_pi: float = 0.35) -> dict:
26     params = complete_parameters({"rho_i": calibration_rho()})
27     params["gamma_pi"] = float(gamma_pi)
28     return params
29
30
31 def solve_hybrid(params: dict, max_iter: int = 5000, tol: float = 1e-14) -> dict:
32     """Unique stable MSV solution by contraction on the state transition A.
33
34     Given a guess for the state transition A (s_t = A s_{t-1}, s=[i,pi]), the
35     structural equations are LINEAR in T=[Tx;Tpi;Ti] (because the expectations
36     become E_t y_{t+1}=T A s_{t-1}). We solve that linear system for T, update
37     A=[Ti;Tpi], and iterate. For a determinate model this converges to the unique
38     forward-stable solution (avoiding the spurious roots a blind root-finder hits).
39     """
40
41     beta, sigma, kappa = params["beta"], params["sigma"], params["kappa"]
42     rho, phi_pi, phi_x = params["rho_i"], params["phi_pi"], params["phi_x"]
43     g = params["gamma_pi"]
44
45     def residual(v: np.ndarray, A: np.ndarray) -> np.ndarray:
46         Tx, Tpi, Ti = v[0:2], v[2:4], v[4:6]
47         r_is = Tx @ (I2 - A) + (1.0 / sigma) * Ti - (1.0 / sigma) * (Tpi @ A)
48         r_pc = Tpi - (1.0 - g) * beta * (Tpi @ A) - (1.0 - g) * kappa * Tx - g * E2
49         r_tr = Ti - (1.0 - rho) * phi_pi * Tpi - (1.0 - rho) * phi_x * Tx - rho * E1
50         return np.concatenate([r_is, r_pc, r_tr])
51
52     def solve_T(A: np.ndarray) -> np.ndarray:
53         # residual is affine in v: L v + c = 0 -> v = solve(L, -c)
54         c = residual(np.zeros(6), A)
55         L = np.zeros((6, 6))
56         for k in range(6):
57             ek = np.zeros(6)
58             ek[k] = 1.0
59             L[:, k] = residual(ek, A) - c
60         return np.linalg.solve(L, -c)
61
62     A = np.zeros((2, 2))
63     v = np.zeros(6)
64     for _ in range(max_iter):
65         v = solve_T(A)
66         A_new = np.vstack([v[4:6], v[2:4]]) # [Ti; Tpi]
67         if np.max(np.abs(A_new - A)) < tol:
68             A = A_new
69             break
70     A = A_new
71
72     Tx, Tpi, Ti = v[0:2], v[2:4], v[4:6]
73     S = np.vstack([Ti, Tpi])
74     T = np.vstack([Tx, Tpi, Ti]) # rows x,pi,i; cols [i_{-1}, pi_{-1}]
75
76     M = np.array([
77         [1.0, -Tx[1] - Tpi[1] / sigma, -Tx[0] + 1.0 / sigma - Tpi[0] / sigma],
78         [-(1.0 - g) * kappa, 1.0 - (1.0 - g) * beta * Tpi[1], -(1.0 - g) * beta * Tpi[0]],
79         [-(1.0 - rho) * phi_x, -(1.0 - rho) * phi_pi, 1.0],
80     ])
81     R = np.linalg.inv(M)
82     return {"T": T, "S": S, "R": R, "eig": np.linalg.eigvals(S),
83             "stable": bool(np.max(np.abs(np.linalg.eigvals(S))) < 1.0)}
84
85
86 def hybrid_irf(sol: dict, shock_std: dict, horizon: int = 20) -> dict:
87     T, S, R = sol["T"], sol["S"], sol["R"]
88     out = {}
89     for j, shock in enumerate(("e_x", "e_pi", "e_i")):
90         e = np.zeros(3)
91         e[j] = shock_std[shock]
92         y0 = R @ e
93         responses = [y0]
94         s = np.array([y0[2], y0[1]]) # [i_0, pi_0]
95         for _ in range(1, horizon):
96             yk = T @ s
97             responses.append(yk)
98             s = np.array([yk[2], yk[1]])
```

```

99         out[shock] = np.array(responses)
100     return out
101
102
103 def hybrid_forecast(sol: dict, last_i_dev: float, last_pi_dev: float,
104                    horizon: int) -> np.ndarray:
105     """h-step forecast of [x, pi, i] deviations from the origin state."""
106     s = np.array([last_i_dev, last_pi_dev])
107     Sp = np.linalg.matrix_power(sol["S"], horizon - 1)
108     return sol["T"] @ Sp @ s
109
110
111 def _validate() -> None:
112     import pandas as pd
113
114     from common import DYNARE_OUTPUTS
115
116     params = hybrid_params(0.35)
117     sol = solve_hybrid(params)
118     print(f"Stable: {sol['stable']}; |eig(S)| = {np.abs(sol['eig']).round(4)}")
119
120     shock_std = {"e_x": 0.005, "e_pi": 0.00277, "e_i": 0.00068}
121     mine = hybrid_irf(sol, shock_std, horizon=20)
122
123     dyn = pd.read_csv(DYNARE_OUTPUTS / "hybrid_nkpc" / "irfs.csv")
124     var_index = {"x": 0, "pi": 1, "i": 2}
125     max_diff = 0.0
126     for shock in ("e_x", "e_pi", "e_i"):
127         for var, vi in var_index.items():
128             col = f"{var}_{shock}"
129             if col not in dyn:
130                 continue
131             diff = np.max(np.abs(mine[shock][:, vi] - dyn[col].to_numpy()[:20]))
132             max_diff = max(max_diff, diff)
133     print(f"Max |IRF difference| vs Dynare hybrid: {max_diff:.3e}")
134     print("VALIDATION OK" if max_diff < 1e-5 else "VALIDATION FAILED")
135
136
137 if __name__ == "__main__":
138     _validate()

```

18 python/calibrate_shocks.py

```

1  """Calibrate shock standard deviations to two transparent FEVD targets.
2
3  The assignment asks for shock sizes that generate plausible impulse responses and
4  moments. We make that choice transparent by specifying an illustrative variance-share
5  target: inflation is mostly driven by cost-push shocks, the policy rate mostly by its
6  own innovation, and the output gap is split between demand and monetary shocks.
7
8  FEVD shares depend only on the products  $\sigma_k^2 * V_k(v)$ , where  $V_k(v)$  is the asymptotic
9  variance of variable  $v$  generated by a unit-variance shock  $k$ . We compute  $V_k(v)$  from the
10 exact linear-RE solution and then search the shock sizes (anchoring  $\sigma_{ex} = 0.005$ ) that
11 best match the target shares at the estimated Chilean calibration.
12
13 The first target is the didactic benchmark stated in the assignment material. The
14 second is the researcher's alternative target originally used in this repository.
15 Neither is a historical decomposition of Chile or an official BCCh estimate.
16 """
17
18 from __future__ import annotations
19
20 import numpy as np
21 import pandas as pd
22 import matplotlib
23
24 matplotlib.use("Agg")
25 import matplotlib.pyplot as plt
26 from scipy.optimize import least_squares
27
28 from common import (
29     FIGURES,
30     INITIAL_SHOCK_STD,
31     TABLES,
32     _state_coefficients,
33     calibration_rho,
34     complete_parameters,
35     ensure_directories,
36 )
37
38 VARS = ("x", "pi", "i")
39 SHOCKS = ("e_x", "e_pi", "e_i")
40
41 TARGETS = {
42     "didactic_benchmark": {
43         "x": {"e_x": 0.375, "e_pi": 0.100, "e_i": 0.524},
44         "pi": {"e_x": 0.014, "e_pi": 0.823, "e_i": 0.162},
45         "i": {"e_x": 0.087, "e_pi": 0.147, "e_i": 0.766},
46     },
47     "researcher_alternative": {
48         "x": {"e_x": 0.45, "e_pi": 0.10, "e_i": 0.45},
49         "pi": {"e_x": 0.02, "e_pi": 0.80, "e_i": 0.18},
50         "i": {"e_x": 0.10, "e_pi": 0.15, "e_i": 0.75},
51     },
52 }
53
54
55 def per_shock_variance(params: dict, horizon: int = 4000) -> np.ndarray:
56     """ $V[v, k]$  = asymptotic variance of variable  $v$  from a unit-variance shock  $k$ ."""
57     a_x, a_pi, a_i = _state_coefficients(params)
58     sigma, beta, kappa = params["sigma"], params["beta"], params["kappa"]
59     rho_i, phi_pi, phi_x = params["rho_i"], params["phi_pi"], params["phi_x"]
60     c_i = a_x - (1.0 - a_pi) / sigma
61     impact_matrix = np.array(
62         [
63             [1.0, 0.0, -c_i],
64             [-kappa, 1.0, -beta * a_pi],
65             [-(1.0 - rho_i) * phi_x, -(1.0 - rho_i) * phi_pi, 1.0],
66         ]
67     )
68     propagation = np.array([a_x, a_pi, a_i])
69     variance = np.zeros((3, 3))
70     for k in range(3):
71         innovation = np.zeros(3)
72         innovation[k] = 1.0
73         response = np.linalg.solve(impact_matrix, innovation)
74         accum = response ** 2
75         lag = response[2]
76         for _ in range(1, horizon):
77             response = propagation * lag
78             accum = accum + response ** 2
79             lag = response[2]
80         variance[:, k] = accum
81     return variance
82
83
84 def fevd_from_sigmas(variance: np.ndarray, sigmas: np.ndarray) -> np.ndarray:
85     contributions = variance * (sigmas ** 2) # column k scaled by  $\sigma_k^2$ 
86     return contributions / contributions.sum(axis=1, keepdims=True)
87
88
89 def calibrate_to_target(
90     variance: np.ndarray,
91     initial_sigmas: np.ndarray,
92     target: np.ndarray,
93 ) -> tuple[np.ndarray, np.ndarray, float]:
94     """Fit  $e_{pi}$  and  $e_i$  standard deviations while anchoring  $\sigma_{ex}$ ."""
95
96     target = target / target.sum(axis=1, keepdims=True)
97
98     def residuals(free_log: np.ndarray) -> np.ndarray:
99         sigmas = np.array(
100             [initial_sigmas[0], np.exp(free_log[0]), np.exp(free_log[1])]
101         )
102         return (fevd_from_sigmas(variance, sigmas) - target).ravel()

```

```

103
104 start = np.log([initial_sigmas[1], initial_sigmas[2]])
105 solution = least_squares(residuals, start, method="lm")
106 calibrated = np.array(
107     [initial_sigmas[0], np.exp(solution.x[0]), np.exp(solution.x[1])]
108 )
109 achieved = fevd_from_sigmas(variance, calibrated)
110 rmse_pp = 100.0 * float(np.sqrt(np.mean((achieved - target) ** 2)))
111 return calibrated, achieved, rmse_pp
112
113
114 def plot_comparison(table: pd.DataFrame) -> None:
115     fig, axes = plt.subplots(2, 3, figsize=(13, 7), sharey=True)
116     labels = {"e_x": "Demand", "e_pi": "Cost-push", "e_i": "Monetary"}
117     for row_index, calibration in enumerate(TARGETS):
118         subset = table[table["calibration"] == calibration]
119         for column_index, variable in enumerate(VARS):
120             ax = axes[row_index, column_index]
121             frame = subset[subset["variable"] == variable].set_index("shock")
122             shocks = list(SHOCKS)
123             positions = np.arange(len(shocks))
124             ax.bar(
125                 positions - 0.18,
126                 frame.loc[shocks, "target_pct"],
127                 width=0.36,
128                 label="Target",
129                 color="#9CA3AF",
130             )
131             ax.bar(
132                 positions + 0.18,
133                 frame.loc[shocks, "achieved_pct"],
134                 width=0.36,
135                 label="Achieved",
136                 color="#1E4E79",
137             )
138             ax.set_xticks(positions, [labels[shock] for shock in shocks], rotation=20)
139             ax.set_title(f"{calibration.replace('_', ' ').title()} - {variable}")
140             ax.grid(axis="y", alpha=0.25)
141             if column_index == 0:
142                 ax.set_ylabel("Variance share (%)")
143             if row_index == 0 and column_index == 2:
144                 ax.legend(fontsize=8)
145     fig.suptitle("FEVD calibration: stated targets versus model-achieved shares")
146     fig.tight_layout(rect=(0, 0, 1, 0.96))
147     fig.savefig(FIGURES / "fevd_calibration_comparison.png", dpi=180)
148     plt.close(fig)
149
150
151 def main() -> None:
152     ensure_directories()
153     params = complete_parameters({"rho_i": calibration_rho()})
154     variance = per_shock_variance(params)
155
156     initial_sigmas = np.array(
157         [INITIAL_SHOCK_STD["e_x"], INITIAL_SHOCK_STD["e_pi"], INITIAL_SHOCK_STD["e_i"]]
158     )
159     fevd_initial = fevd_from_sigmas(variance, initial_sigmas)
160
161     comparison_rows = []
162     sigma_rows = []
163     fitted: dict[str, np.ndarray] = {}
164     for calibration, target_dict in TARGETS.items():
165         raw_target = np.array(
166             [[target_dict[variable][shock] for shock in SHOCKS] for variable in VARS]
167         )
168         target = raw_target / raw_target.sum(axis=1, keepdims=True)
169         calibrated, achieved, rmse_pp = calibrate_to_target(
170             variance, initial_sigmas, target
171         )
172         fitted[calibration] = calibrated
173         for variable_index, variable in enumerate(VARS):
174             for shock_index, shock in enumerate(SHOCKS):
175                 comparison_rows.append(
176                     {
177                         "calibration": calibration,
178                         "variable": variable,
179                         "shock": shock,
180                         "initial_pct": 100.0 * fevd_initial[variable_index, shock_index],
181                         "target_pct": 100.0 * target[variable_index, shock_index],
182                         "achieved_pct": 100.0 * achieved[variable_index, shock_index],
183                         "difference_pp": 100.0
184                         * (
185                             achieved[variable_index, shock_index]
186                             - target[variable_index, shock_index]
187                         ),
188                         "rmse_all_shares_pp": rmse_pp,
189                         "rho_i": params["rho_i"],
190                     }
191                 )
192     for shock, sigma_initial, sigma_calibrated in zip(
193         SHOCKS, initial_sigmas, calibrated
194     ):
195         sigma_rows.append(
196             {
197                 "calibration": calibration,
198                 "shock": shock,
199                 "sigma_initial": sigma_initial,
200                 "sigma_calibrated": sigma_calibrated,
201                 "rho_i": params["rho_i"],
202             }
203         )
204
205     comparison = pd.DataFrame(comparison_rows)
206     sigmas_comparison = pd.DataFrame(sigma_rows)
207     comparison.to_csv(TABLES / "fevd_calibration_comparison.csv", index=False)

```



```

208 sigmas_comparison.to_csv(TABLES / "shock_sigmas_comparison.csv", index=False)
209 plot_comparison(comparison)
210
211 # Preserve the original headline outputs for backwards compatibility. The
212 # baseline remains the researcher alternative; the benchmark is a robustness case.
213 baseline_name = "researcher_alternative"
214 baseline = comparison[comparison["calibration"] == baseline_name].copy()
215 baseline = baseline.rename(
216     columns={
217         "initial_pct": "fevd_initial_sigmas_pct",
218         "target_pct": "illustrative_target_pct",
219         "achieved_pct": "fevd_calibrated_sigmas_pct",
220     }
221 )
222 baseline["calibration_type"] = baseline_name
223 baseline[
224     [
225         "variable",
226         "shock",
227         "fevd_initial_sigmas_pct",
228         "illustrative_target_pct",
229         "fevd_calibrated_sigmas_pct",
230         "calibration_type",
231     ]
232 ].to_csv(TABLES / "shock_calibration.csv", index=False)
233 pd.DataFrame(
234     {
235         "shock": SHOCKS,
236         "sigma_initial": initial_sigmas,
237         "sigma_calibrated": fitted[baseline_name],
238         "calibration_type": baseline_name,
239     }
240 ).to_csv(TABLES / "shock_sigmas.csv", index=False)
241
242 print(f"rho_i (Chile baseline) = {params['rho_i']:.4f}")
243 print(comparison.round(3).to_string(index=False))
244 print("\nShock-standard-deviation comparison:")
245 print(sigmas_comparison.round(6).to_string(index=False))
246 print("Wrote FEVD comparison tables and figure.")
247
248
249 if __name__ == "__main__":
250     main()

```

19 python/analyze_determinacy.py

```

1  """Map Blanchard-Kahn determinacy and convergence speed against phi_pi.
2
3  The assignment's phi_pi grid (1.3-2.2) is entirely determinate, so a verbal "phi_pi>1"
4  rule is not enough. Here we sweep phi_pi (including values below 1) and, for each, count
5  the eigenvalues inside the unit circle from the model's characteristic cubic. With one
6  predetermined variable the model is determinate iff exactly one root is stable. We also
7  record the largest stable-root modulus, which governs how fast the economy converges
8  after a shock. This complements (and is cross-checked by) Dynare's per-scenario check.
9
10 Outputs: outputs/tables/determinacy_map.csv and outputs/figures/determinacy_map.png.
11 """
12
13 from __future__ import annotations
14
15 import matplotlib
16
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import numpy as np
20 import pandas as pd
21
22 from common import (
23     FIGURES,
24     TABLES,
25     calibration_rho,
26     characteristic_roots,
27     complete_parameters,
28     ensure_directories,
29 )
30
31
32 def main() -> None:
33     ensure_directories()
34     rho_i = calibration_rho()
35     grid = np.round(np.arange(0.5, 3.0001, 0.05), 2)
36     rows = []
37     for phi_pi in grid:
38         params = complete_parameters({"rho_i": rho_i, "phi_pi": float(phi_pi)})
39         roots = characteristic_roots(params)
40         moduli = np.abs(roots)
41         n_stable = int(np.sum(moduli < 1.0 - 1e-9))
42         stable_moduli = moduli[moduli < 1.0 - 1e-9]
43         dominant = float(stable_moduli.max()) if stable_moduli.size else np.nan
44         rows.append(
45             {
46                 "phi_pi": float(phi_pi),
47                 "n_stable_roots": n_stable,
48                 "n_unstable_roots": int(np.sum(moduli > 1.0 + 1e-9)),
49                 "determinate": n_stable == 1,
50                 "dominant_stable_modulus": dominant,
51             }
52         )
53     table = pd.DataFrame(rows)
54     table.to_csv(TABLES / "determinacy_map.csv", index=False)
55
56     threshold = table.loc[table["determinate"], "phi_pi"].min()
57
58     fig, ax1 = plt.subplots(figsize=(8.8, 5.0))
59     colors = ["#16a34a" if d else "#b91c1c" for d in table["determinate"]]
60     ax1.bar(table["phi_pi"], table["n_stable_roots"], width=0.045, color=colors, alpha=0.65)
61     ax1.axhline(1.0, color="black", lw=0.8, ls=":")
62     ax1.set_xlabel("phi_pi (Taylor-rule inflation response, phi_x = 0.5)")
63     ax1.set_ylabel("Number of stable eigenvalues")
64     ax1.set_ylim(0, 3.2)
65     ax2 = ax1.twinx()
66     ax2.plot(table["phi_pi"], table["dominant_stable_modulus"], color="#1e4e79",
67              lw=2, marker="o", ms=3, label="Dominant stable |eigenvalue|")
68     ax2.set_ylabel("Dominant stable |eigenvalue| (persistence)")
69     ax1.axvline(1.0, color="gray", lw=1, ls="--")
70     ax1.set_title(
71         f"Determinacy and convergence vs phi_pi (rho_i={rho_i:.2f}); "
72         f"determinate for phi_pi >= {threshold:.2f}"
73     )
74     ax2.legend(loc="upper right", fontsize=8)
75     fig.text(0.01, 0.01, "Green = determinate (1 stable root); red = indeterminate/explosive. "
76            "Source: characteristic roots of the calibrated model (cross-checked with Dynare).",
77            fontsize=7)
78     fig.tight_layout(rect=(0, 0.03, 1, 1))
79     fig.savefig(FIGURES / "determinacy_map.png", dpi=180)
80     plt.close(fig)
81
82     print(table.to_string(index=False))
83     print(f"Determinate for phi_pi >= {threshold:.2f}")
84     print(f"Wrote {TABLES / 'determinacy_map.csv'} and {FIGURES / 'determinacy_map.png'}")
85
86
87 if __name__ == "__main__":
88     main()

```

Estimacao dos parametros

20 python/estimate_nkpc.py

```
1  """Estimate the New Keynesian Phillips curve slope kappa (Chile, real data).
2
3  Inflation pi_t and the output gap x_t are quarterly fractions from the BCCh panel.
4  Three estimators of the slope kappa are reported:
5
6  * Backward / accelerationist OLS (HAC): pi_t = c + gamma_b*pi_{t-1} + kappa*x_t.
7  * Forward-looking IV (2SLS): pi_t = c + gamma_f*pi_{t+1} + kappa*x_t,
8    with E_t pi_{t+1} instrumented by lagged inflation, gap and rate.
9  * Restricted (gamma_f = beta = 0.9926): (pi_t - beta*pi_{t+1}) = c + kappa*x_t.
10
11 Output: outputs/tables/nkpc_estimates.csv. The estimates are compared with the
12 calibration grid kappa in {0.07, 0.10, 0.13}.
13 """
14
15 from __future__ import annotations
16
17 import json
18 import math
19
20 import numpy as np
21 import pandas as pd
22
23 from common import DATA_CLEAN, TABLES, complete_parameters, ensure_directories
24
25 KAPPA_GRID = (0.07, 0.10, 0.13)
26
27 def _tsls_hac(y, X, Z, maxlags: int = 4):
28     ZtZ_inv = np.linalg.inv(Z.T @ Z)
29     Pz = Z @ ZtZ_inv @ Z.T
30     XtPzX_inv = np.linalg.inv(X.T @ Pz @ X)
31     beta = XtPzX_inv @ X.T @ Pz @ y
32     resid = y - X @ beta
33     fitted_x = Pz @ X
34     scores = fitted_x * resid[:, None]
35     meat = scores.T @ scores
36     for lag in range(1, min(maxlags, len(scores) - 1) + 1):
37         weight = 1.0 - lag / (maxlags + 1.0)
38         gamma = scores[lag:].T @ scores[:-lag]
39         meat += weight * (gamma + gamma.T)
40     cov = XtPzX_inv @ meat @ XtPzX_inv
41     r2 = 1.0 - float(resid @ resid) / float(((y - y.mean()) ** 2).sum())
42     return beta, np.sqrt(np.diag(cov)), r2
43
44
45 def _first_stage_f(endogenous, instruments) -> float:
46     import statsmodels.api as sm
47
48     result = sm.OLS(endogenous, instruments).fit()
49     restrictions = np.zeros((instruments.shape[1] - 1, instruments.shape[1]))
50     restrictions[:, 1:] = np.eye(instruments.shape[1] - 1)
51     return float(result.f_test(restrictions).fvalue)
52
53
54 def main() -> None:
55     ensure_directories()
56     panel = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
57     metadata = json.loads(
58         (DATA_CLEAN / "dataset_metadata.json").read_text(encoding="utf-8")
59     )
60     is_synthetic = bool(metadata.get("is_synthetic", True))
61     data_source = str(metadata.get("institution", metadata.get("source", "unknown")))
62     base = pd.DataFrame({"pi": panel["infl_q"], "x": panel["output_gap"]})
63     base["pi_lead"] = base["pi"].shift(-1)
64     base["pi_l1"] = base["pi"].shift(1)
65     base["pi_l2"] = base["pi"].shift(2)
66     base["x_l1"] = base["x"].shift(1)
67     base["x_l2"] = base["x"].shift(2)
68
69     beta_disc = complete_parameters()["beta"]
70     rows = []
71
72     import statsmodels.api as sm
73
74     # Backward-looking OLS (HAC).
75     bw = base.dropna(subset=["pi", "pi_l1", "x"]).reset_index(drop=True)
76     Xb = sm.add_constant(np.column_stack([bw["pi_l1"], bw["x"]]))
77     ols = sm.OLS(bw["pi"].to_numpy(), Xb).fit(cov_type="HAC", cov_kws={"maxlags": 4})
78     rows.append(
79         {
80             "method": "backward_OLS_HAC",
81             "gamma": float(ols.params[1]),
82             "kappa": float(ols.params[2]),
83             "se_kappa": float(ols.bse[2]),
84             "r_squared": float(ols.rsquared),
85             "observations": int(len(bw)),
86             "first_stage_F_pi_lead": np.nan,
87             "first_stage_F_x": np.nan,
88             "is_synthetic": is_synthetic,
89             "data_source": data_source,
90         }
91     )
92
93     # Forward-looking IV (instrument pi_{t+1} with lags).
94     fw = base.dropna().reset_index(drop=True)
95     n = len(fw)
96     y = fw["pi"].to_numpy()
97     X = np.column_stack([np.ones(n), fw["pi_lead"], fw["x"]])
```

```

99 Z = np.column_stack(
100     [np.ones(n), fw["pi_11"], fw["pi_12"], fw["x_11"], fw["x_12"]]
101 )
102 beta_iv, se_iv, r2_iv = _tsls_hac(y, X, Z)
103 rows.append(
104     {
105         "method": "forward_IV_2SLS_HAC",
106         "gamma": float(beta_iv[1]),
107         "kappa": float(beta_iv[2]),
108         "se_kappa": float(se_iv[2]),
109         "r_squared": r2_iv,
110         "observations": n,
111         "first_stage_F_pi_lead": _first_stage_f(fw["pi_lead"].to_numpy(), Z),
112         "first_stage_F_x": _first_stage_f(fw["x"].to_numpy(), Z),
113         "is_synthetic": is_synthetic,
114         "data_source": data_source,
115     }
116 )
117
118 # Restricted: impose gamma_f = beta.
119 rs = base.dropna(subset=["pi", "pi_lead", "x"]).reset_index(drop=True)
120 y_r = (rs["pi"] - beta_disc * rs["pi_lead"]).to_numpy()
121 Xr = sm.add_constant(rs["x"].to_numpy())
122 olsr = sm.OLS(y_r, Xr).fit(cov_type="HAC", cov_kwds={"maxlags": 4})
123 rows.append(
124     {
125         "method": f"restricted_gamma_eq_beta({beta_disc:.4f})",
126         "gamma": beta_disc,
127         "kappa": float(olsr.params[1]),
128         "se_kappa": float(olsr.bse[1]),
129         "r_squared": float(olsr.rsquared),
130         "observations": int(len(rs)),
131         "first_stage_F_pi_lead": np.nan,
132         "first_stage_F_x": np.nan,
133         "is_synthetic": is_synthetic,
134         "data_source": data_source,
135     }
136 )
137
138 table = pd.DataFrame(rows)
139 table["kappa_grid_nearest"] = table["kappa"].apply(
140     lambda k: min(KAPPA_GRID, key=lambda g: abs(g - k))
141 )
142 table.to_csv(TABLES / "nkpc_estimates.csv", index=False)
143 print(table.round(4).to_string(index=False))
144 print(f"Wrote {TABLES / 'nkpc_estimates.csv'}")
145
146
147 if __name__ == "__main__":
148     main()

```

21 python/estimate_taylor_rule.py

```
1  """Estimate a reduced-form Taylor rule with interest-rate smoothing (Chile).
2
3  Specification (all variables in quarterly fractions, from the real BCCh panel):
4
5       $i_t = c + a * i_{t-1} + b * \pi_t + d * x_t + \epsilon_t,$ 
6
7  with the structural mapping  $\rho_i = a$ ,  $\phi_\pi = b / (1 - a)$ ,  $\phi_x = d / (1 - a)$ .
8
9  Two estimators are reported:
10 * OLS with Newey-West (HAC) standard errors.
11 * 2SLS/IV instrumenting current inflation and the gap with their own lags (and the
12   lagged rate), which addresses the simultaneity between the policy rate and the
13   contemporaneous inflation/gap it reacts to.
14
15 Structural-parameter standard errors use the delta method. Output:
16 outputs/tables/taylor_rule_estimates.csv.
17 """
18
19 from __future__ import annotations
20
21 import json
22 import math
23
24 import numpy as np
25 import pandas as pd
26
27 from common import DATA_CLEAN, TABLES, ensure_directories
28
29
30 def _delta_ratio(params: np.ndarray, cov: np.ndarray, i_a: int, i_num: int) -> tuple[float, float]:
31     """phi = params[i_num] / (1 - params[i_a]); delta-method standard error."""
32     a = params[i_a]
33     num = params[i_num]
34     phi = num / (1.0 - a)
35     grad = np.zeros(len(params))
36     grad[i_a] = num / (1.0 - a) ** 2
37     grad[i_num] = 1.0 / (1.0 - a)
38     var = float(grad @ cov @ grad)
39     return phi, math.sqrt(var) if var > 0 else math.nan
40
41
42 def _tsls_hac(y: np.ndarray, X: np.ndarray, Z: np.ndarray, maxlags: int = 4):
43     ZtZ_inv = np.linalg.inv(Z.T @ Z)
44     Pz = Z @ ZtZ_inv @ Z.T
45     XtPzX_inv = np.linalg.inv(X.T @ Pz @ X)
46     beta = XtPzX_inv @ X.T @ Pz @ y
47     resid = y - X @ beta
48     fitted_x = Pz @ X
49     scores = fitted_x * resid[:, None]
50     meat = scores.T @ scores
51     for lag in range(1, min(maxlags, len(scores) - 1) + 1):
52         weight = 1.0 - lag / (maxlags + 1.0)
53         gamma = scores[lag:].T @ scores[:-lag]
54         meat += weight * (gamma + gamma.T)
55     cov = XtPzX_inv @ meat @ XtPzX_inv
56     r2 = 1.0 - float(resid @ resid) / float(((y - y.mean()) ** 2).sum())
57     return beta, cov, r2
58
59
60 def _first_stage_f(endogenous: np.ndarray, instruments: np.ndarray) -> float:
61     import statsmodels.api as sm
62
63     result = sm.OLS(endogenous, instruments).fit()
64     restrictions = np.zeros((instruments.shape[1] - 2, instruments.shape[1]))
65     restrictions[:, 2:] = np.eye(instruments.shape[1] - 2)
66     return float(result.f_test(restrictions).fvalue)
67
68
69 def main() -> None:
70     ensure_directories()
71     panel = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
72     metadata = json.loads(
73         (DATA_CLEAN / "dataset_metadata.json").read_text(encoding="utf-8")
74     )
75     is_synthetic = bool(metadata.get("is_synthetic", True))
76     data_source = str(metadata.get("institution", metadata.get("source", "unknown")))
77     base = pd.DataFrame(
78         {
79             "i": panel["i_q"],
80             "pi": panel["infl_q"],
81             "x": panel["output_gap"],
82         }
83     )
84     for col in ("i", "pi", "x"):
85         base[f"{col}_l1"] = base[col].shift(1)
86         base[f"{col}_l2"] = base[col].shift(2)
87     reg = base.dropna().reset_index(drop=True)
88     n = len(reg)
89
90     y = reg["i"].to_numpy()
91     X = np.column_stack([np.ones(n), reg["i_l1"], reg["pi"], reg["x"]]) # c, a, b, d
92     Z = np.column_stack(
93         [np.ones(n), reg["i_l1"], reg["pi_l1"], reg["pi_l2"], reg["x_l1"], reg["x_l2"]]
94     )
95
96     rows = []
97
98     import statsmodels.api as sm
99
100     ols = sm.OLS(y, X).fit(cov_type="HAC", cov_kws={"maxlags": 4})
101     cov_ols = np.asarray(ols.cov_params())
102     phi_pi, se_pp = _delta_ratio(ols.params, cov_ols, 1, 2)
```

```

103 phi_x, se_px = _delta_ratio(ols.params, cov_ols, 1, 3)
104 rows.append(
105     {
106         "method": "OLS_HAC",
107         "c": float(ols.params[0]),
108         "rho_i": float(ols.params[1]),
109         "se_rho_i": float(ols.bse[1]),
110         "phi_pi": phi_pi,
111         "se_phi_pi": se_pp,
112         "phi_x": phi_x,
113         "se_phi_x": se_px,
114         "r_squared": float(ols.rsquared),
115         "observations": n,
116         "first_stage_F_pi": np.nan,
117         "first_stage_F_x": np.nan,
118         "is_synthetic": is_synthetic,
119         "data_source": data_source,
120     }
121 )
122
123 beta_iv, cov_iv, r2_iv = _tsls_hac(y, X, Z)
124 se_iv = np.sqrt(np.diag(cov_iv))
125 phi_pi_iv, se_pp_iv = _delta_ratio(beta_iv, cov_iv, 1, 2)
126 phi_x_iv, se_px_iv = _delta_ratio(beta_iv, cov_iv, 1, 3)
127 rows.append(
128     {
129         "method": "IV_2SLS_HAC",
130         "c": float(beta_iv[0]),
131         "rho_i": float(beta_iv[1]),
132         "se_rho_i": float(se_iv[1]),
133         "phi_pi": phi_pi_iv,
134         "se_phi_pi": se_pp_iv,
135         "phi_x": phi_x_iv,
136         "se_phi_x": se_px_iv,
137         "r_squared": r2_iv,
138         "observations": n,
139         "first_stage_F_pi": _first_stage_f(reg["pi"].to_numpy(), Z),
140         "first_stage_F_x": _first_stage_f(reg["x"].to_numpy(), Z),
141         "is_synthetic": is_synthetic,
142         "data_source": data_source,
143     }
144 )
145
146 table = pd.DataFrame(rows)
147 table.to_csv(TABLES / "taylor_rule_estimates.csv", index=False)
148 print(table.round(4).to_string(index=False))
149 print(f"Wrote {TABLES / 'taylor_rule_estimates.csv'}")
150
151
152 if __name__ == "__main__":
153     main()

```

22 python/estimate_rhoi_chile.py

```
1  """Estimate the AR(1) persistence of the quarterly policy rate and pick rho_i.
2
3  Implements exactly the assignment's step: regress the quarterly policy rate on its
4  own lag,  $i_t = \alpha + \rho_i * i_{t-1} + u_t$ , by OLS with Newey-West (HAC) standard
5  errors. With real BCh data the estimate is used directly in the calibration; with a
6  synthetic series (or an implausible estimate) it falls back to the baseline 0.80.
7  """
8
9  from __future__ import annotations
10
11  import math
12
13  import numpy as np
14  import pandas as pd
15
16  from common import BASELINE, DATA_CLEAN, TABLES, ensure_directories
17
18
19  def main() -> None:
20      ensure_directories()
21      input_path = DATA_CLEAN / "chile_policy_rate.csv"
22      if not input_path.exists():
23          raise FileNotFoundError(
24              "Missing data/clean/chile_policy_rate.csv. "
25              "Run python/build_chile_dataset.py first."
26          )
27
28      frame = pd.read_csv(input_path)
29      required = {"policy_rate", "is_synthetic"}
30      missing = required.difference(frame.columns)
31      if missing:
32          raise ValueError(f"Clean dataset is missing columns: {sorted(missing)}")
33
34      rate = pd.to_numeric(frame["policy_rate"], errors="coerce")
35      valid = rate.dropna()
36      if len(valid) < 11:
37          raise ValueError("At least 11 valid policy-rate observations are required.")
38
39      dates = frame["date"].astype(str) if "date" in frame.columns else None
40      sample_start = dates.iloc[1] if dates is not None else ""
41      sample_end = dates.iloc[-1] if dates is not None else ""
42
43      current = valid.to_numpy()[1:]
44      lagged = valid.to_numpy()[::-1]
45      n_obs = len(current)
46
47      se_hac = math.nan
48      method = "statsmodels_ols_hac"
49      try:
50          import statsmodels.api as sm
51
52          design = sm.add_constant(lagged)
53          ols = sm.OLS(current, design).fit()
54          hac = sm.OLS(current, design).fit(cov_type="HAC", cov_kwds={"maxlags": 4})
55          alpha = float(ols.params[0])
56          rho_estimate = float(ols.params[1])
57          se_ols = float(ols.bse[1])
58          se_hac = float(hac.bse[1])
59          r_squared = float(ols.rsquared)
60      except ImportError:
61          method = "numpy_lstsq_fallback"
62          design = np.column_stack([np.ones(n_obs), lagged])
63          coefficients, _, _, _ = np.linalg.lstsq(design, current, rcond=None)
64          alpha, rho_estimate = map(float, coefficients)
65          residual = current - design @ coefficients
66          sigma2 = float(residual @ residual / (n_obs - 2))
67          covariance = sigma2 * np.linalg.inv(design.T @ design)
68          se_ols = float(np.sqrt(covariance[1, 1]))
69          r_squared = 1.0 - float(residual @ residual) / float(
70              ((current - current.mean()) ** 2).sum()
71          )
72
73      se_for_ci = se_hac if math.isfinite(se_hac) else se_ols
74      ci_low = rho_estimate - 1.96 * se_for_ci
75      ci_high = rho_estimate + 1.96 * se_for_ci
76      half_life = (
77          math.log(0.5) / math.log(rho_estimate)
78          if 0.0 < rho_estimate < 1.0
79          else math.nan
80      )
81      long_run_mean = alpha / (1.0 - rho_estimate) if rho_estimate != 1.0 else math.nan
82
83      is_synthetic = bool(
84          frame["is_synthetic"].astype(str).str.lower().isin(["true", "1"]).all()
85      )
86      data_source = (
87          str(frame["data_source"].iloc[0])
88          if "data_source" in frame.columns
89          else "unknown"
90      )
91      if is_synthetic or not 0.0 <= rho_estimate < 0.999:
92          rho_used = BASELINE["rho_i"]
93          basis = "fallback_0.80_synthetic_or_implausible_estimate"
94      else:
95          rho_used = min(max(rho_estimate, 0.0), 0.98)
96          basis = "estimated_ar1_real_bcch_tpm"
97
98      output = pd.DataFrame(
99          [
100              {
101                  "alpha": alpha,
102                  "rho_i_estimate": rho_estimate,
```

```

103         "rho_i_se_ols": se_ols,
104         "rho_i_se_hac": se_hac,
105         "rho_i_ci95_low": ci_low,
106         "rho_i_ci95_high": ci_high,
107         "half_life_quarters": half_life,
108         "long_run_mean_pct": long_run_mean,
109         "rho_i_used": rho_used,
110         "r_squared": r_squared,
111         "observations": n_obs,
112         "sample_start": sample_start,
113         "sample_end": sample_end,
114         "is_synthetic": is_synthetic,
115         "data_source": data_source,
116         "calibration_basis": basis,
117         "estimation_method": method,
118     }
119 ]
120 )
121 destination = TABLES / "rhoi_estimate.csv"
122 output.to_csv(destination, index=False)
123 print(output.T.to_string())
124 print(f"Wrote {destination}")
125
126
127 if __name__ == "__main__":
128     main()

```

23 python/estimate_rstar.py

```
1 """Light empirical anchor for the neutral real rate r* (Chile, real data).
2
3 The minimal NK model treats r* as a calibrated constant; the assignment asks for the
4 {2,3,4}% annual scenarios. Here we provide a data-based reference point so the scenario
5 grid can be judged against the realised ex-post real rate:
6
7 * ex-post real quarterly rate r_t = i_q - infl_q (full IT sample and last 5 years);
8 * HP-filter trend of r_t (end-of-sample value as a slow-moving r* proxy).
9
10 These are descriptive anchors (not a Laubach-Williams estimate); the report makes that
11 explicit. Output: outputs/tables/rstar_estimates.csv.
12 """
13
14 from __future__ import annotations
15
16 import json
17 import numpy as np
18 import pandas as pd
19
20 from common import DATA_CLEAN, TABLES, ensure_directories
21
22
23 def annualise(q: float) -> float:
24     return ((1.0 + q) ** 4 - 1.0) * 100.0
25
26
27 def main() -> None:
28     ensure_directories()
29     panel = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
30     metadata = json.loads(
31         (DATA_CLEAN / "dataset_metadata.json").read_text(encoding="utf-8")
32     )
33     is_synthetic = bool(metadata.get("is_synthetic", True))
34     data_source = str(metadata.get("institution", metadata.get("source", "unknown")))
35     real_q = panel["real_rate_q"].dropna().to_numpy()
36
37     from statsmodels.tsa.filters.hp_filter import hpfilter
38
39     cycle, trend = hpfilter(panel["real_rate_q"].dropna(), lamb=1600)
40
41     rows = []
42     full_mean = float(np.mean(real_q))
43     rows.append(
44         {
45             "method": "ex_post_real_rate_mean_full_sample",
46             "rstar_quarterly": full_mean,
47             "rstar_annual_pct": annualise(full_mean),
48             "beta_implied": 1.0 / (1.0 + full_mean),
49             "observations": int(len(real_q)),
50         }
51     )
52     last20 = real_q[-20:]
53     last_mean = float(np.mean(last20))
54     rows.append(
55         {
56             "method": "ex_post_real_rate_mean_last_5y",
57             "rstar_quarterly": last_mean,
58             "rstar_annual_pct": annualise(last_mean),
59             "beta_implied": 1.0 / (1.0 + last_mean),
60             "observations": int(len(last20)),
61         }
62     )
63     trend_end = float(trend.iloc[-1])
64     rows.append(
65         {
66             "method": "hp_trend_real_rate_end_of_sample",
67             "rstar_quarterly": trend_end,
68             "rstar_annual_pct": annualise(trend_end),
69             "beta_implied": 1.0 / (1.0 + trend_end),
70             "observations": int(len(real_q)),
71         }
72     )
73
74     table = pd.DataFrame(rows)
75     table["is_synthetic"] = is_synthetic
76     table["data_source"] = data_source
77     table.to_csv(TABLES / "rstar_estimates.csv", index=False)
78     print(table.round(4).to_string(index=False))
79     print(f"Wrote {TABLES / 'rstar_estimates.csv'}")
80
81
82 if __name__ == "__main__":
83     main()
```

24 python/estimate_time_varying_rstar.py

```
1  """Estimate a transparent time-varying statistical proxy for Chilean r-star.
2
3  Expected inflation is generated recursively with an AR model, so the ex-ante
4  real policy rate does not use future inflation. A local-level state-space model
5  then separates a slow-moving latent component from high-frequency noise, with
6  the lagged output gap as a cyclical control.
7
8  This is deliberately labelled a proxy. It is not a full Laubach-Williams model:
9  potential growth, the IS relation and inflation expectations are not jointly
10 estimated in a structural system.
11 """
12
13 from __future__ import annotations
14
15 import matplotlib
16
17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19 import numpy as np
20 import pandas as pd
21 import statsmodels.api as sm
22 from statsmodels.tsa.statespace.structural import UnobservedComponents
23
24 from common import DATA_CLEAN, FIGURES, TABLES, ensure_directories
25
26
27 def recursive_inflation_expectation(pi: np.ndarray, max_lags: int = 4) -> np.ndarray:
28     expected = np.full(len(pi), np.nan)
29     for t in range(len(pi)):
30         if t < 8:
31             expected[t] = float(np.mean(pi[: t + 1]))
32             continue
33         lags = min(max_lags, t - 2)
34         y = pi[lags:t]
35         x = np.column_stack(
36             [pi[lags - lag - 1:t - lag - 1] for lag in range(lags)]
37         )
38         model = sm.OLS(y, sm.add_constant(x)).fit()
39         predictors = np.r_[1.0, [pi[t - lag - 1] for lag in range(lags)]]
40         expected[t] = float(model.predict(predictors)[0])
41     return expected
42
43
44 def annualized_real_rate(nominal_q: np.ndarray, expected_inflation_q: np.ndarray) -> np.ndarray:
45     return (((1.0 + nominal_q) / (1.0 + expected_inflation_q)) ** 4 - 1.0) * 100.0
46
47
48 def main() -> None:
49     ensure_directories()
50     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
51     pi_q = macro["infl_q"].to_numpy(dtype=float)
52     expected_pi_q = recursive_inflation_expectation(pi_q)
53     real_ex_ante = annualized_real_rate(
54         macro["i_q"].to_numpy(dtype=float), expected_pi_q
55     )
56     gap_lag = (100.0 * macro["output_gap"]).shift(1).fillna(0.0)
57
58     model = UnobservedComponents(
59         real_ex_ante,
60         level="local level",
61         exog=pd.DataFrame({"output_gap_lag_pp": gap_lag}),
62     )
63     result = model.fit(disp=False, maxiter=2000)
64     smoothed = np.asarray(result.level.smoothed)
65     filtered = np.asarray(result.level.filtered)
66     variance = np.maximum(np.asarray(result.level.smoothed_cov), 0.0)
67     se = np.sqrt(variance)
68
69     output = pd.DataFrame(
70         {
71             "date": macro["date"],
72             "policy_rate_annual_pct": macro["tpm_annual_pct"],
73             "expected_inflation_annual_pct": ((1.0 + expected_pi_q) ** 4 - 1.0) * 100.0,
74             "real_rate_ex_ante_annual_pct": real_ex_ante,
75             "rstar_filtered_pct": filtered,
76             "rstar_smoothed_pct": smoothed,
77             "rstar_lower_95_pct": smoothed - 1.96 * se,
78             "rstar_upper_95_pct": smoothed + 1.96 * se,
79             "output_gap_pct": 100.0 * macro["output_gap"],
80         }
81     )
82     output.to_csv(TABLES / "rstar_time_varying.csv", index=False)
83
84     summary = pd.DataFrame(
85         [
86             {
87                 "method": "local_level_state_space_proxy",
88                 "sample_start": macro["date"].iloc[0],
89                 "sample_end": macro["date"].iloc[-1],
90                 "observations": len(macro),
91                 "converged": bool(result.mle_retvals.get("converged", False)),
92                 "latest_rstar_smoothed_pct": float(smoothed[-1]),
93                 "latest_rstar_filtered_pct": float(filtered[-1]),
94                 "sample_mean_rstar_pct": float(np.mean(smoothed)),
95                 "sample_min_rstar_pct": float(np.min(smoothed)),
96                 "sample_max_rstar_pct": float(np.max(smoothed)),
97                 "variance_irregular": float(result.params["sigma2_irregular"]),
98                 "variance_level": float(result.params["sigma2_level"]),
99                 "gap_coefficient": float(result.params["beta_output_gap_lag_pp"]),
100                 "fixed_grid_low_pct": 2.0,
101                 "fixed_grid_baseline_pct": 3.0,
102                 "fixed_grid_high_pct": 4.0,
```

```

103         "interpretation": "statistical proxy; not structural Laubach-Williams r-star",
104     }
105 ]
106 )
107 summary.to_csv(TABLES / "rstar_time_varying_summary.csv", index=False)
108
109 x = np.arange(len(macro))
110 ticks = np.arange(0, len(macro), 8)
111 labels = macro.loc[ticks, "date"]
112 fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
113 axes[0].plot(x, real_ex_ante, color="#9CA3AF", linewidth=1,
114             label="Juro real ex ante (proxy)")
115 axes[0].fill_between(
116     x,
117     output["rstar_lower_95_pct"],
118     output["rstar_upper_95_pct"],
119     color="#93C5FD",
120     alpha=0.35,
121     label="IC 95% do nivel latente",
122 )
123 axes[0].plot(x, smoothed, color="#1E4E79", linewidth=2.2,
124             label="r* suavizado (proxy)")
125 axes[0].plot(x, filtered, color="#16A34A", linewidth=1.4, linestyle="--",
126             label="r* filtrado")
127 axes[0].axhspan(2.0, 4.0, color="#F59E0B", alpha=0.10, label="Grade fixa 2%-4%")
128 axes[0].axhline(3.0, color="#B45309", linestyle=":", linewidth=1.3)
129 axes[0].set_ylabel("% a.a.")
130 axes[0].set_title("Taxa real ex ante e componente neutro variavel")
131 axes[0].legend(ncol=3, fontsize=8)
132 axes[0].grid(alpha=0.25)
133
134 axes[1].plot(x, macro["infl_annual_pct"], color="#B45309", linewidth=1.5,
135             label="Inflacao realizada anualizada")
136 axes[1].plot(x, output["expected_inflation_annual_pct"], color="#1E4E79",
137             linewidth=1.5, label="Inflacao esperada AR recursiva")
138 axes[1].axhline(3.0, color="black", linestyle=":", linewidth=1,
139             label="Meta de 3%")
140 axes[1].set_ylabel("% a.a.")
141 axes[1].set_title("Proxy de expectativas usada no juro real ex ante")
142 axes[1].legend(ncol=3, fontsize=8)
143 axes[1].grid(alpha=0.25)
144 axes[1].set_xticks(ticks)
145 axes[1].set_xticklabels(labels, rotation=70, fontsize=8)
146 fig.suptitle("Chile: proxy estatistica de r* variavel no tempo")
147 fig.text(
148     0.01,
149     0.01,
150     "Modelo local-level com hiato defasado. Nao e uma estimativa estrutural "
151     "Laubach-Williams; expectativas sao previsoes AR recursivas.",
152     fontsize=8,
153 )
154 fig.tight_layout(rect=(0, 0.03, 1, 0.95))
155 fig.savefig(FIGURES / "rstar_time_varying.png", dpi=180)
156 plt.close(fig)
157 print(summary.to_string(index=False))
158
159
160 if __name__ == "__main__":
161     main()

```

Execucao (Octave/Dynare + Bayesiano)

25 python/run_dynare_batch.py

```
1  """Locate Octave and Dynare and run the Dynare models robustly on Windows.
2
3  To avoid the OneDrive file-locking and accented-path problems that occur when Dynare
4  preprocesses inside the synced project folder, this wrapper:
5
6  1. stages the requested .mod files, the export helpers and dynare/run_models.m into a
7     disposable ASCII temp directory (copy is done in Python, which reads the accented
8     source paths reliably);
9  2. runs Octave entirely inside that ASCII directory with NK_REPO_ROOT pointing there;
10 3. copies the resulting CSVs back into outputs/dynare/ in the repository.
11
12 By default only the baseline runs; pass --all to also run every generated scenario.
13 A hard timeout terminates the Octave process tree.
14 """
15
16 from __future__ import annotations
17
18 import argparse
19 import os
20 import shutil
21 import subprocess
22 import sys
23 import tempfile
24 from pathlib import Path
25
26 from common import (
27     DYNARE_OUTPUTS,
28     LOGS,
29     ROOT,
30     ensure_directories,
31     find_dynare_matlab,
32     find_octave,
33 )
34
35
36 def octave_literal(path) -> str:
37     return str(path).replace("\\", "/").replace("'", '"')
38
39
40 def stage_work_dir(run_all: bool) -> Path:
41     work = Path(tempfile.gettempdir()) / "nk_dynare_work"
42     if work.exists():
43         shutil.rmtree(work, ignore_errors=True)
44     (work / "outputs" / "dynare").mkdir(parents=True, exist_ok=True)
45     (work / "outputs" / "logs").mkdir(parents=True, exist_ok=True)
46     dynare_dir = ROOT / "dynare"
47     for helper in ("export_results.m", "export_stability.m", "run_models.m"):
48         shutil.copy(dynare_dir / helper, work / helper)
49     shutil.copy(dynare_dir / "nk_chile_base.mod", work / "nk_chile_base.mod")
50     if run_all:
51         for mod in sorted((dynare_dir / "generated").glob("*.mod")):
52             shutil.copy(mod, work / mod.name)
53     return work
54
55
56 def copy_results_back(work: Path) -> list[str]:
57     staged = work / "outputs" / "dynare"
58     copied = []
59     if not staged.exists():
60         return copied
61     for scenario_dir in sorted(staged.iterdir()):
62         if not scenario_dir.is_dir():
63             continue
64         dest = DYNARE_OUTPUTS / scenario_dir.name
65         dest.mkdir(parents=True, exist_ok=True)
66         for csv in scenario_dir.glob("*.csv"):
67             shutil.copy(csv, dest / csv.name)
68         copied.append(scenario_dir.name)
69     return copied
70
71
72 def kill_tree(proc: subprocess.Popen) -> None:
73     try:
74         subprocess.run(
75             ["taskkill", "/F", "/T", "/PID", str(proc.pid)],
76             capture_output=True,
77             check=False,
78         )
79     except Exception: # noqa: BLE001
80         proc.kill()
81
82
83 def main() -> int:
84     parser = argparse.ArgumentParser()
85     parser.add_argument("--all", action="store_true",
86                         help="Run the baseline and every generated scenario.")
87     parser.add_argument("--timeout", type=int, default=None,
88                         help="Maximum runtime in seconds (default 150 baseline, 600 all).")
89     parser.add_argument("--strict", action="store_true",
90                         help="Return nonzero when Octave or Dynare is unavailable.")
91     args = parser.parse_args()
92     ensure_directories()
93
94     octave = find_octave()
95     dynare_matlab = find_dynare_matlab()
96     if octave is None or dynare_matlab is None:
97         message = (
98             f"Dynare batch skipped. Octave found: {octave is not None}; "
```

```

99         f"Dynare found: {dynare_matlab is not None}. "
100         "Set OCTAVE_BIN and DYNARE_MATLAB if installed elsewhere."
101     )
102     print(f"WARNING: {message}")
103     (LOGS / "dynare_batch.log").write_text(message + "\n", encoding="utf-8")
104     return 1 if args.strict else 0
105
106 work = stage_work_dir(args.all)
107 expression = (
108     f"addpath('{octave_literal(dynare_matlab)}'); run('run_models.m');"
109 )
110 command = [str(octave), "--quiet", "--eval", expression]
111 environment = os.environ.copy()
112 environment["NK_REPO_ROOT"] = str(work)
113 timeout = args.timeout or (600 if args.all else 150)
114 print(f"Octave: {octave}")
115 print(f"Work dir (ASCII): {work}")
116 print(f"Mode: {'all scenarios' if args.all else 'baseline only'}; timeout {timeout}s")
117
118 proc = subprocess.Popen(
119     command, cwd=str(work), env=environment, text=True,
120     stdout=subprocess.PIPE, stderr=subprocess.PIPE,
121 )
122 try:
123     stdout, stderr = proc.communicate(timeout=timeout)
124     timed_out = False
125 except subprocess.TimeoutExpired:
126     kill_tree(proc)
127     stdout, stderr = proc.communicate()
128     timed_out = True
129
130 copied = copy_results_back(work)
131 log_text = (
132     f"COMMAND: {' '.join(command)}\nWORK: {work}\nTIMEOUT: {timed_out}\n"
133     f"COPIED: {copied}\n\nSTDOUT\n{stdout}\n\nSTDERR\n{stderr}\n"
134 )
135 (LOGS / "dynare_wrapper.log").write_text(log_text, encoding="utf-8")
136 print(stdout[-2000:] if stdout else "")
137 if stderr:
138     print(stderr[-1500:], file=sys.stderr)
139 print(f"Scenarios copied back: {copied}")
140 if timed_out:
141     print(f"ERROR: Octave exceeded {timeout}s and was terminated.", file=sys.stderr)
142     return 124
143 return proc.returncode if proc.returncode is not None else 0
144
145
146 if __name__ == "__main__":
147     raise SystemExit(main())

```

26 python/run_bayesian.py

```

1  """Run posterior-mode estimation robustly via ASCII staging.
2
3  Stages nk_chile_estim.mod, export_bayesian.m and a numeric-only observables CSV into a
4  disposable ASCII temp directory (to avoid OneDrive/accnt issues), runs the Dynare
5  estimation there, and copies the posterior CSV back to outputs/tables/. A hard timeout
6  terminates the Octave process tree.
7  """
8
9  from __future__ import annotations
10
11  import argparse
12  import json
13  import os
14  import shutil
15  import subprocess
16  import sys
17  import tempfile
18  from pathlib import Path
19
20  import pandas as pd
21
22  from common import (
23      DATA_CLEAN,
24      LOGS,
25      ROOT,
26      TABLES,
27      ensure_directories,
28      find_dynare_matlab,
29      find_octave,
30  )
31
32
33  def octave_literal(path) -> str:
34      return str(path).replace("\\", "/").replace("'", '"')
35
36
37  def kill_tree(proc: subprocess.Popen) -> None:
38      try:
39          subprocess.run(["taskkill", "/F", "/T", "/PID", str(proc.pid)],
40                          capture_output=True, check=False)
41      except Exception: # noqa: BLE001
42          proc.kill()
43
44
45  def augment_with_std(work: Path, produced: Path) -> None:
46      """Add Laplace standard errors and 95% intervals from the saved mode Hessian.
47
48      Dynare saves the posterior mode and Hessian (hh) in <model>.mode.mat. The rows of
49      xparam1/hh follow the same order as the CSV written by export_bayesian.m, so we map
50      by row index. Done in Python (scipy) because it is far more robust than reading the
51      Hessian out of Octave structures.
52      """
53      if not produced.exists():
54          return
55      mode_files = list(work.rglob("*_mode.mat"))
56      if not mode_files:
57          print("No mode .mat found; reporting posterior mode without standard errors.")
58          return
59      try:
60          import numpy as np
61          from scipy.io import loadmat
62
63          frame = pd.read_csv(produced)
64          payload = loadmat(str(mode_files[0]))
65          hessian = payload.get("hh")
66          if hessian is None or hessian.shape[0] != len(frame):
67              print(f"Mode Hessian shape {None if hessian is None else hessian.shape} "
68                    f"!= {len(frame)} rows; skipping std.")
69              return
70          std = np.sqrt(np.diag(np.linalg.inv(hessian)))
71          xparam = payload.get("xparam1")
72          if xparam is not None:
73              xparam = np.asarray(xparam).ravel()
74              if len(xparam) == len(frame):
75                  frame["posterior_mode"] = xparam # authoritative mode (all params, in order)
76                  frame["posterior_std"] = std
77                  frame["ci95_low"] = frame["posterior_mode"] - 1.96 * std
78                  frame["ci95_high"] = frame["posterior_mode"] + 1.96 * std
79                  frame.to_csv(produced, index=False)
80                  print(f"Added mode/std from {mode_files[0].name}.")
81      except Exception as exc: # noqa: BLE001
82          print(f"Standard-error augmentation failed ({exc}); reporting mode only.")
83
84
85  def main() -> int:
86      parser = argparse.ArgumentParser()
87      parser.add_argument("--timeout", type=int, default=900)
88      args = parser.parse_args()
89      ensure_directories()
90
91      octave = find_octave()
92      dynare_matlab = find_dynare_matlab()
93      if octave is None or dynare_matlab is None:
94          print("WARNING: Octave/Dynare not found; Bayesian estimation skipped.")
95          return 0
96
97      observables = DATA_CLEAN / "chile_observables.csv"
98      if not observables.exists():
99          print("ERROR: chile_observables.csv missing; run build_chile_dataset.py first.",
100                file=sys.stderr)
101      return 1
102

```

```

103 work = Path(tempfile.gettempdir()) / "nk_estim_work"
104 if work.exists():
105     shutil.rmtree(work, ignore_errors=True)
106 (work / "outputs" / "tables").mkdir(parents=True, exist_ok=True)
107 (work / "outputs" / "logs").mkdir(parents=True, exist_ok=True)
108 shutil.copy(ROOT / "dynare" / "nk_chile_estim.mod", work / "nk_chile_estim.mod")
109 shutil.copy(ROOT / "dynare" / "export_bayesian.m", work / "export_bayesian.m")
110 # Numeric-only observables for Dynare's CSV reader (no date column).
111 pd.read_csv(observables)[["pi", "i", "x"]].to_csv(
112     work / "chile_observables_dynare.csv", index=False
113 )
114
115 expression = (
116     f"addpath('{octave_literal(dynare_matlab)}'); "
117     "dynare('nk_chile_estim.mod','noclearall','nolog');"
118 )
119 command = [str(octave), "--quiet", "--eval", expression]
120 environment = os.environ.copy()
121 environment["NK_REPO_ROOT"] = str(work)
122 print(f"Octave: {octave}\nWork dir: {work}\nTimeout: {args.timeout}s")
123
124 proc = subprocess.Popen(command, cwd=str(work), env=environment, text=True,
125                         stdout=subprocess.PIPE, stderr=subprocess.PIPE)
126
127 try:
128     stdout, stderr = proc.communicate(timeout=args.timeout)
129     timed_out = False
130 except subprocess.TimeoutExpired:
131     kill_tree(proc)
132     stdout, stderr = proc.communicate()
133     timed_out = True
134
135 produced = work / "outputs" / "tables" / "bayesian_estimates.csv"
136 augment_with_std(work, produced)
137 copied = False
138 if produced.exists():
139     metadata = json.loads(
140         (DATA_CLEAN / "dataset_metadata.json").read_text(encoding="utf-8")
141     )
142     frame = pd.read_csv(produced)
143     frame["is_synthetic"] = bool(metadata.get("is_synthetic", True))
144     frame["data_source"] = str(
145         metadata.get("institution", metadata.get("source", "unknown"))
146     )
147     frame["estimation_scope"] = "posterior_mode_laplace_not_mcmc"
148     frame.to_csv(produced, index=False)
149     shutil.copy(produced, TABLES / "bayesian_estimates.csv")
150     copied = True
151
152 (LOGS / "bayesian_run.log").write_text(
153     f"TIMEOUT: {timed_out}\nCOPIED: {copied}\nRC: {proc.returncode}\n\n"
154     f"STDOUT\n{stdout}\n\nSTDERR\n{stderr}\n",
155     encoding="utf-8",
156 )
157 print(stdout[-2500:] if stdout else "")
158 if stderr:
159     print(stderr[-1200:], file=sys.stderr)
160 print(f"bayesian_estimates.csv copied back: {copied}; timed_out: {timed_out}")
161 if timed_out and not copied:
162     return 124
163 return 0
164
165 if __name__ == "__main__":
166     raise SystemExit(main())

```

27 python/run_mcmc.py

```

1  """Run the full Dynare MCMC extension and compute chain diagnostics."""
2
3  from __future__ import annotations
4
5  import argparse
6  import os
7  import re
8  import shutil
9  import subprocess
10 import sys
11 import tempfile
12 from pathlib import Path
13
14 import numpy as np
15 import pandas as pd
16 from scipy.io import loadmat
17
18 from common import DATA_CLEAN, LOGS, ROOT, TABLES, find_dynare_matlab, find_octave
19
20
21 PARAMETER_ORDER = [
22     "stderr_e_x",
23     "stderr_e_pi",
24     "stderr_e_i",
25     "sigma",
26     "kappa",
27     "rho_i",
28     "phi_pi",
29     "phi_x",
30 ]
31
32
33 def octave_literal(path: Path) -> str:
34     return str(path).replace("\\", "/").replace("'", "'")
35
36
37 def split_rhat(chains: list[np.ndarray]) -> np.ndarray:
38     if len(chains) < 2:
39         return np.full(chains[0].shape[1], np.nan)
40     n = min(len(chain) for chain in chains)
41     arrays = np.stack([chain[-n:] for chain in chains])
42     chain_means = arrays.mean(axis=1)
43     between = n * chain_means.var(axis=0, ddof=1)
44     within = arrays.var(axis=1, ddof=1).mean(axis=0)
45     variance = ((n - 1) / n) * within + between / n
46     return np.sqrt(variance / within)
47
48
49 def effective_sample_size(chains: list[np.ndarray]) -> np.ndarray:
50     n = min(len(chain) for chain in chains)
51     arrays = np.stack([chain[-n:] for chain in chains])
52     m, _, k = arrays.shape
53     result = np.empty(k)
54     for parameter in range(k):
55         autocorrelations = []
56         for chain in arrays[:, :, parameter]:
57             centered = chain - chain.mean()
58             fft_size = 1 << (2 * n - 1).bit_length()
59             spectrum = np.fft.rfft(centered, fft_size)
60             covariance = np.fft.irfft(spectrum * spectrum.conjugate(), fft_size)[-n:]
61             covariance /= np.arange(n, 0, -1)
62             autocorrelations.append(covariance / covariance[0])
63         mean_acf = np.mean(autocorrelations, axis=0)
64         autocorr_sum = 0.0
65         for rho in mean_acf[1 : min(1001, n)]:
66             if rho <= 0:
67                 break
68             autocorr_sum += float(rho)
69         result[parameter] = m * n / (1.0 + 2.0 * autocorr_sum)
70     return result
71
72
73 def load_chains(work: Path) -> list[np.ndarray]:
74     files = sorted(work.rglob("*_mh*_blk*.mat"))
75     grouped: dict[int, list[np.ndarray]] = {}
76     for path in files:
77         match = re.search(r"blk(\d+)", path.name)
78         block = int(match.group(1)) if match else len(grouped) + 1
79         payload = loadmat(path)
80         draws = payload.get("x2")
81         if draws is None or draws.ndim != 2:
82             continue
83         grouped.setdefault(block, []).append(np.asarray(draws, dtype=float))
84     return [np.vstack(parts) for _, parts in sorted(grouped.items())]
85
86
87 def save_chain_outputs(chains: list[np.ndarray], burn_fraction: float = 0.30) -> None:
88     retained = [chain[int(len(chain) * burn_fraction) :] for chain in chains]
89     draws = np.vstack(retained)
90     posterior = pd.DataFrame(
91         {
92             "parameter": PARAMETER_ORDER,
93             "type": ["shock_std"] * 3 + ["param"] * 5,
94             "posterior_mean": draws.mean(axis=0),
95             "posterior_median": np.median(draws, axis=0),
96             "posterior_std": draws.std(axis=0, ddof=1),
97             "hpd90_low": np.quantile(draws, 0.05, axis=0),
98             "hpd90_high": np.quantile(draws, 0.95, axis=0),
99         }
100     )
101     posterior.to_csv(TABLES / "mcmc_posterior.csv", index=False)
102

```



```

103 diagnostics = pd.DataFrame(
104     {
105         "parameter": PARAMETER_ORDER,
106         "rhat": split_rhat(retained),
107         "effective_sample_size": effective_sample_size(retained),
108         "chains": len(retained),
109         "draws_per_chain_loaded": min(len(chain) for chain in retained),
110     }
111 )
112 diagnostics["rhat_pass_1p05"] = diagnostics["rhat"] < 1.05
113 diagnostics.to_csv(TABLES / "mcmc_diagnostics.csv", index=False)
114 print(diagnostics.to_string(index=False))
115
116
117 def main() -> int:
118     parser = argparse.ArgumentParser()
119     parser.add_argument("--timeout", type=int, default=1800)
120     parser.add_argument(
121         "--recover-dir",
122         type=Path,
123         help="Post-process completed Dynare chain files without rerunning MCMC.",
124     )
125     args = parser.parse_args()
126     if args.recover_dir:
127         chains = load_chains(args.recover_dir)
128         if len(chains) < 2 or any(
129             chain.shape[1] != len(PARAMETER_ORDER) for chain in chains
130         ):
131             print(f"ERROR: invalid MCMC chain shapes: {[c.shape for c in chains]}")
132             return 1
133         save_chain_outputs(chains)
134         print(f"Recovered MCMC outputs from {args.recover_dir}")
135         return 0
136
137     octave = find_octave()
138     dynare = find_dynare_matlab()
139     if octave is None or dynare is None:
140         print("ERROR: Octave/Dynare not found.", file=sys.stderr)
141         return 1
142
143     work = Path(tempfile.gettempdir()) / "nk_mcmc_work"
144     if work.exists():
145         shutil.rmtree(work, ignore_errors=True)
146     (work / "outputs" / "tables").mkdir(parents=True, exist_ok=True)
147     for filename in ["nk_chile_mcmc.mod", "export_mcmc.m"]:
148         shutil.copy(ROOT / "dynare" / filename, work / filename)
149     pd.read_csv(DATA_CLEAN / "chile_observables.csv")[["pi", "i", "x"]].to_csv(
150         work / "chile_observables_dynare.csv", index=False
151     )
152
153     expression = (
154         f"addpath('{octave_literal(dynare)}'); "
155         "dynare('nk_chile_mcmc.mod','noclearall','nolog');"
156     )
157     command = [str(octave), "--quiet", "--eval", expression]
158     env = os.environ.copy()
159     env["NK_REPO_ROOT"] = str(work)
160     proc = subprocess.Popen(
161         command,
162         cwd=work,
163         env=env,
164         text=True,
165         stdout=subprocess.PIPE,
166         stderr=subprocess.PIPE,
167     )
168     try:
169         stdout, stderr = proc.communicate(timeout=args.timeout)
170         timed_out = False
171     except subprocess.TimeoutExpired:
172         subprocess.run(
173             ["taskkill", "/F", "/T", "/PID", str(proc.pid)],
174             capture_output=True,
175             check=False,
176         )
177         stdout, stderr = proc.communicate()
178         timed_out = True
179
180     posterior = work / "outputs" / "tables" / "mcmc_posterior.csv"
181     copied = False
182     if posterior.exists():
183         shutil.copy(posterior, TABLES / posterior.name)
184         copied = True
185
186     chains = load_chains(work)
187     if chains and all(chain.shape[1] == len(PARAMETER_ORDER) for chain in chains):
188         save_chain_outputs(chains)
189     else:
190         print(
191             f"WARNING: could not load two {len(PARAMETER_ORDER)}-column MCMC chains; "
192             f"shapes={[[chain.shape for chain in chains]]}",
193             file=sys.stderr,
194         )
195
196     (LOGS / "mcmc_run.log").write_text(
197         f"TIMEOUT: {timed_out}\nRC: {proc.returncode}\nCOPIED: {copied}\n"
198         f"CHAIN_SHAPES: {[chain.shape for chain in chains]}\n"
199         f"STDOUT\n{stdout}\n\nSTDERR\n{stderr}\n",
200         encoding="utf-8",
201     )
202     print(stdout[-4000:] if stdout else "")
203     if stderr:
204         print(stderr[-1600:], file=sys.stderr)
205     return 0 if copied and not timed_out else 1
206
207

```

```
208 if __name__ == "__main__":  
209     raise SystemExit(main())
```

28 python/run_forecast.py

```

1  """Run the native Dynare forecast (estimation forecast=8 + conditional_forecast).
2
3  Mirrors run_bayesian.py: stages nk_chile_forecast.mod, the export helpers and a
4  numeric-only observables CSV into a disposable ASCII temp directory (to avoid the
5  OneDrive/accents issues that break Dynare's preprocessing inside the synced folder),
6  runs Dynare there, and copies the produced CSVs back to outputs/dynare/forecast/.
7  """
8
9  from __future__ import annotations
10
11  import argparse
12  import os
13  import shutil
14  import subprocess
15  import sys
16  import tempfile
17  from pathlib import Path
18
19  import pandas as pd
20
21  from common import (
22      DATA_CLEAN,
23      DYNARE_OUTPUTS,
24      LOGS,
25      ROOT,
26      ensure_directories,
27      find_dynare_matlab,
28      find_octave,
29  )
30
31
32  def octave_literal(path) -> str:
33      return str(path.replace("\\", "/").replace("'", '"'))
34
35
36  def kill_tree(proc: subprocess.Popen) -> None:
37      try:
38          subprocess.run(["taskkill", "/F", "/T", "/PID", str(proc.pid)],
39                          capture_output=True, check=False)
40      except Exception: # noqa: BLE001
41          proc.kill()
42
43
44  def copy_results_back(work: Path) -> list[str]:
45      staged = work / "outputs" / "dynare" / "forecast"
46      if not staged.exists():
47          return []
48      dest = DYNARE_OUTPUTS / "forecast"
49      dest.mkdir(parents=True, exist_ok=True)
50      copied = []
51      for item in sorted(staged.glob("*.csv")) + sorted(staged.glob("*.txt")):
52          shutil.copy(item, dest / item.name)
53          copied.append(item.name)
54      return copied
55
56
57  def main() -> int:
58      parser = argparse.ArgumentParser()
59      parser.add_argument("--timeout", type=int, default=600)
60      args = parser.parse_args()
61      ensure_directories()
62
63      octave = find_octave()
64      dynare_matlab = find_dynare_matlab()
65      if octave is None or dynare_matlab is None:
66          print("WARNING: Octave/Dynare not found; forecast skipped.", file=sys.stderr)
67          return 1
68
69      observables = DATA_CLEAN / "chile_observables.csv"
70      if not observables.exists():
71          print("ERROR: chile_observables.csv missing; run build_chile_dataset.py first.",
72                file=sys.stderr)
73          return 1
74
75      work = Path(tempfile.gettempdir()) / "nk_forecast_work"
76      if work.exists():
77          shutil.rmtree(work, ignore_errors=True)
78      (work / "outputs" / "dynare" / "forecast").mkdir(parents=True, exist_ok=True)
79      (work / "outputs" / "logs").mkdir(parents=True, exist_ok=True)
80
81      dynare_dir = ROOT / "dynare"
82      for helper in ("nk_chile_forecast.mod", "export_forecast.m", "export_conditional.m"):
83          shutil.copy(dynare_dir / helper, work / helper)
84      # Numeric-only observables for Dynare's CSV reader (no date column).
85      pd.read_csv(observables)[["pi", "i", "x"]].to_csv(
86          work / "chile_observables_dynare.csv", index=False
87      )
88
89      expression = (
90          f"addpath('{octave_literal(dynare_matlab)}'); "
91          "dynare('nk_chile_forecast.mod','noclearall','nolog');"
92      )
93      command = [str(octave), "--quiet", "--eval", expression]
94      environment = os.environ.copy()
95      environment["NK_REPO_ROOT"] = str(work)
96      print(f"Octave: {octave}\nWork dir: {work}\nTimeout: {args.timeout}s")
97
98      proc = subprocess.Popen(command, cwd=str(work), env=environment, text=True,
99                              stdout=subprocess.PIPE, stderr=subprocess.PIPE)
100     try:
101         stdout, stderr = proc.communicate(timeout=args.timeout)
102         timed_out = False

```

```

103     except subprocess.TimeoutExpired:
104         kill_tree(proc)
105         stdout, stderr = proc.communicate()
106         timed_out = True
107
108     copied = copy_results_back(work)
109     (LOGS / "forecast_run.log").write_text(
110         f"TIMEOUT: {timed_out}\nRC: {proc.returncode}\nCOPIED: {copied}\n\n"
111         f"STDOUT\n{stdout}\n\nSTDERR\n{stderr}\n",
112         encoding="utf-8",
113     )
114     print(stdout[-3000:] if stdout else "")
115     if stderr:
116         print(stderr[-1500:], file=sys.stderr)
117     print(f"Forecast CSVs copied back: {copied}; timed_out: {timed_out}")
118     if timed_out and not copied:
119         return 124
120     return 0
121
122
123 if __name__ == "__main__":
124     raise SystemExit(main())

```

29 python/run_history.py

```

1  """Run the historical shock decomposition (Kalman smoother + shock_decomposition).
2
3  Stages nk_chile_history.mod, export_history.m and a numeric observables CSV into
4  an ASCII temp directory, runs Dynare, and copies outputs/dynare/history/*.csv back.
5  Mirrors run_forecast.py.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import os
12 import shutil
13 import subprocess
14 import sys
15 import tempfile
16 from pathlib import Path
17
18 import pandas as pd
19
20 from common import (
21     DATA_CLEAN,
22     DYNARE_OUTPUTS,
23     LOGS,
24     ROOT,
25     ensure_directories,
26     find_dynare_matlab,
27     find_octave,
28 )
29
30
31 def octave_literal(path) -> str:
32     return str(path).replace("\\", "/").replace("'", "'")
33
34
35 def kill_tree(proc: subprocess.Popen) -> None:
36     try:
37         subprocess.run(["taskkill", "/F", "/T", "/PID", str(proc.pid)],
38             capture_output=True, check=False)
39     except Exception: # noqa: BLE001
40         proc.kill()
41
42
43 def main() -> int:
44     parser = argparse.ArgumentParser()
45     parser.add_argument("--timeout", type=int, default=420)
46     parser.add_argument("--mod", default="nk_chile_history.mod")
47     args = parser.parse_args()
48     ensure_directories()
49
50     octave = find_octave()
51     dynare_matlab = find_dynare_matlab()
52     if octave is None or dynare_matlab is None:
53         print("WARNING: Octave/Dynare not found; history skipped.", file=sys.stderr)
54         return 1
55
56     observables = DATA_CLEAN / "chile_observables.csv"
57     if not observables.exists():
58         print("ERROR: chile_observables.csv missing.", file=sys.stderr)
59         return 1
60
61     work = Path(tempfile.gettempdir()) / "nk_history_work"
62     if work.exists():
63         shutil.rmtree(work, ignore_errors=True)
64     (work / "outputs" / "dynare").mkdir(parents=True, exist_ok=True)
65     (work / "outputs" / "logs").mkdir(parents=True, exist_ok=True)
66
67     dynare_dir = ROOT / "dynare"
68     for helper in (args.mod, "export_history.m"):
69         shutil.copy(dynare_dir / helper, work / helper)
70     pd.read_csv(observables)[["pi", "i", "x"]].to_csv(
71         work / "chile_observables_dynare.csv", index=False
72     )
73
74     expression = (
75         f"addpath('{octave_literal(dynare_matlab)}'); "
76         f"dynare('{args.mod}', 'noclearall', 'nolog');"
77     )
78     command = [str(octave), "--quiet", "--eval", expression]
79     environment = os.environ.copy()
80     environment["NK_REPO_ROOT"] = str(work)
81     print(f"Octave: {octave}\nWork dir: {work}\nTimeout: {args.timeout}s")
82
83     proc = subprocess.Popen(command, cwd=str(work), env=environment, text=True,
84         stdout=subprocess.PIPE, stderr=subprocess.PIPE)
85
86     try:
87         stdout, stderr = proc.communicate(timeout=args.timeout)
88         timed_out = False
89     except subprocess.TimeoutExpired:
90         kill_tree(proc)
91         stdout, stderr = proc.communicate()
92         timed_out = True
93
94     staged_root = work / "outputs" / "dynare"
95     copied = []
96     if staged_root.exists():
97         for sub in sorted(staged_root.iterdir()):
98             if not sub.is_dir():
99                 continue
100             dest = DYNARE_OUTPUTS / sub.name
101             dest.mkdir(parents=True, exist_ok=True)
102             for csv in sorted(sub.glob("*.csv")):
103                 shutil.copy(csv, dest / csv.name)

```

```

103         copied.append(f"{sub.name}/{csv.name}")
104
105     (LOGS / "history_run.log").write_text(
106         f"TIMEOUT: {timed_out}\nRC: {proc.returncode}\nCOPIED: {copied}\n\n"
107         f"STDOUT\n{stdout}\n\nSTDERR\n{stderr}\n",
108         encoding="utf-8",
109     )
110     print(stdout[-2500:] if stdout else "")
111     if stderr:
112         print(stderr[-1200:], file=sys.stderr)
113     print(f"History CSVs copied back: {copied}; timed_out: {timed_out}")
114     return 0 if copied else (124 if timed_out else 1)
115
116
117 if __name__ == "__main__":
118     raise SystemExit(main())

```

30 python/run_macro_extensions.py

```
1 """Run the open-economy and hybrid-NKPC Dynare extensions via ASCII staging."""
2
3 from __future__ import annotations
4
5 import argparse
6 import os
7 import shutil
8 import subprocess
9 import sys
10 import tempfile
11 from pathlib import Path
12
13 from common import DYNARE_OUTPUTS, LOGS, ROOT, find_dynare_matlab, find_octave
14
15
16 def octave_literal(path: Path) -> str:
17     return str(path).replace("\\", "/").replace("'", '"')
18
19
20 def kill_tree(proc: subprocess.Popen) -> None:
21     subprocess.run(
22         ["taskkill", "/F", "/T", "/PID", str(proc.pid)],
23         capture_output=True,
24         check=False,
25     )
26
27
28 def main() -> int:
29     parser = argparse.ArgumentParser()
30     parser.add_argument("--timeout", type=int, default=300)
31     args = parser.parse_args()
32     octave = find_octave()
33     dynare = find_dynare_matlab()
34     if octave is None or dynare is None:
35         print("ERROR: Octave/Dynare not found.", file=sys.stderr)
36         return 1
37
38     work = Path(tempfile.gettempdir()) / "nk_macro_extensions"
39     if work.exists():
40         shutil.rmtree(work, ignore_errors=True)
41     (work / "outputs" / "dynare").mkdir(parents=True, exist_ok=True)
42     for name in [
43         "nk_chile_open.mod",
44         "nk_chile_hybrid.mod",
45         "export_extension.m",
46         "export_stability.m",
47     ]:
48         shutil.copy(ROOT / "dynare" / name, work / name)
49     runner = (
50         "addpath('" + octave_literal(dynare) + "');"
51         "dynare('nk_chile_open.mod','noclearall','nolog');"
52         "dynare('nk_chile_hybrid.mod','noclearall','nolog');"
53     )
54     command = [str(octave), "--quiet", "--eval", runner]
55     env = os.environ.copy()
56     env["NK_REPO_ROOT"] = str(work)
57     proc = subprocess.Popen(
58         command,
59         cwd=work,
60         env=env,
61         text=True,
62         stdout=subprocess.PIPE,
63         stderr=subprocess.PIPE,
64     )
65     try:
66         stdout, stderr = proc.communicate(timeout=args.timeout)
67         timed_out = False
68     except subprocess.TimeoutExpired:
69         kill_tree(proc)
70         stdout, stderr = proc.communicate()
71         timed_out = True
72
73     copied = []
74     for scenario in ["open_economy", "hybrid_nkpc"]:
75         source = work / "outputs" / "dynare" / scenario
76         if not source.exists():
77             continue
78         destination = DYNARE_OUTPUTS / scenario
79         destination.mkdir(parents=True, exist_ok=True)
80         for item in source.glob("*"):
81             if item.is_file():
82                 shutil.copy(item, destination / item.name)
83         copied.append(scenario)
84     (LOGS / "macro_extensions.log").write_text(
85         f"TIMEOUT: {timed_out}\nRC: {proc.returncode}\nCOPIED: {copied}\n\n"
86         f"STDOUT\n{stdout}\n\nSTDERR\n{stderr}\n",
87         encoding="utf-8",
88     )
89     print(stdout[-3500:] if stdout else "")
90     if stderr:
91         print(stderr[-1500:], file=sys.stderr)
92     print(f"Copied: {copied}; timeout={timed_out}; rc={proc.returncode}")
93     return 0 if len(copied) == 2 and not timed_out else 1
94
95
96 if __name__ == "__main__":
97     raise SystemExit(main())
```

Previsao e avaliacao

31 python/forecast_model.py

```
1  """Visualise and cross-check the native Dynare forecast (slides 48-55).
2
3  The forecast itself is produced by Dynare (python/run_forecast.py runs
4  dynare/nk_chile_forecast.mod):
5
6  * the UNCONDITIONAL projection comes from estimation(..., forecast=8); Dynare
7  returns oo_.forecast.Mean and the HPDinf/HPDsup credibility bands, launched
8  from the smoothed end-of-sample state. We read those CSVs and draw the fan
9  chart spliced onto the observed history.
10 * the CONDITIONAL projections come from Dynare's conditional_forecast command
11   (hold the rate, tighten, force inflation to target). Dynare's
12   conditional_forecast launches from the steady state, so here we ALSO recompute
13   the same three experiments anchored to the current state with the exact
14   first-order reduced form (identical to Dynare's solution: the unconditional
15   mean matches oo_.forecast to the last digit). This anchored version connects
16   continuously to history and recovers the monetary-policy innovation e_i needed
17   to enforce each path.
18
19 All series are mapped back to interpretable annualised levels using the sample
20 means that build_chile_dataset.py removed when it created the observables
21 (pi = infl_q - mean(infl_q); i = i_q - mean(i_q); x = output_gap - mean).
22 """
23
24 from __future__ import annotations
25
26 import json
27
28 import matplotlib
29
30 matplotlib.use("Agg")
31 import matplotlib.pyplot as plt
32 import numpy as np
33 import pandas as pd
34
35 from common import (
36     DATA_CLEAN,
37     DYNARE_OUTPUTS,
38     FIGURES,
39     TABLES,
40     calibration_rho,
41     complete_parameters,
42     ensure_directories,
43     _state_coefficients,
44 )
45
46 HORIZON = 8
47 INFLATION_TARGET_ANNUAL = 0.03
48 FORECAST_DIR = DYNARE_OUTPUTS / "forecast"
49
50 # Palette (kept consistent with the rest of the figure set).
51 COLORS = {
52     "unconditional": "#111827",
53     "hold": "#FF6B35",
54     "hike": "#16A34A",
55     "pi_target": "#7C3AED",
56 }
57
58
59 def annual_pct(quarterly_fraction):
60     return ((1.0 + quarterly_fraction) ** 4 - 1.0) * 100.0
61
62
63 def solution_matrices(params: dict) -> tuple[np.ndarray, np.ndarray]:
64     """Return the reduced form  $y_t = transition * i_{t-1} + impact * \epsilon_t$ """
65     a_x, a_pi, a_i = _state_coefficients(params)
66     sigma, beta, kappa = params["sigma"], params["beta"], params["kappa"]
67     rho_i, phi_pi, phi_x = params["rho_i"], params["phi_pi"], params["phi_x"]
68     c_i = a_x - (1.0 - a_pi) / sigma
69     structural = np.array(
70         [
71             [1.0, 0.0, -c_i],
72             [-kappa, 1.0, -beta * a_pi],
73             [-(1.0 - rho_i) * phi_x, -(1.0 - rho_i) * phi_pi, 1.0],
74         ]
75     )
76     transition = np.array([a_x, a_pi, a_i])
77     impact = np.linalg.inv(structural)
78     return transition, impact
79
80
81 def conditional_path(transition, impact, initial_i_gap, controlled_variable, targets):
82     """Hit a constrained endogenous path using e_i, anchored at the current state."""
83     index = {"x": 0, "pi": 1, "i": 2}[controlled_variable]
84     policy_shock_index = 2
85     values = np.zeros((HORIZON, 3))
86     implied_policy_shocks = np.zeros(HORIZON)
87     previous_i = initial_i_gap
88     for horizon in range(HORIZON):
89         mean = transition * previous_i
90         innovations = np.zeros(3)
91         if horizon in targets:
92             denominator = impact[index, policy_shock_index]
93             innovations[policy_shock_index] = (targets[horizon] - mean[index]) / denominator
94         current = mean + impact @ innovations
95         values[horizon] = current
96         implied_policy_shocks[horizon] = innovations[policy_shock_index]
97         previous_i = current[2]
98     return values, implied_policy_shocks
```



```

99
100
101 def load_unconditional() -> pd.DataFrame:
102     return pd.read_csv(FORECAST_DIR / "forecast_unconditional.csv")
103
104
105 def main() -> None:
106     ensure_directories()
107     params = complete_parameters({"rho_i": calibration_rho()})
108     transition, impact = solution_matrices(params)
109
110     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
111     observables = pd.read_csv(DATA_CLEAN / "chile_observables.csv")
112     metadata = json.loads((DATA_CLEAN / "dataset_metadata.json").read_text(encoding="utf-8"))
113     data_source = str(metadata.get("institution", metadata.get("source", "unknown")))
114
115     # Level mapping: inflation is now centred on the TARGET, the rate on its mean.
116     mean_infl_q = float(macro["infl_q"].mean())
117     mean_i_q = float(macro["i_q"].mean())
118     mean_tpm_annual = annual_pct(mean_i_q)
119     mean_infl_annual = annual_pct(mean_infl_q)
120     target_q = (1.0 + INFLATION_TARGET_ANNUAL) ** 0.25 - 1.0
121     pi_target_dev = 0.0 # the target IS the steady state now (pi deviation = 0)
122
123     def to_level(variable: str, deviation):
124         if variable == "x":
125             return 100.0 * np.asarray(deviation)
126         if variable == "pi":
127             return annual_pct(target_q + np.asarray(deviation))
128         return annual_pct(mean_i_q + np.asarray(deviation))
129
130     last_i_gap = float(observables.iloc[-1]["i"])
131     last_period = pd.Period(str(observables.iloc[-1]["date"]), freq="Q")
132     future = [str(last_period + h) for h in range(1, HORIZON + 1)]
133
134     # ----- #
135     # 1) Unconditional forecast: read Dynare's mean and HPD bands. #
136     # ----- #
137     uncond = load_unconditional()
138     uncond = uncond.assign(
139         mean_level=lambda d: [to_level(v, m) for v, m in zip(d["variable"], d["mean"])],
140         inf_level=lambda d: [to_level(v, m) for v, m in zip(d["variable"], d["hpd_inf"])],
141         sup_level=lambda d: [to_level(v, m) for v, m in zip(d["variable"], d["hpd_sup"])],
142         date=lambda d: [future[h - 1] for h in d["horizon"]],
143     )
144
145     # ----- #
146     # 2) Conditional experiments anchored at the current state. #
147     # ----- #
148     scenarios = {
149         "hold": {
150             "label": "Hold rate at current level (2q)",
151             "controlled": "i",
152             "targets": {0: last_i_gap, 1: last_i_gap},
153         },
154         "hike": {
155             "label": "Tighten +1pp for 4q",
156             "controlled": "i",
157             "targets": {h: 0.0025 for h in range(4)},
158         },
159         "pi_target": {
160             "label": "Inflation to 3% target for 4q",
161             "controlled": "pi",
162             "targets": {h: pi_target_dev for h in range(4)},
163         },
164     }
165     # Unconditional reduced-form mean (matches Dynare oo_.forecast to last digit).
166     uncond_dev = np.zeros((HORIZON, 3))
167     prev = last_i_gap
168     for h in range(HORIZON):
169         uncond_dev[h] = transition * prev
170         prev = uncond_dev[h, 2]
171
172     results = {"unconditional": (uncond_dev, np.zeros(HORIZON))}
173     for key, spec in scenarios.items():
174         values, shocks = conditional_path(
175             transition, impact, last_i_gap, spec["controlled"], spec["targets"]
176         )
177         results[key] = (values, shocks)
178
179     # ----- #
180     # 3) Tidy forecast table (deviations + levels), combining both views. #
181     # ----- #
182     var_index = {"x": 0, "pi": 1, "i": 2}
183     records: list[dict] = []
184     for scenario, (values, shocks) in results.items():
185         for h in range(HORIZON):
186             for variable, idx in var_index.items():
187                 row = {
188                     "horizon": h + 1,
189                     "date": future[h],
190                     "scenario": scenario,
191                     "variable": variable,
192                     "value_dev": values[h, idx],
193                     "value_level": float(to_level(variable, values[h, idx])),
194                     "implied_e_i_q": shocks[h] if variable == "i" else np.nan,
195                 }
196                 if scenario == "unconditional":
197                     band = uncond[(uncond["horizon"] == h + 1) & (uncond["variable"] == variable)]
198                     row["p05_level"] = float(band["inf_level"].iloc[0]) if len(band) else np.nan
199                     row["p95_level"] = float(band["sup_level"].iloc[0]) if len(band) else np.nan
200                 records.append(row)
201     table = pd.DataFrame(records)
202     table["data_source"] = data_source
203     table["mean_tpm_annual_pct"] = mean_tpm_annual

```

```

204 table["mean_infl_annual_pct"] = mean_infl_annual
205 table.to_csv(TABLES / "forecast.csv", index=False)
206 table[table["horizon"].isin([1, 4, 8]).to_csv(TABLES / "forecast_summary.csv", index=False)
207
208 # ----- #
209 # Figure 1: history + unconditional Dynare fan chart (annual levels). #
210 # ----- #
211 history = macro.tail(16).copy()
212 fig, axes = plt.subplots(3, 1, figsize=(11, 9))
213 panels = (
214     ("i", history["tpm_annual_pct"], "Policy rate TPM (% per year)"),
215     ("pi", history["infl_annual_pct"], "Inflation (% per year)"),
216     ("x", 100.0 * history["output_gap"], "Output gap (%)" ),
217 )
218 for ax, (variable, hist_series, title) in zip(axes, panels):
219     ax.plot(history["date"], hist_series, color="black", marker="o",
220             markersize=3, label="Observed history")
221     sub = uncond[uncond["variable"] == variable]
222     ax.plot(sub["date"], sub["mean_level"], color="#1E4E79", linewidth=2.4,
223             label="Dynare forecast mean")
224     ax.fill_between(sub["date"], sub["inf_level"], sub["sup_level"],
225                     color="#1E4E79", alpha=0.16, label="Dynare HPD band")
226     if variable == "pi":
227         ax.axhline(3.0, color="#B45309", linestyle="--", linewidth=1, label="3% target")
228         ax.axhline(mean_infl_annual, color="#6B7280", linestyle=":", linewidth=1,
229                     label=f"Sample mean ({mean_infl_annual:.1f}%)")
230     elif variable == "i":
231         ax.axhline(mean_tpm_annual, color="#6B7280", linestyle=":", linewidth=1,
232                     label=f"Sample mean ({mean_tpm_annual:.1f}%)")
233     else:
234         ax.axhline(0.0, color="#6B7280", linewidth=0.7)
235     ax.set_title(title)
236     ax.grid(alpha=0.25)
237     ax.tick_params(axis="x", rotation=75, labels=7)
238     ax.legend(fontsize=7, ncol=2)
239 fig.suptitle("Chile: unconditional forecast (Dynare estimation, forecast=8)")
240 fig.text(0.01, 0.005,
241          f"Source: {data_source}. Native Dynare forecast launched from the smoothed "
242          "end-of-sample state. Mechanical model projection, not an official BOCh forecast.",
243          fontsize=7)
244 fig.tight_layout(rect=(0, 0.03, 1, 0.96))
245 fig.savefig(FIGURES / "forecast_fanchart.png", dpi=180)
246 plt.close(fig)
247
248 # ----- #
249 # Figure 2: conditional scenarios anchored to the current state. #
250 # ----- #
251 fig, axes = plt.subplots(1, 3, figsize=(14, 4.8))
252 keymap = (("i", "Policy rate TPM (% per year)"),
253           ("pi", "Inflation (% per year)"),
254           ("x", "Output gap (%)" ))
255 for ax, (variable, title) in zip(axes, keymap):
256     idx = var_index[variable]
257     htail = macro.tail(4)
258     hist_series = {"i": htail["tpm_annual_pct"], "pi": htail["infl_annual_pct"],
259                    "x": 100.0 * htail["output_gap"]}[variable]
260     ax.plot(range(-3, 1), hist_series, color="black", marker="o", markersize=3,
261             linewidth=1.2, label="History")
262     for scenario, (values, _) in results.items():
263         color = COLORS["unconditional" if scenario == "unconditional" else scenario]
264         label = "Unconditional" if scenario == "unconditional" else scenarios[scenario]["label"]
265         level = to_level(variable, values[:, idx])
266         # splice: connect the last observed point to the forecast path
267         xs = list(range(0, HORIZON + 1))
268         ys = [float(hist_series.iloc[-1])] + list(level)
269         ax.plot(xs, ys, color=color, linewidth=2, label=label,
270                 linestyle="--" if scenario == "unconditional" else "-")
271     if variable == "pi":
272         ax.axhline(3.0, color="#B45309", linestyle="--", linewidth=1)
273     elif variable == "i":
274         ax.axhline(mean_tpm_annual, color="#6B7280", linestyle=":", linewidth=1)
275     else:
276         ax.axhline(0.0, color="#6B7280", linewidth=0.8)
277         ax.axvline(0, color="#9CA3AF", linewidth=0.8)
278     ax.set_title(title)
279     ax.set_xlabel("Quarter (0 = last data)")
280     ax.grid(alpha=0.25)
281 axes[0].legend(fontsize=7)
282 fig.suptitle("Conditional scenarios (anchored at the current state; e_i solves each path)")
283 fig.tight_layout(rect=(0, 0, 1, 0.94))
284 fig.savefig(FIGURES / "conditional_scenarios.png", dpi=180)
285 plt.close(fig)
286
287 # ----- #
288 # Figure 3: monetary-policy innovations e_i required by each path. #
289 # ----- #
290 fig, ax = plt.subplots(figsize=(9.5, 4.8))
291 for scenario in ("hold", "hike", "pi_target"):
292     _, shocks = results[scenario]
293     ax.plot(np.arange(1, HORIZON + 1), 100.0 * shocks, marker="o", linewidth=2,
294             color=COLORS[scenario], label=scenarios[scenario]["label"])
295 ax.axhline(0, color="black", linewidth=0.8)
296 ax.set(title="Monetary-policy innovations e_i required by each conditional path",
297        xlabel="Forecast quarter", ylabel="e_i (quarterly percentage points)")
298 ax.grid(alpha=0.25)
299 ax.legend(fontsize=8)
300 fig.tight_layout()
301 fig.savefig(FIGURES / "conditional_policy_shocks.png", dpi=180)
302 plt.close(fig)
303
304 print(f"Sample-mean TPM {mean_tpm_annual:.2f}% | mean inflation {mean_infl_annual:.2f}% "
305       f"| 3% target as gap {pi_target_dev:+.5f}")
306 print(f"Initial state: last policy-rate gap {100*last_i_gap:+.3f} quarterly pp")
307 print(table[table["horizon"].isin([1, 4, 8])
308         & table["variable"].isin(["pi", "i", "x"])].to_string(index=False))

```

```
309     print("Wrote forecast tables and three figures (fan chart, scenarios, policy shocks).")
310
311
312 if __name__ == "__main__":
313     main()
```

32 python/evaluate_forecasts.py

```

1  """Pseudo-out-of-sample forecast evaluation for the Chilean variables.
2
3  Expanding-window forecasts compare the fixed calibrated NK solution with
4  univariate AR(1), random-walk and VAR benchmarks at one- and four-quarter
5  horizons. The exercise is pseudo-real-time: revised data and the full-sample HP
6  output gap remain in the dataset, so it is not a genuine vintage-data test.
7  """
8
9  from __future__ import annotations
10
11 import matplotlib
12
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16 import pandas as pd
17 import statsmodels.api as sm
18 from statsmodels.tsa.api import VAR
19
20 from common import (
21     DATA_CLEAN,
22     FIGURES,
23     TABLES,
24     _state_coefficients,
25     calibration_rho,
26     complete_parameters,
27     ensure_directories,
28 )
29 from hybrid_solution import hybrid_params, solve_hybrid, hybrid_forecast
30
31
32 VARIABLES = ["x", "pi", "i"]
33 MODELS = ["NK", "NK_hibrido", "AR1", "Random walk", "VAR"]
34 HORIZONS = [1, 4]
35 MIN_TRAIN = 40
36 SCALES = {"x": 100.0, "pi": 400.0, "i": 400.0}
37 # The hybrid reduced form is calibration-fixed, so solve it once (gamma_pi=0.35,
38 # validated to reproduce Dynare's hybrid IRFs to machine precision).
39 HYBRID_SOLUTION = solve_hybrid(hybrid_params(0.35))
40
41
42 def ar1_forecast(series: np.ndarray, horizon: int) -> float:
43     y = series[1:]
44     x = sm.add_constant(series[:-1])
45     result = sm.OLS(y, x).fit()
46     value = float(series[-1])
47     for _ in range(horizon):
48         value = float(result.params[0] + result.params[1] * value)
49     return value
50
51
52 def nk_forecast(train: pd.DataFrame, horizon: int) -> np.ndarray:
53     params = complete_parameters({"rho_i": calibration_rho()})
54     a_x, a_pi, a_i = _state_coefficients(params)
55     means = train.mean()
56     lagged_rate_gap = float(train["i"].iloc[-1] - means["i"])
57     forecast = None
58     for _ in range(horizon):
59         forecast = np.array([a_x, a_pi, a_i]) * lagged_rate_gap
60         lagged_rate_gap = float(forecast[2])
61     return forecast + means[VARIABLES].to_numpy()
62
63
64 def nk_hybrid_forecast(train: pd.DataFrame, horizon: int) -> np.ndarray:
65     means = train.mean()
66     i_dev = float(train["i"].iloc[-1] - means["i"])
67     pi_dev = float(train["pi"].iloc[-1] - means["pi"])
68     forecast = hybrid_forecast(HYBRID_SOLUTION, i_dev, pi_dev, horizon)
69     return forecast + means[VARIABLES].to_numpy()
70
71
72 def var_forecast(train: pd.DataFrame, horizon: int) -> np.ndarray:
73     selection = VAR(train[VARIABLES]).select_order(maxlags=4, trend="c")
74     lags = max(1, int(selection.selected_orders.get("bic") or 1))
75     result = VAR(train[VARIABLES]).fit(lags, trend="c")
76     return result.forecast(train[VARIABLES].to_numpy()[-lags:], steps=horizon)[-1]
77
78
79 def main() -> None:
80     ensure_directories()
81     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
82     data = pd.DataFrame(
83         {
84             "date": macro["date"],
85             "x": macro["output_gap"],
86             "pi": macro["infl_q"],
87             "i": macro["i_q"],
88         }
89     )
90
91     records: list[dict] = []
92     for horizon in HORIZONS:
93         for origin in range(MIN_TRAIN - 1, len(data) - horizon):
94             train = data.iloc[: origin + 1]
95             target = data.iloc[origin + horizon]
96             predictions = {
97                 "NK": nk_forecast(train[VARIABLES], horizon),
98                 "NK_hibrido": nk_hybrid_forecast(train[VARIABLES], horizon),
99                 "AR1": np.array(
100                     [ar1_forecast(train[v].to_numpy(), horizon) for v in VARIABLES]
101                 ),
102                 "Random walk": train[VARIABLES].iloc[-1].to_numpy(),

```

```

103     }
104     try:
105         predictions["VAR"] = var_forecast(train, horizon)
106     except Exception: # noqa: BLE001
107         predictions["VAR"] = predictions["AR1"].copy()
108
109     for model, prediction in predictions.items():
110         for index, variable in enumerate(VARIABLES):
111             scale = SCALES[variable]
112             actual = float(target[variable] * scale)
113             forecast = float(prediction[index] * scale)
114             records.append(
115                 {
116                     "origin": train["date"].iloc[-1],
117                     "target_date": target["date"],
118                     "horizon": horizon,
119                     "model": model,
120                     "variable": variable,
121                     "actual": actual,
122                     "forecast": forecast,
123                     "error": actual - forecast,
124                     "absolute_error": abs(actual - forecast),
125                     "squared_error": (actual - forecast) ** 2,
126                 }
127             )
128
129 forecasts = pd.DataFrame(records)
130 forecasts.to_csv(TABLES / "forecast_oos_predictions.csv", index=False)
131 metrics = (
132     forecasts.groupby(["horizon", "variable", "model"], as_index=False)
133     .agg(
134         observations=("error", "size"),
135         bias=("error", "mean"),
136         mae=("absolute_error", "mean"),
137         mse=("squared_error", "mean"),
138     )
139 )
140 metrics["rmse"] = np.sqrt(metrics["mse"])
141 random_walk_rmse = metrics[metrics["model"] == "Random walk"][
142     ["horizon", "variable", "rmse"]
143 ].rename(columns={"rmse": "random_walk_rmse"})
144 metrics = metrics.merge(random_walk_rmse, on=["horizon", "variable"])
145 metrics["relative_rmse_vs_random_walk"] = (
146     metrics["rmse"] / metrics["random_walk_rmse"]
147 )
148 metrics.to_csv(TABLES / "forecast_oos_metrics.csv", index=False)
149
150 fig, axes = plt.subplots(2, 3, figsize=(14, 8), sharey=False)
151 colors = {
152     "NK": "#B45309",
153     "NK hibrido": "#B91C1C",
154     "AR1": "#1E4E79",
155     "Random walk": "#6B7280",
156     "VAR": "#16A34A",
157 }
158 for row, horizon in enumerate(HORIZONS):
159     for col, variable in enumerate(VARIABLES):
160         ax = axes[row, col]
161         sub = metrics[
162             (metrics["horizon"] == horizon) & (metrics["variable"] == variable)
163         ].set_index("model").reindex(MODELS)
164         ax.bar(
165             np.arange(len(MODELS)),
166             sub["rmse"],
167             color=[colors[m] for m in MODELS],
168         )
169         ax.set_xticks(np.arange(len(MODELS)))
170         ax.set_xticklabels(MODELS, rotation=25, ha="right", fontsize=8)
171         ax.set_title(f"{variable}: horizonte {horizon}")
172         ax.set_ylabel("RMSE (p.p.)" if variable != "x" else "RMSE (% PIB)")
173         ax.grid(alpha=0.2, axis="y")
174 fig.suptitle("Avaliacao pseudo-fora-da-amostra: modelo NK versus benchmarks")
175 fig.text(
176     0.01,
177     0.01,
178     "Janela expansiva, treino inicial de 40 trimestres. Dados revisados e hiato HP "
179     "de amostra completa: diagnostico pseudo-real-time, nao backtest em vintages.",
180     fontsize=8,
181 )
182 fig.tight_layout(rect=(0, 0.04, 1, 0.95))
183 fig.savefig(FIGURES / "forecast_oos_comparison.png", dpi=180)
184 plt.close(fig)
185
186 print(
187     metrics[
188         ["horizon", "variable", "model", "rmse", "mae",
189          "relative_rmse_vs_random_walk"]
190     ].to_string(index=False)
191 )
192
193
194 if __name__ == "__main__":
195     main()

```

Analise (SVAR, Bayes, resultados)

33 python/analyze_svar.py

```
1  """Estimate a recursive VAR and compare its monetary IRF with the NK model.
2
3  Identification uses the quarterly Cholesky ordering [x, pi, i]. Output and
4  inflation may affect the policy rate within the quarter, while a monetary shock
5  affects them with a lag. This is a transparent reduced-form robustness check,
6  not an incontrovertible identification of Chilean monetary shocks.
7  """
8
9  from __future__ import annotations
10
11  import matplotlib
12
13  matplotlib.use("Agg")
14  import matplotlib.pyplot as plt
15  import numpy as np
16  import pandas as pd
17  from statsmodels.tsa.api import VAR
18
19  from common import DATA_CLEAN, DYNARE_OUTPUTS, FIGURES, TABLES, ensure_directories
20
21
22  VARIABLES = ["x", "pi", "i"]
23  LABELS = {"x": "Hiato do produto", "pi": "Inflacao", "i": "Taxa de politica"}
24  SCALES = {"x": 100.0, "pi": 400.0, "i": 400.0}
25  MONETARY_IMPACT_ANNUAL_PP = 0.25
26  # Dynare exports horizons 0..19 (20 observations).
27  HORIZON = 19
28
29
30  def main() -> None:
31      ensure_directories()
32      observables = pd.read_csv(DATA_CLEAN / "chile_observables.csv")
33      data = observables[VARIABLES].astype(float)
34
35      selection = VAR(data).select_order(maxlags=4, trend="c")
36      selected_lag = max(1, int(selection.selected_orders.get("bic") or 1))
37      lag_candidates = []
38      for lag in range(1, 5):
39          candidate = VAR(data).fit(lag, trend="c")
40          try:
41              whiteness = float(candidate.test_whiteness(nlags=8).pvalue)
42          except Exception: # noqa: BLE001
43              whiteness = np.nan
44          lag_candidates.append((lag, candidate, whiteness))
45      acceptable = [item for item in lag_candidates if item[2] >= 0.05]
46      if acceptable:
47          estimated_lag, result, selected_whiteness = acceptable[0]
48          lag_reason = "smallest lag with residual-whiteness p>=0.05"
49      else:
50          estimated_lag = selected_lag
51          result = VAR(data).fit(estimated_lag, trend="c")
52          selected_whiteness = float(
53              result.test_whiteness(nlags=max(8, estimated_lag + 2)).pvalue
54          )
55          lag_reason = "BIC; no lag up to 4 passed residual-whiteness test"
56
57      orth = result.orth_ma_rep(HORIZON)
58      lower, upper = result.irf_errband_mc(
59          orth=True, repl=2000, steps=HORIZON, signif=0.10, seed=20260615
60      )
61      policy_index = VARIABLES.index("i")
62      impact_i = float(orth[0, policy_index, policy_index])
63      normalization = MONETARY_IMPACT_ANNUAL_PP / (SCALES["i"] * impact_i)
64
65      baseline = pd.read_csv(DYNARE_OUTPUTS / "baseline" / "irfs.csv")
66      dsge_impact = float(baseline.loc[0, "i_e_i"])
67      dsge_normalization = MONETARY_IMPACT_ANNUAL_PP / (
68          SCALES["i"] * dsge_impact
69      )
70
71      records: list[dict] = []
72      fig, axes = plt.subplots(1, 3, figsize=(14, 4.4), sharex=True)
73      horizons = np.arange(HORIZON + 1)
74      for response_index, variable in enumerate(VARIABLES):
75          scale = SCALES[variable]
76          point = orth[:, response_index, policy_index] * normalization * scale
77          lo = lower[:, response_index, policy_index] * normalization * scale
78          hi = upper[:, response_index, policy_index] * normalization * scale
79          dsge = (
80              baseline[f"{variable}_e_i"].to_numpy()[: HORIZON + 1]
81              * dsge_normalization
82              * scale
83          )
84          ax = axes[response_index]
85          ax.fill_between(horizons, lo, hi, color="#9DB9D1", alpha=0.5,
86                          label="SVAR: banda 90%")
87          ax.plot(horizons, point, color="#1E4E79", linewidth=2.2,
88                  label=f"SVAR recursivo, VAR({estimated_lag})")
89          ax.plot(horizons, dsge, color="#B45309", linestyle="--", linewidth=2.2,
90                  label="DSGE/NK")
91          ax.axhline(0, color="black", linewidth=0.7)
92          ax.set_title(LABELS[variable])
93          ax.set_xlabel("Trimestres")
94          ax.set_ylabel("p.p." if variable != "x" else "% do PIB")
95          ax.grid(alpha=0.25)
96          for h in horizons:
97              records.append(
98                  {
```

```

99         "horizon": int(h),
100         "variable": variable,
101         "svar_response": float(point[h]),
102         "svar_lower_90": float(lo[h]),
103         "svar_upper_90": float(hi[h]),
104         "dsge_response": float(dsge[h]),
105         "normalization": "policy-rate impact = +0.25 annual p.p.",
106         "ordering": "x,pi,i",
107     }
108 )
109
110 handles, labels = axes[0].get_legend_handles_labels()
111 fig.legend(handles, labels, ncol=3, loc="lower center", frameon=False)
112 fig.suptitle(
113     "Choque monetario contracionista: SVAR recursivo versus modelo NK"
114 )
115 fig.text(
116     0.01,
117     0.01,
118     f"SVAR com ordenacao [x, pi, i], VAR({estimated_lag}); bandas Monte Carlo de 90%. "
119     "Ambas as IRFs normalizadas para elevar a taxa em 0,25 p.p. anual no impacto.",
120     fontsize=8,
121 )
122 fig.tight_layout(rect=(0, 0.10, 1, 0.94))
123 fig.savefig(FIGURES / "svar_vs_dsge_irf.png", dpi=180)
124 plt.close(fig)
125
126 try:
127     whiteness_p = float(result.test_whiteness(nlags=max(8, estimated_lag + 2)).pvalue)
128 except Exception: # noqa: BLE001
129     whiteness_p = np.nan
130
131 try:
132     normality_p = float(result.test_normality().pvalue)
133 except Exception: # noqa: BLE001
134     normality_p = np.nan
135
136 diagnostics = pd.DataFrame(
137     [
138         {
139             "observations": int(result.nobs),
140             "selected_lags_bic": selected_lag,
141             "estimated_lags": estimated_lag,
142             "lag_choice_reason": lag_reason,
143             "aic": float(result.aic),
144             "bic": float(result.bic),
145             "hqic": float(result.hqic),
146             "stable": bool(result.is_stable()),
147             "max_inverse_root": float(np.max(np.abs(result.roots) * -1)),
148             "whiteness_test_pvalue": whiteness_p,
149             "normality_test_pvalue": normality_p,
150             "cholesky_ordering": "x,pi,i",
151             "monetary_shock": "third orthogonal innovation",
152             "impact_normalization_annual_pp": MONETARY_IMPACT_ANNUAL_PP,
153         }
154     ]
155 )
156 pd.DataFrame(records).to_csv(TABLES / "svar_vs_dsge_irfs.csv", index=False)
157 diagnostics.to_csv(TABLES / "svar_diagnostics.csv", index=False)
158
159 print(diagnostics.to_string(index=False))
160 for variable in VARIABLES:
161     frame = pd.DataFrame(records)
162     row = frame[(frame["variable"] == variable) & (frame["horizon"] == 4)].iloc[0]
163     print(
164         f"h=4 {variable}: SVAR={row['svar_response']:.4f}, "
165         f"DSGE={row['dsge_response']:.4f}"
166     )
167
168 if __name__ == "__main__":
169     main()

```

34 python/compare_bayesian_models.py

```

1  """Compare the forward-looking and hybrid NK models by marginal likelihood.
2
3  Both models use the same Chilean observables and common priors. Dynare finds
4  each posterior mode and Hessian. The script then applies the Laplace
5  approximation:
6
7      log p(y|M) ~ log p(y, theta_hat|M)
8                  + k/2 log(2*pi) - 1/2 log|H(theta_hat)|.
9
10 This is a formal marginal-data-density comparison, but an approximation around
11 one mode. It complements, rather than replaces, the full baseline MCMC.
12 """
13
14 from __future__ import annotations
15
16 import argparse
17 import os
18 import shutil
19 import subprocess
20 import sys
21 import tempfile
22 from pathlib import Path
23
24 import matplotlib.pyplot as plt
25 import numpy as np
26 import pandas as pd
27 from scipy.io import loadmat
28
29 from common import DATA_CLEAN, FIGURES, LOGS, ROOT, TABLES, find_dynare_matlab, find_octave
30
31
32 MODELS = {
33     "forward_nkpc": "nk_chile_estim.mod",
34     "hybrid_nkpc": "nk_chile_hybrid_estim.mod",
35 }
36
37
38 def octave_literal(path: Path) -> str:
39     return str(path).replace("\\", "/").replace("'", '"')
40
41
42 def kill_tree(proc: subprocess.Popen) -> None:
43     subprocess.run(
44         ["taskkill", "/F", "/T", "/PID", str(proc.pid)],
45         capture_output=True,
46         check=False,
47     )
48
49
50 def run_mode(
51     model_id: str,
52     filename: str,
53     work: Path,
54     octave: Path,
55     dynare: Path,
56     timeout: int,
57 ) -> tuple[Path, str]:
58     model_work = work / model_id
59     model_work.mkdir(parents=True, exist_ok=True)
60     shutil.copy(ROOT / "dynare" / filename, model_work / filename)
61     pd.read_csv(DATA_CLEAN / "chile_observables.csv")[[ "pi", "i", "x"]].to_csv(
62         model_work / "chile_observables_dynare.csv", index=False
63     )
64     expression = (
65         f"addpath('{octave_literal(dynare)}'); "
66         f"dynare('{filename}', 'noclearall', 'nolog');"
67     )
68     proc = subprocess.Popen(
69         [str(octave), "--quiet", "--eval", expression],
70         cwd=model_work,
71         env=os.environ.copy(),
72         text=True,
73         stdout=subprocess.PIPE,
74         stderr=subprocess.PIPE,
75     )
76     try:
77         stdout, stderr = proc.communicate(timeout=timeout)
78     except subprocess.TimeoutExpired:
79         kill_tree(proc)
80         stdout, stderr = proc.communicate()
81         raise TimeoutError(f"{model_id} posterior mode exceeded {timeout}s")
82     log = f"RC={proc.returncode}\nSTDOUT\n{stdout}\nSTDERR\n{stderr}\n"
83     mode_files = list(model_work.rglob("*.mode.mat"))
84     if proc.returncode != 0 or not mode_files:
85         raise RuntimeError(
86             f"{model_id} mode failed (rc={proc.returncode}); "
87             f"tail=[log[-1800:]]"
88         )
89     return mode_files[0], log
90
91
92 def laplace_summary(model_id: str, path: Path) -> dict[str, object]:
93     payload = loadmat(path)
94     mode = np.asarray(payload["xparam1"], dtype=float).ravel()
95     hessian = np.asarray(payload["hh"], dtype=float)
96     objective = float(np.asarray(payload["fval"]).ravel()[0])
97     sign, logdet = np.linalg.slogdet(hessian)
98     eigenvalues = np.linalg.eigvalsh(hessian)
99     if sign <= 0 or eigenvalues.min() <= 0:
100         raise ValueError(f"{model_id}: mode Hessian is not positive definite")
101     parameter_names = [
102         str(item[0]) if isinstance(item, np.ndarray) else str(item)

```



```

103     for item in np.asarray(payload["parameter_names"]).ravel()
104 ]
105 k = len(mode)
106 log_joint_mode = -objective
107 log_mdd = log_joint_mode + 0.5 * k * np.log(2 * np.pi) - 0.5 * logdet
108 gamma = np.nan
109 if "gamma_pi" in parameter_names:
110     gamma = float(mode[parameter_names.index("gamma_pi")])
111 return {
112     "model": model_id,
113     "parameters": k,
114     "log_joint_at_mode": log_joint_mode,
115     "hessian_log_determinant": logdet,
116     "laplace_log_marginal_likelihood": log_mdd,
117     "gamma_pi_mode": gamma,
118     "hessian_min_eigenvalue": float(eigenvalues.min()),
119     "method": "Laplace approximation at Dynare posterior mode",
120     "same_observables": "pi,i,x",
121     "observations": len(pd.read_csv(DATA_CLEAN / "chile_observables.csv")),
122 }
123
124
125 def make_figure(frame: pd.DataFrame) -> None:
126     best = frame["laplace_log_marginal_likelihood"].max()
127     relative = frame["laplace_log_marginal_likelihood"] - best
128     labels = frame["model"].map(
129         {"forward_nkpc": "NKPC prospectiva", "hybrid_nkpc": "NKPC hibrida"}
130     )
131     colors = ["#7A9E9F" if value < 0 else "#145DA0" for value in relative]
132     fig, ax = plt.subplots(figsize=(8.5, 5.2))
133     bars = ax.bar(labels, relative, color=colors, width=0.58)
134     ax.axhline(0, color="#333333", linewidth=0.9)
135     ax.set_ylabel("Log evidencia relativa ao melhor modelo")
136     ax.set_title("Comparacao bayesiana de modelos: evidencia marginal (Laplace)")
137     ax.grid(axis="y", alpha=0.25)
138     for bar, value in zip(bars, relative, strict=True):
139         ax.text(
140             bar.get_x() + bar.get_width() / 2,
141             value + (0.25 if value >= 0 else -0.25),
142             f"{value:.2f}",
143             ha="center",
144             va="bottom" if value >= 0 else "top",
145             fontweight="bold",
146         )
147     fig.text(
148         0.5,
149         0.01,
150         "Mesma amostra e mesmos observaveis. A evidencia penaliza o parametro extra "
151         "da Phillips hibrida; aproximacao local, nao estimativa MCMC da evidencia.",
152         ha="center",
153         fontsize=9,
154     )
155     fig.tight_layout(rect=(0, 0.06, 1, 1))
156     fig.savefig(FIGURES / "bayesian_model_comparison.png", dpi=180, bbox_inches="tight")
157     plt.close(fig)
158
159
160 def main() -> int:
161     parser = argparse.ArgumentParser()
162     parser.add_argument("--timeout", type=int, default=420)
163     parser.add_argument(
164         "--reuse-baseline",
165         action="store_true",
166         help="Reuse the existing baseline mode in the system temp directory.",
167     )
168     args = parser.parse_args()
169     octave = find_octave()
170     dynare = find_dynare_matlab()
171     if octave is None or dynare is None:
172         print("ERROR: Octave/Dynare not found.", file=sys.stderr)
173         return 1
174
175     work = Path(tempfile.gettempdir()) / "nk_model_comparison_work"
176     if work.exists():
177         shutil.rmtree(work, ignore_errors=True)
178     work.mkdir(parents=True)
179     logs: list[str] = []
180     summaries: list[dict[str, object]] = []
181
182     for model_id, filename in MODELS.items():
183         if model_id == "forward_nkpc" and args.reuse_baseline:
184             cached = (
185                 Path(tempfile.gettempdir())
186                 / "nk_estim_work"
187                 / "nk_chile_estim"
188                 / "Output"
189                 / "nk_chile_estim_mode.mat"
190             )
191             if cached.exists():
192                 mode_file = cached
193                 log = f"Reused baseline mode: {cached}\n"
194             else:
195                 mode_file, log = run_mode(
196                     model_id, filename, work, octave, dynare, args.timeout
197                 )
198             else:
199                 mode_file, log = run_mode(
200                     model_id, filename, work, octave, dynare, args.timeout
201                 )
202             logs.append(f"\n==== {model_id} =====\n{log}")
203             summaries.append(laplace_summary(model_id, mode_file))
204
205     frame = pd.DataFrame(summaries)
206     best = float(frame["laplace_log_marginal_likelihood"].max())
207     frame["log_bayes_factor_vs_best"] = (

```

```

208     frame["laplace_log_marginal_likelihood"] - best
209 )
210 weights = np.exp(frame["log_bayes_factor_vs_best"])
211 frame["posterior_model_probability_equal_prior"] = weights / weights.sum()
212 preferred = frame.loc[
213     frame["laplace_log_marginal_likelihood"].idxmax(), "model"
214 ]
215 frame["preferred_model"] = preferred
216 frame.to_csv(TABLES / "bayesian_model_comparison.csv", index=False)
217 make_figure(frame)
218 (LOGS / "bayesian_model_comparison.log").write_text(
219     "".join(logs), encoding="utf-8"
220 )
221 print(frame.to_string(index=False))
222 print(f"Preferred model under equal prior odds: {preferred}")
223 return 0
224
225
226 if __name__ == "__main__":
227     raise SystemExit(main())

```

35 python/analyze_model_results.py

```

1  """Build diagnostic tables that connect model outputs to the assignment.
2
3  The script compares theoretical and empirical moments, summarizes IRF impact and
4  convergence metrics, and checks the expected impact signs of the three structural
5  shocks. It also creates a model-versus-data moments figure.
6  """
7
8  from __future__ import annotations
9
10 import json
11 import math
12
13 import matplotlib
14
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 import numpy as np
18 import pandas as pd
19
20 from common import (
21     DATA_CLEAN,
22     DYNARE_OUTPUTS,
23     FIGURES,
24     SHOCK_STD,
25     TABLES,
26     _state_coefficients,
27     calibration_rho,
28     complete_parameters,
29     ensure_directories,
30 )
31
32 VARIABLES = ("x", "pi", "i")
33 LABELS = {"x": "Output gap", "pi": "Inflation", "i": "Policy rate"}
34
35
36 def acf1(values: pd.Series) -> float:
37     clean = pd.to_numeric(values, errors="coerce").dropna()
38     return float(clean.autocorr(lag=1)) if len(clean) > 2 else math.nan
39
40
41 def empirical_moment_rows(panel: pd.DataFrame, sample: str) -> list[dict]:
42     series = {
43         "x": panel["output_gap"],
44         "pi": panel["infl_q"],
45         "i": panel["i_q"],
46     }
47     rows = []
48     for variable, values in series.items():
49         clean = pd.to_numeric(values, errors="coerce").dropna()
50         rows.append(
51             {
52                 "source": "BCC data",
53                 "sample": sample,
54                 "variable": variable,
55                 "mean_pp_quarterly": 100.0 * float(clean.mean()),
56                 "std_dev_pp_quarterly": 100.0 * float(clean.std(ddof=1)),
57                 "variance": float(clean.var(ddof=1)),
58                 "autocorrelation_1": acf1(clean),
59                 "observations": int(len(clean)),
60             }
61         )
62     return rows
63
64
65 def model_moment_rows(moments: pd.DataFrame, scenario: str, sample: str) -> list[dict]:
66     selected = moments[moments["scenario"] == scenario]
67     rows = []
68     for _, row in selected.iterrows():
69         variable = str(row["variable"])
70         model_mean = float(row["mean"])
71         if variable == "i":
72             # Dynare reports the level r*; comparisons focus on cyclical deviations.
73             model_mean = 0.0
74         rows.append(
75             {
76                 "source": "Dynare model",
77                 "sample": sample,
78                 "variable": variable,
79                 "mean_pp_quarterly": 100.0 * model_mean,
80                 "std_dev_pp_quarterly": 100.0 * float(row["std_dev"]),
81                 "variance": float(row["variance"]),
82                 "autocorrelation_1": float(row["autocorrelation_1"]),
83                 "observations": np.nan,
84             }
85         )
86     return rows
87
88
89 def build_moment_comparison() -> pd.DataFrame:
90     panel = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
91     moments = pd.read_csv(TABLES / "moments_all_scenarios.csv")
92     dates = panel["date"].astype(str)
93     pandemic = dates.between("2020Q2", "2021Q4")
94
95     rows = empirical_moment_rows(panel, "2001Q1-2026Q1")
96     rows += empirical_moment_rows(
97         panel.loc[~pandemic].reset_index(drop=True),
98         "excluding 2020Q2-2021Q4",
99     )
100     rows += model_moment_rows(moments, "baseline", "rho_i estimated (0.934)")
101     rows += model_moment_rows(
102         moments,

```

```

103     "rho_calibrated_0p80",
104     "rho_i calibrated (0.80)",
105 )
106 table = pd.DataFrame(rows)
107 table.to_csv(TABLES / "moments_model_vs_data.csv", index=False)
108 return table
109
110
111 def plot_moments(table: pd.DataFrame) -> None:
112     order = [
113         "2001Q1-2026Q1",
114         "excluding 2020Q2-2021Q4",
115         "rho_i estimated (0.934)",
116         "rho_i calibrated (0.80)",
117     ]
118     colors = ["#111827", "#6B7280", "#1E4E79", "#FF6B35"]
119     fig, axes = plt.subplots(1, 2, figsize=(13, 5.2))
120     width = 0.19
121     positions = np.arange(len(VARIABLES))
122     for index, (sample, color) in enumerate(zip(order, colors)):
123         selected = table[table["sample"] == sample].set_index("variable")
124         offset = (index - 1.5) * width
125         axes[0].bar(
126             positions + offset,
127             selected.loc[list(VARIABLES), "std_dev_pp_quarterly"],
128             width=width,
129             label=sample,
130             color=color,
131         )
132         axes[1].bar(
133             positions + offset,
134             selected.loc[list(VARIABLES), "autocorrelation_1"],
135             width=width,
136             label=sample,
137             color=color,
138         )
139     axes[0].set_title("Quarterly standard deviations")
140     axes[0].set_ylabel("Percentage points")
141     axes[1].set_title("First-order autocorrelation")
142     axes[1].axhline(0, color="black", linewidth=0.7)
143     for ax in axes:
144         ax.set_xticks(positions, [LABELS[var] for var in VARIABLES])
145         ax.grid(axis="y", alpha=0.25)
146     axes[0].legend(fontsize=8)
147     fig.suptitle("Chile: theoretical moments versus empirical moments")
148     fig.tight_layout(rect=(0, 0, 1, 0.95))
149     fig.savefig(FIGURES / "moments_model_vs_data.png", dpi=180)
150     plt.close(fig)
151
152
153 def first_sign_change(frame: pd.DataFrame) -> float:
154     ordered = frame.sort_values("horizon")
155     initial = float(ordered.iloc[0]["response"])
156     if abs(initial) < 1e-14:
157         return math.nan
158     initial_sign = np.sign(initial)
159     changed = ordered[
160         (np.sign(ordered["response"]) != initial_sign)
161         & (ordered["response"].abs() > 1e-12)
162     ]
163     return float(changed.iloc[0]["horizon"]) if not changed.empty else math.nan
164
165
166 def build_irf_metrics() -> pd.DataFrame:
167     irfs = pd.read_csv(TABLES / "irfs_long.csv")
168     manifest = pd.read_csv(TABLES / "scenario_manifest.csv")
169     types = dict(zip(manifest["scenario"], manifest["scenario_type"]))
170     types["baseline"] = "baseline"
171     rows = []
172     for (scenario, variable, shock), frame in irfs.groupby(
173         ["scenario", "variable", "shock"]
174     ):
175         ordered = frame.sort_values("horizon")
176         peak_row = ordered.loc[ordered["response"].abs().idxmax()]
177         by_horizon = ordered.set_index("horizon")["response"]
178         rows.append(
179             {
180                 "scenario": scenario,
181                 "scenario_type": types.get(scenario, ""),
182                 "variable": variable,
183                 "shock": shock,
184                 "impact_pp": 100.0 * float(ordered.iloc[0]["response"]),
185                 "peak_response_pp": 100.0 * float(peak_row["response"]),
186                 "peak_abs_pp": 100.0 * abs(float(peak_row["response"])),
187                 "peak_horizon": int(peak_row["horizon"]),
188                 "cumulative_abs_pp": 100.0 * float(ordered["response"].abs().sum()),
189                 "first_sign_change_horizon": first_sign_change(ordered),
190                 "response_h4_pp": 100.0 * float(by_horizon.get(4, np.nan)),
191                 "response_h8_pp": 100.0 * float(by_horizon.get(8, np.nan)),
192                 "response_h19_pp": 100.0 * float(by_horizon.get(19, np.nan)),
193                 "source": str(ordered.iloc[0]["source"]),
194             }
195         )
196     table = pd.DataFrame(rows)
197     table.to_csv(TABLES / "irf_summary_metrics.csv", index=False)
198     return table
199
200
201 def build_sign_checks(metrics: pd.DataFrame) -> pd.DataFrame:
202     expected = {
203         ("e_x", "x"): 1,
204         ("e_x", "pi"): 1,
205         ("e_x", "i"): 1,
206         ("e_pi", "x"): -1,
207         ("e_pi", "pi"): 1,

```

```

208         ("e_pi", "i"): 1,
209         ("e_i", "x"): -1,
210         ("e_i", "pi"): -1,
211         ("e_i", "i"): 1,
212     }
213     baseline = metrics[metrics["scenario"] == "baseline"].copy()
214     baseline["expected_impact_sign"] = baseline.apply(
215         lambda row: expected[(row["shock"], row["variable"])], axis=1
216     )
217     baseline["observed_impact_sign"] = np.sign(baseline["impact_pp"]).astype(int)
218     baseline["impact_sign_matches"] = (
219         baseline["expected_impact_sign"] == baseline["observed_impact_sign"]
220     )
221     output = baseline[
222         [
223             "shock",
224             "variable",
225             "expected_impact_sign",
226             "observed_impact_sign",
227             "impact_sign_matches",
228             "impact_pp",
229             "first_sign_change_horizon",
230         ]
231     ]
232     output.to_csv(TABLES / "irf_sign_checks.csv", index=False)
233     return output
234
235
236 def build_policy_functions() -> pd.DataFrame:
237     params = complete_parameters({"rho_i": calibration_rho()})
238     a_x, a_pi, a_i = _state_coefficients(params)
239     sigma, beta, kappa = params["sigma"], params["beta"], params["kappa"]
240     rho_i, phi_pi, phi_x = params["rho_i"], params["phi_pi"], params["phi_x"]
241     c_i = a_x - (1.0 - a_pi) / sigma
242     structural = np.array(
243         [
244             [1.0, 0.0, -c_i],
245             [-kappa, 1.0, -beta * a_pi],
246             [-(1.0 - rho_i) * phi_x, -(1.0 - rho_i) * phi_pi, 1.0],
247         ]
248     )
249     impact = np.linalg.inv(structural)
250     transition = np.array([a_x, a_pi, a_i])
251     drivers = [
252         ("constant", np.array([0.0, 0.0, params["rstar"]])),
253         ("i(-1)", transition),
254         ("e_x", impact[:, 0]),
255         ("e_pi", impact[:, 1]),
256         ("e_i", impact[:, 2]),
257     ]
258     rows = []
259     for driver, coefficients in drivers:
260         for variable, coefficient in zip(VARIABLES, coefficients):
261             rows.append(
262                 {
263                     "driver": driver,
264                     "variable": variable,
265                     "coefficient_unit_shock": float(coefficient),
266                     "coefficient_calibrated_shock": (
267                         float(coefficient) * SHOCK_STD[driver]
268                         if driver in SHOCK_STD
269                         else float(coefficient)
270                     ),
271                 }
272             )
273     table = pd.DataFrame(rows)
274     table.to_csv(TABLES / "policy_transition_coefficients.csv", index=False)
275     return table
276
277
278 def build_correlation_comparison() -> pd.DataFrame:
279     panel = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
280     empirical = pd.DataFrame(
281         {
282             "x": panel["output_gap"],
283             "pi": panel["infl_q"],
284             "i": panel["i_q"],
285         }
286     ).corr()
287     rows = []
288     for variable in VARIABLES:
289         for other in VARIABLES:
290             rows.append(
291                 {
292                     "source": "BCCh data",
293                     "variable": variable,
294                     "other_variable": other,
295                     "correlation": float(empirical.loc[variable, other]),
296                 }
297             )
298     model_path = DYNARE_OUTPUTS / "baseline" / "correlations.csv"
299     if model_path.exists():
300         model = pd.read_csv(model_path).set_index("variable")
301         for variable in VARIABLES:
302             for other in VARIABLES:
303                 rows.append(
304                     {
305                         "source": "Dynare baseline",
306                         "variable": variable,
307                         "other_variable": other,
308                         "correlation": float(model.loc[variable, other]),
309                     }
310                 )
311     table = pd.DataFrame(rows)
312     table.to_csv(TABLES / "correlations_model_vs_data.csv", index=False)

```

```

313     return table
314
315
316 def main() -> None:
317     ensure_directories()
318     metadata = json.loads(
319         (DATA_CLEAN / "dataset_metadata.json").read_text(encoding="utf-8")
320     )
321     if bool(metadata.get("is_synthetic", True)):
322         print("WARNING: empirical moment rows use the labelled synthetic fallback.")
323     moment_table = build_moment_comparison()
324     plot_moments(moment_table)
325     metrics = build_irf_metrics()
326     signs = build_sign_checks(metrics)
327     policy = build_policy_functions()
328     correlations = build_correlation_comparison()
329     print(f"Moment comparison rows: {len(moment_table)}")
330     print(f"IRF metric rows: {len(metrics)}")
331     print(f"Policy-function rows: {len(policy)}")
332     print(f"Correlation rows: {len(correlations)}")
333     print(
334         "Baseline impact sign checks: "
335         f"{int(signs['impact_sign_matches'].sum())}/{len(signs)}"
336     )
337
338
339 if __name__ == "__main__":
340     main()

```

36 python/analyze_macro_extensions.py

```

1  """Plot and summarize the open-economy and hybrid-Phillips extensions."""
2
3  from __future__ import annotations
4
5  import matplotlib
6
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11
12 from common import DYNARE_OUTPUTS, FIGURES, TABLES, ensure_directories
13
14
15 def load_irfs(scenario: str) -> pd.DataFrame:
16     return pd.read_csv(DYNARE_OUTPUTS / scenario / "irfs.csv")
17
18
19 def main() -> None:
20     ensure_directories()
21     open_irf = load_irfs("open_economy")
22     hybrid_irf = load_irfs("hybrid_nkpc")
23     baseline = load_irfs("baseline")
24     h = open_irf["horizon"].to_numpy()
25
26     variables = ["x", "pi", "i", "q", "copper"]
27     shocks = ["e_i", "e_q", "e_copper"]
28     labels = {
29         "x": "Hiato",
30         "pi": "Inflacao",
31         "i": "Juro",
32         "q": "Cambio real/nominal gap",
33         "copper": "Cobre",
34         "e_i": "Choque monetario",
35         "e_q": "Choque cambial/risco",
36         "e_copper": "Choque de cobre",
37     }
38     scales = {"x": 100, "pi": 400, "i": 400, "q": 100, "copper": 100}
39     fig, axes = plt.subplots(5, 3, figsize=(14, 14), sharex=True)
40     for row, variable in enumerate(variables):
41         for col, shock in enumerate(shocks):
42             ax = axes[row, col]
43             ax.plot(h, open_irf[f"{variable}_{shock}"] * scales[variable],
44                    color="#1E4E79", linewidth=2)
45             ax.axhline(0, color="black", linewidth=0.6)
46             if row == 0:
47                 ax.set_title(labels[shock])
48             if col == 0:
49                 ax.set_ylabel(labels[variable])
50             ax.grid(alpha=0.2)
51     fig.suptitle("Modelo NK aberto: transmissao monetaria, cambial e do cobre")
52     fig.text(
53         0.01,
54         0.01,
55         "Extensao calibrada ilustrativa. q>0 representa depreciacao; respostas de x/q/cobre "
56         "em %, pi/i em p.p. anualizados.",
57         fontsize=8,
58     )
59     fig.tight_layout(rect=(0, 0.03, 1, 0.96))
60     fig.savefig(FIGURES / "open_economy_irfs.png", dpi=180)
61     plt.close(fig)
62
63     fig, axes = plt.subplots(2, 3, figsize=(14, 7), sharex=True)
64     for col, shock in enumerate(["e_x", "e_pi", "e_i"]):
65         for row, variable in enumerate(["pi", "x"]):
66             ax = axes[row, col]
67             scale = 400 if variable == "pi" else 100
68             ax.plot(
69                 baseline["horizon"],
70                 baseline[f"{variable}_{shock}"] * scale,
71                 color="#6B7280",
72                 linewidth=2,
73                 label="NK prospectivo",
74             )
75             ax.plot(
76                 hybrid_irf["horizon"],
77                 hybrid_irf[f"{variable}_{shock}"] * scale,
78                 color="#B45309",
79                 linestyle="--",
80                 linewidth=2,
81                 label="NKPC hibrida",
82             )
83             ax.axhline(0, color="black", linewidth=0.6)
84             ax.set_title(f"{variable} a {shock}")
85             ax.grid(alpha=0.2)
86     axes[0, 0].legend(fontsize=8)
87     fig.suptitle("Persistencia intrinseca: Phillips prospectiva versus hibrida")
88     fig.tight_layout(rect=(0, 0, 1, 0.95))
89     fig.savefig(FIGURES / "hybrid_nkpc_irfs.png", dpi=180)
90     plt.close(fig)
91
92     fig, axes = plt.subplots(1, 3, figsize=(13, 4.2), sharex=True)
93     for ax, variable in zip(axes, ["x", "pi", "i"]):
94         scale = 100 if variable == "x" else 400
95         ax.plot(
96             baseline["horizon"],
97             baseline[f"{variable}_e_i"] * scale,
98             color="#6B7280",
99             linewidth=2,
100            label="Fechado",
101        )
102        ax.plot(

```

```

103         open_irf["horizon"],
104         open_irf[f"{variable}_e_i"] * scale,
105         color="#1E4E79",
106         linewidth=2,
107         linestyle="--",
108         label="Aberto",
109     )
110     ax.axhline(0, color="black", linewidth=0.6)
111     ax.set_title(labels[variable])
112     ax.grid(alpha=0.2)
113     axes[0].legend()
114     fig.suptitle("Choque monetário: modelo fechado versus economia aberta")
115     fig.tight_layout(rect=(0, 0, 1, 0.93))
116     fig.savefig(FIGURES / "open_vs_closed_monetary_irf.png", dpi=180)
117     plt.close(fig)
118
119     rows = []
120     for scenario, frame in [
121         ("baseline", baseline),
122         ("hybrid_nkpc", hybrid_irf),
123         ("open_economy", open_irf),
124     ]:
125         available_shocks = ["e_x", "e_pi", "e_i"]
126         if scenario == "open_economy":
127             available_shocks += ["e_q", "e_copper"]
128         for variable in ["x", "pi", "i"]:
129             scale = 100 if variable == "x" else 400
130             for shock in available_shocks:
131                 values = frame[f"{variable}_{shock}"].to_numpy() * scale
132                 rows.append(
133                     {
134                         "scenario": scenario,
135                         "variable": variable,
136                         "shock": shock,
137                         "impact": float(values[0]),
138                         "peak_abs": float(np.max(np.abs(values))),
139                         "cumulative_abs_20q": float(np.sum(np.abs(values))),
140                         "response_h4": float(values[4]),
141                     }
142                 )
143     pd.DataFrame(rows).to_csv(TABLES / "macro_extension_irf_metrics.csv", index=False)
144     print("Wrote macro-extension figures and metrics.")
145
146
147 if __name__ == "__main__":
148     main()

```

Nucleos da inflacao e mercado de trabalho

37 python/build_inflation_cores.py

```
1  """Decompose Chilean CPI (IPC) inflation by core/sub-index, from public FRED data.
2
3  FRED OECD series (no credentials). The component indices are only published to
4  ~2023 (energy/services/core) so the analysis covers the build-up and the 2021-23
5  surge but not the most recent disinflation -- stated honestly on the figure.
6
7  Produces:
8  * data/clean/chile_inflation_cores.csv -- YoY % inflation by component;
9  * ipc_cores_panel.png -- one panel per core vs the 3% target and 2-4% band;
10 * ipc_cores_drivers.png -- what drove the surge: components through 2018-2023 +
11   the peak-by-component ranking (what was most impactful).
12 """
13
14 from __future__ import annotations
15
16 import io
17 import urllib.request
18
19 import matplotlib
20
21 matplotlib.use("Agg")
22 import matplotlib.pyplot as plt
23 import numpy as np
24 import pandas as pd
25
26 from common import DATA_CLEAN, FIGURES, ensure_directories
27
28 FRED_CSV = "https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}"
29 SERIES = {
30     "Cheia (todos os itens)": "CHLCPIALLMINMEI",
31     "Nucleo (s/ alim. e energia)": "CHLCPICORMINMEI",
32     "Energia": "CHLCPIENGMINMEI",
33     "Servicos": "CHLCPCRSE01IXOBM",
34 }
35 COLORS = {
36     "Cheia (todos os itens)": "#111827",
37     "Nucleo (s/ alim. e energia)": "#1E4E79",
38     "Energia": "#B45309",
39     "Servicos": "#16A34A",
40 }
41 TARGET, BAND = 3.0, (2.0, 4.0)
42
43
44 def download_fred(series_id: str) -> pd.Series:
45     url = FRED_CSV.format(series_id=series_id)
46     with urllib.request.urlopen(url, timeout=60) as response:
47         raw = response.read().decode("utf-8", "replace")
48     frame = pd.read_csv(io.StringIO(raw))
49     frame.columns = ["date", "value"]
50     frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
51     frame["value"] = pd.to_numeric(frame["value"], errors="coerce")
52     frame = frame.dropna()
53     return pd.Series(frame["value"].to_numpy(), index=pd.DatetimeIndex(frame["date"]))
54
55
56 def main() -> None:
57     ensure_directories()
58     yoy = {}
59     for name, sid in SERIES.items():
60         index = download_fred(sid).sort_index()
61         # 12-month change of the monthly index = annualised (YoY) inflation rate.
62         yoy[name] = (index / index.shift(12) - 1.0) * 100.0
63     data = pd.DataFrame(yoy).dropna(how="all")
64     data.index.name = "date"
65     data.to_csv(DATA_CLEAN / "chile_inflation_cores.csv")
66     last_full = data["Cheia (todos os itens)"].dropna().index.max()
67
68     # ----- #
69     # Figure 1: one panel per core vs the target band. #
70     # ----- #
71     fig, axes = plt.subplots(2, 2, figsize=(12.5, 7.2), sharex=True)
72     for ax, name in zip(axes.ravel(), SERIES):
73         series = data[name].dropna().loc["2000-01-01":] # inflation-targeting era only
74         ax.axhspan(BAND[0], BAND[1], color="#FDE68A", alpha=0.35, label="Banda 2-4%")
75         ax.axhline(TARGET, color="#B45309", linestyle="--", linewidth=1, label="Meta 3%")
76         ax.plot(series.index, series.values, color=COLORS[name], linewidth=1.8)
77         latest = series.iloc[-1]
78         inside = BAND[0] <= latest <= BAND[1]
79         ax.set_title(f"{name} -- ultimo {latest:.1f}% "
80                     f"({'dentro' if inside else 'fora'} da banda)", fontsize=10)
81         ax.set_ylabel("% a.a. (YoY)")
82         ax.grid(alpha=0.25)
83     axes[0, 0].legend(fontsize=8, loc="upper left")
84     fig.suptitle("Inflacao do IPC chileno por nucleo (variacao anual) vs meta de 3%")
85     fig.text(0.01, 0.01, f"Fonte: OECD via FRED. Componentes ate {last_full:%Y-%m} "
86                      f"(OECD descontinuou; perde a desinflacao recente). Variacao interanual = anualizada.",
87            fontsize=7.5)
88     fig.tight_layout(rect=(0, 0.03, 1, 0.96))
89     fig.savefig(FIGURES / "ipc_cores_panel.png", dpi=170)
90     plt.close(fig)
91
92     # ----- #
93     # Figure 2: what drove the surge (2018-2023) + peak ranking. #
94     # ----- #
95     window = data.loc["2018-01-01":]
96     fig, (axL, axR) = plt.subplots(1, 2, figsize=(13.5, 5.0),
97                                   gridspec_kw={"width_ratios": [2.0, 1.0]})
98     for name in SERIES:
```

```

99         s = window[name].dropna()
100         axL.plot(s.index, s.values, color=COLORS[name], linewidth=2.0, label=name)
101         axL.axhspan(BAND[0], BAND[1], color="#FDE68A", alpha=0.3)
102         axL.axhline(TARGET, color="#B45309", linestyle="--", linewidth=1)
103         axL.set_title("0 surto de 2021--2023 por nucleo")
104         axL.set_ylabel("Inflacao anual (%)")
105         axL.grid(alpha=0.25)
106         axL.legend(fontsize=8)
107
108     # Peak YoY during 2021-2023 by component = "what was most impactful".
109     surge = data.loc["2021-01-01":"2023-12-31"]
110     peaks = {name: float(surge[name].max()) for name in SERIES}
111     order = sorted(peaks, key=peaks.get)
112     axR.barh(range(len(order)), [peaks[n] for n in order],
113              color=[COLORS[n] for n in order])
114     axR.set_yticks(range(len(order)))
115     axR.set_yticklabels([n.split(" ")[0] for n in order], fontsize=8)
116     axR.axvline(TARGET, color="#B45309", linestyle="--", linewidth=1)
117     axR.set_title("Pico de inflacao 2021--23")
118     axR.set_xlabel("% a.a.")
119     axR.grid(alpha=0.25, axis="x")
120     for i, n in enumerate(order):
121         axR.text(peaks[n], i, f"{peaks[n]:.0f}%", va="center", fontsize=8)
122
123     fig.suptitle("Quem mais empurrou a inflacao chilena: energia lidera o pico, nucleo/servicos sao persistentes")
124     fig.tight_layout(rect=(0, 0, 1, 0.94))
125     fig.savefig(FIGURES / "ipc_cores_drivers.png", dpi=170)
126     plt.close(fig)
127
128     print("Peak YoY 2021-23 by component:")
129     for n in reversed(order):
130         print(f"    {n}: {peaks[n]:.1f}%")
131     print(f"Wrote chile_inflation_cores.csv, ipc_cores_panel.png, ipc_cores_drivers.png "
132           f"through {last_full:%Y-%m}")
133
134
135 if __name__ == "__main__":
136     main()

```

38 python/build_inflation_cores_bcch.py

```
1 """Chilean CPI cores from the BCCh (real, recent: base 2023=100, up to 2026).
2
3 Pulls the BCCh analytical CPI breakdown (general, sin volatiles, goods/services
4 sin volatiles, volatile, volatile food/energy) as monthly variations (MoM, %)
5 from the SieteRestWS, then builds the central-bank "inflation momentum" view per
6 core:
7
8 * MoM (monthly change, annualised) -> bars
9 * MM3M (3-month change, annualised) -> line
10 * MM6M (6-month change, annualised) -> line
11 * YoY (12-month change) -> line
12
13 All four are expressed as annual % so they are directly comparable and sit
14 against the 2-4% tolerance band (in/out of target). This replaces the FRED cores
15 (frozen at 2023-12) with real data through the latest BCCh release.
16
17 Credentials: env BCCH_USER/BCCH_PASS or data/raw/bcch_credentials.json.
18 Outputs: data/clean/chile_inflation_cores_bcch.csv, ipc_cores_momentum.png.
19 """
20
21 from __future__ import annotations
22
23 import matplotlib
24
25 matplotlib.use("Agg")
26 import matplotlib.pyplot as plt
27 import numpy as np
28 import pandas as pd
29
30 from common import DATA_CLEAN, FIGURES, ensure_directories
31 from build_chile_dataset import fetch_series, get_credentials
32
33 # Monthly CPI variation (MoM, %) series ids -- filled from discover_ipc_series.py.
34 # label -> F074.IPC... id
35 MAPPING: dict[str, str] = {
36     "Cheia (IPC geral)": "F074.IPC.VAR.Z.EP23.C.M",
37     "Nucleo (SAE)": "F074.IPCSAE.VAR.Z.EP23.Z.M",
38     "Servicos": "F074.IPCS.VAR.Z.EP23.Z.M",
39     "Bens": "F074.IPCB.VAR.Z.EP23.Z.M",
40     "Energia": "F074.IPCE.VAR.Z.EP23.Z.M",
41     "Alimentos": "F074.IPCA.VAR.Z.EP23.Z.M",
42 }
43 COLORS = {
44     "Cheia (IPC geral)": "#111827",
45     "Nucleo (SAE)": "#1E4E79",
46     "Servicos": "#16A34A",
47     "Bens": "#7C3AED",
48     "Energia": "#B45309",
49     "Alimentos": "#0891B2",
50 }
51 TARGET, BAND = 3.0, (2.0, 4.0)
52 START_PLOT = "2021-01-01"
53
54
55 def momentum(mom_pct: pd.Series) -> pd.DataFrame:
56     """Monthly variation (MoM, %), its 3/6-month moving averages (%/mo), YoY (%).
57
58     MoM/MM3M/MM6M stay on the monthly scale (left axis); YoY is the accumulated
59     12-month change on the annual scale (right axis). Annualising single monthly
60     prints is avoided -- it explodes for volatile cores (energy tariff months).
61     """
62     s = mom_pct.sort_index()
63     f = 1.0 + s / 100.0 # gross monthly factors
64     out = pd.DataFrame(index=s.index)
65     out["MoM"] = s # monthly variation, %
66     out["MM3M"] = s.rolling(3).mean() # 3-month moving average, %/mo
67     out["MM6M"] = s.rolling(6).mean() # 6-month moving average, %/mo
68     out["YoY"] = (f.rolling(12).apply(np.prod, raw=True) - 1.0) * 100.0 # 12-month, %
69     return out
70
71
72 def main() -> None:
73     ensure_directories()
74     creds = get_credentials()
75     if not creds:
76         raise SystemExit("No BCCh credentials (set BCCH_USER/BCCH_PASS or "
77                         "data/raw/bcch_credentials.json).")
78     user, password = creds
79     if len(MAPPING) < 2:
80         raise SystemExit("Fill MAPPING with the core series ids first "
81                         "(run discover_ipc_series.py).")
82
83     cores = {label: momentum(fetch_series(sid, user, password, first="2018-01-01"))
84             for label, sid in MAPPING.items()}
85
86     # Tidy CSV: every core x metric.
87     wide = pd.concat({label: df for label, df in cores.items()}, axis=1)
88     wide.index.name = "date"
89     wide.to_csv(DATA_CLEAN / "chile_inflation_cores_bcch.csv")
90     last = max(df["YoY"].dropna().index.max() for df in cores.values())
91
92     labels = list(cores)
93     ncol = 3
94     nrow = int(np.ceil(len(labels) / ncol))
95     fig, axes = plt.subplots(nrow, ncol, figsize=(5.6 * ncol, 3.4 * nrow),
96                             sharex=True)
97     axes = np.atleast_1d(axes).ravel()
98     handles: list = []
99     hlabels: list[str] = []
100     for ax, label in zip(axes, labels):
101         df = cores[label].loc[START_PLOT:]
102         color = COLORS.get(label, "#1E4E79")
```

```

103     # Left axis (monthly %): MoM bars + 3/6-month moving averages.
104     ax.axhline(0, color="#9CA3AF", lw=0.6, zorder=1)
105     b = ax.bar(df.index, df["MoM"], width=22, color=color, alpha=0.28,
106               label="MoM", zorder=2)
107     l3, = ax.plot(df.index, df["MM3M"], color=color, lw=1.6, label="MM3M",
108                  zorder=3)
109     l6, = ax.plot(df.index, df["MM6M"], color=color, lw=1.2, ls=(0, (4, 2)),
110                  label="MM6M", zorder=3)
111     ax.tick_params(axis="y", labels=7, colors=color)
112     ax.tick_params(axis="x", labels=8)
113     # Right axis (annual %): YoY vs the 2-4% tolerance band.
114     ax2 = ax.twinx()
115     ax2.axhspan(BAND[0], BAND[1], color="#FDE68A", alpha=0.40, zorder=0)
116     ax2.axhline(TARGET, color="#B45309", ls="--", lw=0.8, zorder=1)
117     ly, = ax2.plot(df.index, df["YoY"], color="#11827", lw=1.9, label="YoY",
118                   zorder=4)
119     ax2.tick_params(axis="y", labels=7)
120     yoy_last = df["YoY"].dropna().iloc[-1]
121     inside = BAND[0] <= yoy_last <= BAND[1]
122     ax.set_title(f"{label} -- YoY {yoy_last:.1f}% "
123                 f"({'dentro' if inside else 'fora'} da banda)", fontsize=9.5)
124     ax.grid(alpha=0.18)
125     if not handles:
126         handles = [b, l3, l6, ly]
127         hlabels = ["MoM (mensal, esq.)", "MM3M (esq.)", "MM6M (esq.)",
128                  "YoY (anual, dir.)"]
129     for ax in axes[len(labels):]:
130         ax.set_visible(False)
131     fig.legend(handles, hlabels, loc="lower center", fontsize=9, ncol=4,
132               bbox_to_anchor=(0.5, 0.03), frameon=False)
133     fig.suptitle("IPC chileno por nucleo -- barras: MoM e linhas finas MM3M/MM6M "
134                 "(eixo esq., %/mes)\n* linha preta: YoY (eixo dir., % a.a.) vs banda "
135                 "de tolerancia 2-4%", fontsize=11.5)
136     fig.text(0.01, 0.004, "Fonte: Banco Central de Chile -- IPC Analiticos empalmados "
137               f"({cifras oficiais INE), base 2023=100, SieteRestWS. Ultima obs.: {last:%Y-%m}.",
138             fontsize=7.5)
139     fig.tight_layout(rect=(0, 0.06, 1, 0.93))
140     fig.savefig(FIGURES / "ipc_cores_momentum.png", dpi=170)
141     plt.close(fig)
142
143     # ----- #
144     # Figure 2: drivers -- YoY timeline + recent momentum (MM3M) ranking. #
145     # ----- #
146     timeline = [l for l in ("Cheia (IPC geral)", "Nucleo (SAE)", "Energia",
147                            "Servicos") if l in cores]
148     fig, (axL, axR) = plt.subplots(1, 2, figsize=(13.5, 5.0),
149                                   gridspec_kw={"width_ratios": [2.1, 1.0]})
150     for label in timeline:
151         s = cores[label]["YoY"].loc[START_PLOT:].dropna()
152         axL.plot(s.index, s.values, color=COLORS.get(label, "#1E4E79"), lw=2.0,
153                 label=label)
154         axL.axhspan(BAND[0], BAND[1], color="#FDE68A", alpha=0.30)
155         axL.axhline(TARGET, color="#B45309", ls="--", lw=1.0)
156         axL.set_title("Inflacao anual (YoY) por nucleo")
157         axL.set_ylabel("% a.a.")
158         axL.grid(alpha=0.25)
159         axL.legend(fontsize=8)
160         latest_mm3 = {l: float(cores[l]["MM3M"].dropna().iloc[-1]) for l in cores}
161         order = sorted(latest_mm3, key=latest_mm3.get)
162         axR.barh(range(len(order)), [latest_mm3[l] for l in order],
163                 color=[COLORS.get(l, "#1E4E79") for l in order])
164         axR.set_yticks(range(len(order)))
165         axR.set_yticklabels(order, fontsize=8)
166         axR.axvline(0, color="#9CA3AF", lw=0.6)
167         axR.set_title(f"Momentum recente -- MM3M (%/mes), {last:%Y-%m}")
168         axR.set_xlabel("%/mes")
169         axR.grid(alpha=0.25, axis="x")
170         for i, l in enumerate(order):
171             axR.text(latest_mm3[l], i, f"{latest_mm3[l]:.1f}", va="center", fontsize=8)
172         fig.suptitle("Quem empurra a inflacao chilena: a energia lidera o momentum; "
173                     "servicos e o persistente")
174         fig.tight_layout(rect=(0, 0, 1, 0.94))
175         fig.savefig(FIGURES / "ipc_cores_drivers_bcch.png", dpi=170)
176         plt.close(fig)
177
178     print(f"Wrote chile_inflation_cores_bcch.csv + ipc_cores_momentum.png "
179          f"+ ipc_cores_drivers_bcch.png (through {last:%Y-%m}).")
180     for label, df in cores.items():
181         print(f"{label:28s} YoY {df['YoY'].dropna().iloc[-1]:5.1f}% "
182               f"MM3M {df['MM3M'].dropna().iloc[-1]:5.1f}%")
183
184
185 if __name__ == "__main__":
186     main()

```

39 python/build_labor_activity.py

```

1  """Chilean activity and labour-market analysis from public FRED data, with an
2  Atlanta-Fed-style labour-market spider chart.
3
4  FRED has no Chilean vacancies/hires/quits (JOLTS is US-only), so the spider uses
5  the available Chilean equivalents: unemployment, youth unemployment, employment
6  rate, participation, industrial production and the model's (Kalman) output gap --
7  each as a percentile of its own history, oriented so OUTWARD = stronger labour
8  market/activity, current versus pre-pandemic (2019).
9
10 Outputs: labor_activity_dashboard.png, labor_spider.png, data/clean/chile_labor.csv.
11 """
12
13 from __future__ import annotations
14
15 import io
16 import urllib.request
17
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import numpy as np
23 import pandas as pd
24
25 from common import DATA_CLEAN, FIGURES, ensure_directories
26
27 FRED_CSV = "https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}"
28 SERIES = {
29     "unemployment": "LRHUTTTTCLM156S", # unemployment rate, %
30     "youth_unemp": "SLUEM1524ZSCHL", # youth (15-24) unemployment, %
31     "employment": "LREM64TTCLM156S", # employment rate 15-64, %
32     "participation": "LRACTTTTCLM156S", # participation rate, %
33     "industrial": "CHLPROINDMISMEI", # industrial production index
34 }
35
36
37 def download_fred(series_id: str) -> pd.Series:
38     with urllib.request.urlopen(FRED_CSV.format(series_id=series_id), timeout=60) as resp:
39         raw = resp.read().decode("utf-8", "replace")
40         frame = pd.read_csv(io.StringIO(raw))
41         frame.columns = ["date", "value"]
42         frame["date"] = pd.to_datetime(frame["date"], errors="coerce")
43         frame["value"] = pd.to_numeric(frame["value"], errors="coerce")
44         frame = frame.dropna()
45         return pd.Series(frame["value"].to_numpy(), index=pd.DatetimeIndex(frame["date"])).sort_index()
46
47
48 def to_quarterly(series: pd.Series) -> pd.Series:
49     q = series.resample("QS").mean()
50     q.index = q.index.to_period("Q").astype(str)
51     return q
52
53
54 def main() -> None:
55     ensure_directories()
56     raw = {name: download_fred(sid) for name, sid in SERIES.items()}
57     quarterly = pd.DataFrame({name: to_quarterly(s) for name, s in raw.items()})
58     quarterly["industrial_yoy"] = (quarterly["industrial"] /
59                                   quarterly["industrial"].shift(4) - 1.0) * 100.0
60
61     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv").set_index("date")
62     quarterly["output_gap"] = macro["output_gap"] * 100.0 # Kalman gap, %
63     quarterly = quarterly.loc["2001Q1:"]
64     quarterly.index.name = "date"
65     quarterly.to_csv(DATA_CLEAN / "chile_labor.csv")
66     dates = quarterly.index.tolist()
67     xt = list(range(0, len(dates), 8))
68
69     # ----- #
70     # Figure 1: activity + labour dashboard.
71     # ----- #
72     fig, axes = plt.subplots(1, 3, figsize=(14, 4.6))
73     a = axes[0]
74     a.plot(range(len(dates)), quarterly["unemployment"], color="#B91C1C", linewidth=2, label="Desemprego")
75     a.plot(range(len(dates)), quarterly["youth_unemp"], color="#F59E0B", linewidth=1.6,
76            linestyle="--", label="Desemprego jovem")
77     a.set_title("Desemprego (%)"); a.legend(fontsize=8); a.grid(alpha=0.25)
78     b = axes[1]
79     b.plot(range(len(dates)), quarterly["employment"], color="#1E4E79", linewidth=2, label="Taxa de emprego")
80     b.plot(range(len(dates)), quarterly["participation"], color="#16A34A", linewidth=2, label="Participacao")
81     b.set_title("Emprego e participacao (%)"); b.legend(fontsize=8); b.grid(alpha=0.25)
82     c = axes[2]
83     c.axhline(0, color="black", linewidth=0.7)
84     c.plot(range(len(dates)), quarterly["output_gap"], color="#7C3AED", linewidth=2, label="Hiato (Kalman)")
85     c.plot(range(len(dates)), quarterly["industrial_yoy"], color="#6B7280", linewidth=1.4,
86            linestyle="--", label="Prod. industrial (YoY)")
87     c.set_title("Atividade (%)"); c.legend(fontsize=8); c.grid(alpha=0.25)
88     for ax in axes:
89         ax.set_xticks(xt); ax.set_xticklabels([dates[t] for t in xt], rotation=70, fontsize=7)
90     fig.suptitle("Chile: mercado de trabalho e atividade (FRED + hiato do modelo)")
91     fig.text(0.01, 0.01, "Fonte: OECD via FRED; hiato pelo filtro de Kalman. Sem dados de vagas/"
92            "admissoes para o Chile. Okun: desemprego move-se inversamente ao hiato.", fontsize=7.5)
93     fig.tight_layout(rect=(0, 0.03, 1, 0.95))
94     fig.savefig(FIGURES / "labor_activity_dashboard.png", dpi=170)
95     plt.close(fig)
96
97     # ----- #
98     # Figure 2: Atlanta-Fed-style spider (percentiles, outward=stronger).
99     # ----- #
100     # Orientation: higher percentile = stronger labour market.
101     indicators = {
102         "Emprego": ("employment", +1),

```

```

103     "Participacao": ("participation", +1),
104     "Atividade\n(hiato)": ("output_gap", +1),
105     "Prod.\nindustrial": ("industrial_yoy", +1),
106     "Baixo desemp.\njovem": ("youth_unemp", -1),
107     "Baixo\ndesemprego": ("unemployment", -1),
108 }
109
110 def percentile(series: pd.Series, value: float, sign: int) -> float:
111     s = series.dropna()
112     pct = 100.0 * (s < value).mean()
113     return pct if sign > 0 else 100.0 - pct
114
115 prepan = "2019Q4"
116 labels, now_vals, pre_vals = [], [], []
117 for label, (col, sign) in indicators.items():
118     s = quarterly[col]
119     labels.append(label)
120     now_vals.append(percentile(s, float(s.dropna().iloc[-1]), sign))
121     pre_vals.append(percentile(s, float(s.loc[prepan]) if prepan in s.index else float(s.dropna().iloc[-1]), sign))
122
123 angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
124 angles += angles[:1]
125 fig, ax = plt.subplots(figsize=(8.2, 7.2), subplot_kw=dict(polar=True))
126 for vals, color, name in ((pre_vals, "#9CA3AF", "Pre-pandemia (2019)"),
127                          (now_vals, "#1E4E79", "Atual (ultimo dado)")):
128     v = vals + vals[:1]
129     ax.plot(angles, v, color=color, linewidth=2.2, label=name)
130     ax.fill(angles, v, color=color, alpha=0.18)
131 ax.set_xticks(angles[:-1]); ax.set_xticklabels(labels, fontsize=9)
132 ax.set_yticks([25, 50, 75]); ax.set_yticklabels(["25", "50", "75"], fontsize=7)
133 ax.set_ylim(0, 100)
134 ax.set_title("Spider do mercado de trabalho chileno (percentis vs historia)\n"
135             "outward = mais forte", fontsize=11)
136 ax.legend(loc="upper right", bbox_to_anchor=(1.25, 1.1), fontsize=9)
137 fig.text(0.01, 0.01, "Estilo FED Atlanta: cada indicador em percentil vs sua historia (2001+), "
138         "orientado p/ fora=mercado mais forte. Sem JOLTS/vagas para o Chile.", fontsize=7.5)
139 fig.tight_layout()
140 fig.savefig(FIGURES / "labor_spider.png", dpi=170)
141 plt.close(fig)
142
143 print("Latest labour percentiles (outward=stronger):")
144 for label, nv in zip(labels, now_vals):
145     print(f" {label.replace(chr(10),' ')}: {nv:.0f}")
146 print(f"Unemployment last {quarterly['unemployment'].dropna().iloc[-1]:.1f}% | "
147       f"employment {quarterly['employment'].dropna().iloc[-1]:.1f}% | "
148       f"gap {quarterly['output_gap'].dropna().iloc[-1]:.1f}%")
149 print("Wrote labor_activity_dashboard.png, labor_spider.png, chile_labor.csv")
150
151
152 if __name__ == "__main__":
153     main()

```

Graficos

40 python/plot_history.py

```
1  """Phase 1 of the deep macro analysis: historical narrative + policy counterfactual.
2
3  Reads the Kalman-smoothed shocks and the Dynare historical shock decomposition
4  (produced by run_history.py) and answers two macro questions for Chile, 2001-2026:
5
6  1. Narrative: which structural shocks (demand, cost-push, monetary) drove the
7  observed inflation, output gap and policy rate each quarter?
8  2. Counterfactual: feeding the same recovered demand and cost-push shocks through
9  the model under a MORE HAWKISH Taylor rule, how different would inflation and
10 output have been (especially in the 2021-2023 inflation surge)?
11
12 The reduced form  $y_t = T i_{t-1} + R \text{eps}_t$  is the exact first-order solution
13 (identical to Dynare's). Counterfactual = re-solve with new policy parameters and
14 re-run the same recovered shock sequence.
15 """
16
17 from __future__ import annotations
18
19 import matplotlib
20
21 matplotlib.use("Agg")
22 import matplotlib.pyplot as plt
23 import numpy as np
24 import pandas as pd
25
26 from common import (
27     DATA_CLEAN,
28     DYNARE_OUTPUTS,
29     FIGURES,
30     TABLES,
31     _state_coefficients,
32     calibration_rho,
33     complete_parameters,
34     ensure_directories,
35 )
36
37 HIST = DYNARE_OUTPUTS / "history"
38 SHOCK_LABELS = {
39     "e_x": ("Demand", "#1E4E79"),
40     "e_pi": ("Cost-push", "#B45309"),
41     "e_i": ("Monetary", "#16A34A"),
42     "initial": ("Initial cond.", "#9CA3AF"),
43 }
44
45
46 def solution_matrices(params: dict) -> tuple[np.ndarray, np.ndarray]:
47     a_x, a_pi, a_i = _state_coefficients(params)
48     sigma, beta, kappa = params["sigma"], params["beta"], params["kappa"]
49     rho_i, phi_pi, phi_x = params["rho_i"], params["phi_pi"], params["phi_x"]
50     c_i = a_x - (1.0 - a_pi) / sigma
51     structural = np.array(
52         [
53             [1.0, 0.0, -c_i],
54             [-kappa, 1.0, -beta * a_pi],
55             [-(1.0 - rho_i) * phi_x, -(1.0 - rho_i) * phi_pi, 1.0],
56         ]
57     )
58     return np.array([a_x, a_pi, a_i]), np.linalg.inv(structural)
59
60
61 def simulate(params: dict, shocks: np.ndarray, i0: float = 0.0) -> np.ndarray:
62     transition, impact = solution_matrices(params)
63     n = len(shocks)
64     out = np.zeros((n, 3))
65     prev_i = i0
66     for t in range(n):
67         out[t] = transition * prev_i + impact @ shocks[t]
68         prev_i = out[t, 2]
69     return out
70
71
72 def main() -> None:
73     ensure_directories()
74     params = complete_parameters({"rho_i": calibration_rho()})
75
76     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
77     dates = macro["date"].tolist()
78     n = len(dates)
79     decomp = pd.read_csv(HIST / "shock_decomp.csv")
80     shocks_df = pd.read_csv(HIST / "smoothed_shocks.csv")
81
82     mean_infl_q = float(macro["infl_q"].mean())
83     mean_i_q = float(macro["i_q"].mean())
84     target_q = (1.03) ** 0.25 - 1.0 # inflation observable centred on the 3% target
85
86     # Annualised percentage-point scaling (linear, keeps the decomposition additive).
87     scale = {"pi": 400.0, "i": 400.0, "x": 100.0}
88     unit = {"pi": "p.p. anual", "i": "p.p. anual", "x": "% do PIB"}
89     titlevar = {"pi": "Inflacao", "i": "Taxa de politica (TPM)", "x": "Hiato do produto"}
90
91     xticks = list(range(0, n, 8))
92     xlabel = [dates[t] for t in xticks]
93
94     # ----- #
95     # Figure 1: historical shock decomposition (the macro narrative). #
96     # ----- #
97     fig, axes = plt.subplots(3, 1, figsize=(12, 10), sharex=True)
98     comps = ["e_x", "e_pi", "e_i", "initial"]
```

```

99 for ax, var in zip(axes, ["pi", "i", "x"]):
100     sub = decomp[decomp["variable"] == var].sort_values("period").reset_index(drop=True)
101     s = scale[var]
102     pos_bottom = np.zeros(n)
103     neg_bottom = np.zeros(n)
104     xs = np.arange(n)
105     for comp in comps:
106         vals = sub[comp].to_numpy() * s
107         pos = np.where(vals > 0, vals, 0.0)
108         neg = np.where(vals < 0, vals, 0.0)
109         label, color = SHOCK_LABELS[comp]
110         ax.bar(xs, pos, bottom=pos_bottom, color=color, width=0.9,
111              label=label, edgecolor="none")
112         ax.bar(xs, neg, bottom=neg_bottom, color=color, width=0.9, edgecolor="none")
113         pos_bottom += pos
114         neg_bottom += neg
115     ax.plot(xs, sub["smoothed"].to_numpy() * s, color="black", linewidth=1.3,
116           label="Observado (desvio)")
117     ax.axhline(0, color="black", linewidth=0.6)
118     ax.set_title(f"#{titlevar[var]} -- contribuicao de cada choque")
119     ax.set_ylabel(unit[var])
120     ax.grid(alpha=0.2, axis="y")
121 axes[0].legend(ncol=5, fontsize=8, loc="upper left")
122 axes[-1].set_xticks(xticks)
123 axes[-1].set_xticklabels(xlabels, rotation=70, fontsize=7)
124 fig.suptitle("Chile: decomposicao historica de choques (modelo NK calibrado, suavizador de Kalman)")
125 fig.text(0.01, 0.005, "Desvios da media amostral; escala anualizada linear. "
126        "Fonte: BCh; decomposicao do modelo, nao identificacao historica oficial.", fontsize=7)
127 fig.tight_layout(rect=(0, 0.02, 1, 0.97))
128 fig.savefig(FIGURES / "history_shock_decomposition.png", dpi=170)
129 plt.close(fig)
130
131 # ----- #
132 # Figure 2: recovered structural shocks over time. #
133 # ----- #
134 fig, axes = plt.subplots(3, 1, figsize=(12, 7), sharex=True)
135 for ax, comp in zip(axes, ["e_x", "e_pi", "e_i"]):
136     label, color = SHOCK_LABELS[comp]
137     ax.bar(np.arange(n), shocks_df[comp].to_numpy() * 100, color=color, width=0.9)
138     ax.axhline(0, color="black", linewidth=0.6)
139     ax.set_title(f"{label} shock ({comp})")
140     ax.set_ylabel("p.p. trimestral")
141     ax.grid(alpha=0.2, axis="y")
142 axes[-1].set_xticks(xticks)
143 axes[-1].set_xticklabels(xlabels, rotation=70, fontsize=7)
144 fig.suptitle("Chile: choques estruturais recuperados pelo suavizador de Kalman")
145 fig.tight_layout(rect=(0, 0, 1, 0.96))
146 fig.savefig(FIGURES / "history_smoothed_shocks.png", dpi=170)
147 plt.close(fig)
148
149 # ----- #
150 # Figure 3: policy counterfactual (hawkish rule). #
151 # ----- #
152 shocks = shocks_df[["e_x", "e_pi", "e_i"]].to_numpy()
153 base = simulate(params, shocks) # reproduces the data
154 hawk_params = complete_parameters({"rho_i": params["rho_i"], "phi_pi": 2.5})
155 hawk = simulate(hawk_params, shocks)
156 # Validation: base-sim vs observed (deviations), after initial transient.
157 obs = pd.read_csv(DATA_CLEAN / "chile_observables.csv")[["x", "pi", "i"]].to_numpy()
158 rmse = float(np.sqrt(np.mean((base[15:] - obs[15:]) ** 2)))
159
160 def to_annual_infl(dev):
161     return ((1.0 + target_q + dev) ** 4 - 1.0) * 100.0
162
163 def to_annual_tpm(dev):
164     return ((1.0 + mean_i_q + dev) ** 4 - 1.0) * 100.0
165
166 start = max(0, n - 40) # focus on the last ~10 years
167 xs = np.arange(start, n)
168 fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
169 axes[0].plot(xs, [to_annual_infl(d) for d in base[start:, 1]], color="black",
170             linewidth=2, label="Atual (regra estimada, phi_pi=1,75)")
171 axes[0].plot(xs, [to_annual_infl(d) for d in hawk[start:, 1]], color="#B91C1C",
172             linewidth=2, linestyle="--", label="Contrafactual (phi_pi=2,5)")
173 axes[0].axhline(3.0, color="#B45309", linestyle=":", linewidth=1, label="Meta 3%")
174 axes[0].set_title("Inflacao (% a.a.)")
175 axes[1].plot(xs, 100 * base[start:, 0], color="black", linewidth=2, label="Atual")
176 axes[1].plot(xs, 100 * hawk[start:, 0], color="#B91C1C", linewidth=2,
177             linestyle="--", label="Contrafactual (phi_pi=2,5)")
178 axes[1].axhline(0, color="#6B7280", linewidth=0.7)
179 axes[1].set_title("Hiato do produto (%)")
180 for ax in axes:
181     ax.set_xticks(list(range(start, n, 4)))
182     ax.set_xticklabels([dates[t] for t in range(start, n, 4)], rotation=70, fontsize=7)
183     ax.grid(alpha=0.25)
184     ax.legend(fontsize=8)
185 fig.suptitle("Contrafactual de politica: regra mais agressiva contra a inflacao "
186            f"(validacao base vs dados: RMSE={rmse:.2e})")
187 fig.tight_layout(rect=(0, 0, 1, 0.95))
188 fig.savefig(FIGURES / "history_counterfactual.png", dpi=170)
189 plt.close(fig)
190
191 # Tidy export of the decomposition (annualised pp) for the report.
192 rows = []
193 for var in ["pi", "i", "x"]:
194     sub = decomp[decomp["variable"] == var].sort_values("period").reset_index(drop=True)
195     for t in range(n):
196         rows.append({
197             "date": dates[t], "variable": var,
198             "demand": sub["e_x"][t] * scale[var],
199             "cost_push": sub["e_pi"][t] * scale[var],
200             "monetary": sub["e_i"][t] * scale[var],
201             "initial": sub["initial"][t] * scale[var],
202             "observed": sub["smoothed"][t] * scale[var],
203         })

```



```
204     pd.DataFrame(rows).to_csv(TABLES / "history_shock_decomposition.csv", index=False)
205
206     print(f"Counterfactual validation RMSE (base-sim vs data, t>=15): {rmse:.3e}")
207     print("Wrote history decomposition, smoothed shocks and counterfactual figures.")
208
209
210 if __name__ == "__main__":
211     main()
```

41 python/plot_ipc_decomposition.py

```

1  """Decomposition of Chilean IPC (CPI) inflation into structural drivers.
2
3  Two figures:
4
5  1. ipc_inflation_decomposition.png -- the observed IPC inflation (annualised %) and
6     how much of each quarter's deviation from the sample mean (3.8%) came from
7     DEMAND, COST-PUSH and MONETARY shocks, per the Kalman-smoothed baseline model.
8     The 2008 and 2022 inflation surges are highlighted.
9
10  2. ipc_inflation_models.png -- the SAME IPC inflation read by TWO models: the
11     forward-looking baseline and the hybrid NKPC (with inflation memory). The hybrid
12     needs much smaller cost-push shocks, because intrinsic persistence carries part
13     of the inflation -- i.e., the macro story of 2022 depends on the model.
14  """
15
16  from __future__ import annotations
17
18  import matplotlib
19
20  matplotlib.use("Agg")
21  import matplotlib.pyplot as plt
22  import numpy as np
23  import pandas as pd
24
25  from common import DATA_CLEAN, DYNARE_OUTPUTS, FIGURES, ensure_directories
26
27  HIST = DYNARE_OUTPUTS / "history"
28  HIST_HYB = DYNARE_OUTPUTS / "history_hybrid"
29  SHOCKS = {
30      "e_x": ("Demanda", "#1E4E79"),
31      "e_pi": ("Custo (cost-push)", "#B45309"),
32      "e_i": ("Politica monetaria", "#16A34A"),
33      "initial": ("Condicao inicial", "#9CA3AF"),
34  }
35  ANN = 400.0 # linear annualisation of a quarterly inflation deviation (keeps additivity)
36
37
38  def main() -> None:
39      ensure_directories()
40      macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
41      dates = macro["date"].tolist()
42      n = len(dates)
43      mean_infl_annual = ((1.0 + macro["infl_q"].mean()) ** 4 - 1.0) * 100.0
44      target_annual = 3.0 # observable is centred on the target; bars stack from 3%
45      ipc = macro["infl_annual_pct"].to_numpy()
46
47      decomp = pd.read_csv(HIST / "shock_decomp.csv")
48      pi = decomp[decomp["variable"] == "pi"].sort_values("period").reset_index(drop=True)
49
50      xticks = list(range(0, n, 8))
51      xlabels = [dates[t] for t in xticks]
52      # Episode shading (2008 and 2022 surges).
53      def idx(label):
54          return dates.index(label) if label in dates else None
55      episodes = [("2008Q3", "2009Q2"), ("2021Q4", "2023Q1")]
56
57      # ----- #
58      # Figure 1: IPC inflation decomposition (baseline). #
59      # ----- #
60      fig, ax = plt.subplots(figsize=(12.5, 6.2))
61      xs = np.arange(n)
62      pos = np.full(n, target_annual)
63      neg = np.full(n, target_annual)
64      for comp in ("e_x", "e_pi", "e_i", "initial"):
65          vals = pi[comp].to_numpy() * ANN
66          p = np.where(vals > 0, vals, 0.0)
67          m = np.where(vals < 0, vals, 0.0)
68          label, color = SHOCKS[comp]
69          ax.bar(xs, p, bottom=pos, color=color, width=0.92, label=label, edgecolor="none")
70          ax.bar(xs, m, bottom=neg, color=color, width=0.92, edgecolor="none")
71          pos += p
72          neg += m
73      ax.plot(xs, ipc, color="black", linewidth=1.6, label="IPC observado (% a.a.)")
74      ax.axhline(mean_infl_annual, color="#374151", linewidth=1.0, linestyle="-",
75                label=f"Media 2001--2026 ({mean_infl_annual:.1f}%)")
76      ax.axhline(3.0, color="#B45309", linewidth=1.0, linestyle="--", label="Meta 3%")
77      for a, b in episodes:
78          ia, ib = idx(a), idx(b)
79          if ia is not None and ib is not None:
80              ax.axvspan(ia, ib, color="#FCD34D", alpha=0.18)
81      ax.set_title("Decomposicao da inflacao do IPC chileno (modelo baseline, suavizador de Kalman)")
82      ax.set_ylabel("Inflacao anualizada (% a.a.)")
83      ax.set_xticks(xticks)
84      ax.set_xticklabels(xlabels, rotation=70, fontsize=7)
85      ax.grid(alpha=0.2, axis="y")
86      ax.legend(ncol=3, fontsize=8, loc="upper center")
87      fig.text(0.01, 0.005, "Barras = contribuicao de cada choque ao desvio da media (anualizado, "
88                "aproximacao linear). Faixas amarelas: surtos de 2008 e 2022. "
89                "Decomposicao do modelo, nao identificacao oficial.", fontsize=7)
90      fig.tight_layout(rect=(0, 0.03, 1, 1))
91      fig.savefig(FIGURES / "ipc_inflation_decomposition.png", dpi=170)
92      plt.close(fig)
93
94      # ----- #
95      # Figure 2: the same IPC inflation, two models. #
96      # ----- #
97      sh_base = pd.read_csv(HIST / "smoothed_shocks.csv")
98      sh_hyb = pd.read_csv(HIST_HYB / "smoothed_shocks.csv")
99      std_base = sh_base["e_pi"].std()
100      std_hyb = sh_hyb["e_pi"].std()
101
102      fig, (axA, axB) = plt.subplots(1, 2, figsize=(13.5, 5.0))

```

```

103
104 # Left: cost-push shock series, baseline vs hybrid.
105 axA.bar(xs - 0.2, sh_base["e_pi"].to_numpy() * 100, width=0.4, color="#B45309",
106         label=f"Baseline (sigma={std_base*100:.2f} p.p.)")
107 axA.bar(xs + 0.2, sh_hyb["e_pi"].to_numpy() * 100, width=0.4, color="#B91C1C",
108         label=f"Hibrida (sigma={std_hyb*100:.2f} p.p.)")
109 axA.axhline(0, color="black", linewidth=0.7)
110 for a, b in episodes:
111     ia, ib = idx(a), idx(b)
112     if ia is not None and ib is not None:
113         axA.axvspan(ia, ib, color="#FCD34D", alpha=0.18)
114 axA.set_title("Choque de custo recuperado (e_pi)")
115 axA.set_ylabel("p.p. trimestral")
116 axA.set_xticks(xticks)
117 axA.set_xticklabels(xlabels, rotation=70, fontsize=7)
118 axA.grid(alpha=0.2, axis="y")
119 axA.legend(fontsize=9)
120
121 # Right: cost-push contribution to inflation, baseline vs hybrid.
122 pi_hyb = pd.read_csv(HIST_HYB / "shock_decomp.csv")
123 pi_hyb = pi_hyb[pi_hyb["variable"] == "pi"].sort_values("period").reset_index(drop=True)
124 axB.plot(xs, pi["e_pi"].to_numpy() * ANN, color="#B45309", linewidth=2.2,
125         label="Baseline")
126 axB.plot(xs, pi_hyb["e_pi"].to_numpy() * ANN, color="#B91C1C", linewidth=2.2,
127         linestyle="--", label="Hibrida")
128 axB.axhline(0, color="black", linewidth=0.7)
129 for a, b in episodes:
130     ia, ib = idx(a), idx(b)
131     if ia is not None and ib is not None:
132         axB.axvspan(ia, ib, color="#FCD34D", alpha=0.18)
133 axB.set_title("Contribuicao do custo a inflacao (% a.a.)")
134 axB.set_ylabel("p.p. anual")
135 axB.set_xticks(xticks)
136 axB.set_xticklabels(xlabels, rotation=70, fontsize=7)
137 axB.grid(alpha=0.2, axis="y")
138 axB.legend(fontsize=9)
139
140 reduction = 100.0 * (std_base - std_hyb) / std_base
141 fig.suptitle(f"A mesma inflacao do IPC, dois modelos: com inercia (hibrida) os choques de custo "
142             f"caem {reduction:.0f}%", fontsize=12)
143 fig.tight_layout(rect=(0, 0, 1, 0.95))
144 fig.savefig(FIGURES / "ipc_inflation_models.png", dpi=170)
145 plt.close(fig)
146
147 print(f"Mean IPC inflation {mean_infl_annual:.2f}% a.a. | cost-push std baseline "
148       f"{std_base*100:.2f} vs hybrid {std_hyb*100:.2f} p.p. (-{reduction:.0f}%)")
149 print("Wrote ipc_inflation_decomposition.png and ipc_inflation_models.png")
150
151
152 if __name__ == "__main__":
153     main()

```

42 python/plot_forecast_outlook.py

```

1  """A direct-answer 'outlook' figure: where is Chile's economy heading?
2
3  Reads the native Dynare unconditional forecast and draws a clean, presentation-
4  ready dashboard for the next 8 quarters (TPM, inflation, output gap), with the
5  1-year-ahead point clearly annotated. The 1-year horizon is highlighted because
6  the pseudo-out-of-sample test (evaluate_forecasts.py) is where the NK model
7  actually beats naive benchmarks for inflation and the gap.
8  """
9
10 from __future__ import annotations
11
12 import matplotlib
13
14 matplotlib.use("Agg")
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import pandas as pd
18
19 from common import DATA_CLEAN, DYNARE_OUTPUTS, FIGURES, ensure_directories
20 from hybrid_solution import hybrid_forecast, hybrid_params, solve_hybrid
21
22 FORECAST = DYNARE_OUTPUTS / "forecast" / "forecast_unconditional.csv"
23
24
25 def main() -> None:
26     ensure_directories()
27     macro = pd.read_csv(DATA_CLEAN / "chile_macro_quarterly.csv")
28     fc = pd.read_csv(FORECAST)
29
30     mean_infl_q = float(macro["infl_q"].mean())
31     mean_i_q = float(macro["i_q"].mean())
32     target_q = (1.03) ** 0.25 - 1.0 # inflation observable is centred on the 3% target
33     mean_infl_annual = ((1.0 + mean_infl_q) ** 4 - 1.0) * 100.0
34     mean_tpm_annual = ((1.0 + mean_i_q) ** 4 - 1.0) * 100.0
35
36     def to_level(var, dev):
37         dev = np.asarray(dev, dtype=float)
38         if var == "x":
39             return 100.0 * dev
40         if var == "pi":
41             return ((1.0 + target_q + dev) ** 4 - 1.0) * 100.0
42         return ((1.0 + mean_i_q + dev) ** 4 - 1.0) * 100.0
43
44     last_period = pd.Period(str(macro.iloc[-1]["date"]), freq="Q")
45     horizon = int(fc["horizon"].max())
46     future = [str(last_period + h) for h in range(1, horizon + 1)]
47
48     # Hybrid (data-preferred) forecast from the current state, for robustness.
49     hybrid_sol = solve_hybrid(hybrid_params(0.35))
50     i_dev0 = float(macro["i_q"].iloc[-1] - mean_i_q)
51     pi_dev0 = float(macro["infl_q"].iloc[-1] - target_q)
52     hybrid_dev = np.array(
53         [hybrid_forecast(hybrid_sol, i_dev0, pi_dev0, h) for h in range(1, horizon + 1)]
54     ) # (horizon, 3) deviations of [x, pi, i]
55     var_col = {"x": 0, "pi": 1, "i": 2}
56
57     hist = macro.tail(8).copy()
58     hist_dates = hist["date"].tolist()
59     n_hist = len(hist_dates)
60     all_dates = hist_dates + future
61     x_hist = np.arange(n_hist)
62     x_fc = np.arange(n_hist - 1, n_hist - 1 + horizon + 1) # connect last obs to forecast
63
64     panels = [
65         ("i", hist["tpm_annual_pct"].to_numpy(), "Taxa de politica (TPM, % a.a.)", mean_tpm_annual, None),
66         ("pi", hist["infl_annual_pct"].to_numpy(), "Inflacao (% a.a.)", mean_infl_annual, 3.0),
67         ("x", 100.0 * hist["output_gap"].to_numpy(), "Hiato do produto (%)", 0.0, None),
68     ]
69
70     fig, axes = plt.subplots(1, 3, figsize=(13.5, 5.0))
71     for ax, (var, hist_vals, title, neutral, target) in zip(axes, panels):
72         sub = fc[fc["variable"] == var].sort_values("horizon")
73         mean_lv = to_level(var, sub["mean"].to_numpy())
74         inf_lv = to_level(var, sub["hpd_inf"].to_numpy())
75         sup_lv = to_level(var, sub["hpd_sup"].to_numpy())
76         last_obs = float(hist_vals[-1])
77
78         ax.plot(x_hist, hist_vals, color="black", marker="o", markersize=3.5,
79               linewidth=1.6, label="Observado")
80         ax.plot(x_fc, np.concatenate([[last_obs], mean_lv]), color="#1E4E79",
81               linewidth=2.6, label="Previsao (central)")
82         ax.fill_between(x_fc, np.concatenate([[last_obs], inf_lv]),
83                        np.concatenate([[last_obs], sup_lv]),
84                        color="#1E4E79", alpha=0.15, label="Banda 90%")
85         # Hybrid forecast (data-preferred model) for robustness.
86         hyb_lv = to_level(var, hybrid_dev[:, var_col[var]])
87         ax.plot(x_fc, np.concatenate([[last_obs], hyb_lv]), color="#B91C1C",
88               linewidth=1.7, linestyle="--", label="Hibrida (preferida pelos dados)")
89         # Highlight the 1-year-ahead (horizon 4) point.
90         h1y = 4
91         x1y = n_hist - 1 + h1y
92         y1y = float(mean_lv[h1y - 1])
93         ax.scatter([x1y], [y1y], color="#B91C1C", zorder=5, s=45)
94         ax.annotate(f"1 ano:\n{y1y:.1f}%", (x1y, y1y), textcoords="offset points",
95                xytext=(8, 10), fontsize=9, color="#B91C1C", fontweight="bold")
96
97         ax.axvline(n_hist - 1, color="#9CA3AF", linewidth=0.9, linestyle="--")
98         if target is not None:
99             ax.axhline(target, color="#B45309", linestyle="--", linewidth=1, label="Meta 3%")
100         ax.axhline(neutral, color="#6B7280", linestyle=":", linewidth=1,
101                label="Neutro/media" if var != "x" else "Potencial")
102         ax.set_title(title, fontsize=11)

```

```

103     step = 2
104     ticks = list(range(0, len(all_dates), step))
105     ax.set_xticks(ticks)
106     ax.set_xticklabels([all_dates[t] for t in ticks], rotation=70, fontsize=7)
107     ax.grid(alpha=0.22)
108     ax.legend(fontsize=7, loc="best")
109
110 fig.suptitle("Previsao do modelo para o Chile -- proximos 8 trimestres (incondicional, Dynare)",
111             fontsize=13)
112 fig.text(0.01, 0.005,
113         "Linha vertical = ultimo dado (2026Q1). Ponto vermelho = horizonte de 1 ano. A linha "
114         "tracejada e a NKPC hibrida (preferida pelos dados): segura mais a inflacao no curto "
115         "prazo, mas converge para a mesma previsao de 1 ano. Nao e previsao oficial do BCCh.",
116         fontsize=7.5)
117 fig.tight_layout(rect=(0, 0.04, 1, 0.95))
118 fig.savefig(FIGURES / "forecast_outlook.png", dpi=180)
119 plt.close(fig)
120 print("Wrote forecast_outlook.png")
121
122
123 if __name__ == "__main__":
124     main()

```

43 python/plot_hybrid_test.py

```
1  """A two-panel 'what the hybrid is, and whether it pays off' figure for the slide.
2
3  Left : inflation response to a cost-push shock, baseline vs hybrid -- the hybrid
4         carries inflation forward (intrinsic persistence 0.05 -> 0.38).
5  Right : out-of-sample inflation RMSE, baseline vs hybrid, at 1 and 4 quarters --
6         the inertia cuts the 1-quarter error by ~19% and ties at 1 year.
7  """
8
9  from __future__ import annotations
10
11  import matplotlib
12
13  matplotlib.use("Agg")
14  import matplotlib.pyplot as plt
15  import numpy as np
16  import pandas as pd
17
18  from common import DYNARE_OUTPUTS, FIGURES, TABLES, ensure_directories
19
20  BASE = "#1E4E79"
21  HYB = "#B91C1C"
22
23
24  def main() -> None:
25      ensure_directories()
26      base_irf = pd.read_csv(DYNARE_OUTPUTS / "baseline" / "irfs.csv")
27      hyb_irf = pd.read_csv(DYNARE_OUTPUTS / "hybrid_nkpc" / "irfs.csv")
28      oos = pd.read_csv(TABLES / "forecast_oos_metrics.csv")
29
30      fig, (axL, axR) = plt.subplots(1, 2, figsize=(12, 4.7))
31
32      # ---- Left: inflation IRF to a cost-push shock (annualised p.p.) ----
33      h = base_irf["horizon"].to_numpy()
34      axL.plot(h, base_irf["pi_e_pi"].to_numpy() * 400, color=BASE, linewidth=2.4,
35              label="Baseline (sem inercia)")
36      axL.plot(h, hyb_irf["pi_e_pi"].to_numpy() * 400, color=HYB, linewidth=2.4,
37              linestyle="--", label="Hibrida (com inercia)")
38      axL.axhline(0, color="black", linewidth=0.7)
39      axL.set_title("Como a inflacao reage a um choque de custo")
40      axL.set_xlabel("Trimestres apos o choque")
41      axL.set_ylabel("Resposta da inflacao (p.p. anual)")
42      axL.grid(alpha=0.25)
43      axL.legend(fontsize=9)
44      axL.annotate("Persistencia da inflacao: \nbaseline 0,05 -> hibrida 0,38",
45                  xy=(7.5, axL.get_ylim()[1] * 0.55), fontsize=9,
46                  bbox=dict(boxstyle="round", fc="#FEF3C7", ec="#B45309", alpha=0.9))
47
48      # ---- Right: out-of-sample inflation RMSE, baseline vs hybrid ----
49      pi = oos[(oos["variable"] == "pi") & oos["model"].isin(["NK", "NK hibrido"])]
50      def rmse(model, horizon):
51          return float(pi[(pi["model"] == model) & (pi["horizon"] == horizon)]["rmse"].iloc[0])
52
53      groups = ["1 trimestre", "1 ano (4 trim.)"]
54      base_vals = [rmse("NK", 1), rmse("NK", 4)]
55      hyb_vals = [rmse("NK hibrido", 1), rmse("NK hibrido", 4)]
56      xpos = np.arange(2)
57      width = 0.36
58      axR.bar(xpos - width / 2, base_vals, width, color=BASE, label="Baseline")
59      axR.bar(xpos + width / 2, hyb_vals, width, color=HYB, label="Hibrida")
60      improvement = 100.0 * (base_vals[0] - hyb_vals[0]) / base_vals[0]
61      axR.annotate(f"-{improvement:.0f}%", xy=(0, hyb_vals[0]), xytext=(0, hyb_vals[0] + 0.4),
62                  ha="center", fontsize=11, fontweight="bold", color=HYB,
63                  arrowprops=dict(arrowstyle="->", color=HYB))
64      axR.annotate("empate", xy=(1, max(base_vals[1], hyb_vals[1])),
65                  xytext=(1, max(base_vals[1], hyb_vals[1]) + 0.4), ha="center",
66                  fontsize=10, color="#374151")
67      axR.set_xticks(xpos)
68      axR.set_xticklabels(groups)
69      axR.set_title("Erro de previsao da inflacao (fora-da-amostra)")
70      axR.set_ylabel("RMSE (p.p.)")
71      axR.set_ylim(0, max(base_vals) * 1.25)
72      axR.grid(alpha=0.25, axis="y")
73      axR.legend(fontsize=9)
74
75      fig.suptitle("A NKPC hibrida: inercia de inflacao e ganho de previsao de curto prazo",
76                  fontsize=12)
77      fig.tight_layout(rect=(0, 0, 1, 0.95))
78      fig.savefig(FIGURES / "hybrid_test.png", dpi=180)
79      plt.close(fig)
80      print(f"Wrote hybrid_test.png (1q improvement {improvement:.1f}%)")
81
82
83  if __name__ == "__main__":
84      main()
```

44 python/plot_irfs.py

```

1  """Create the three required IRF figures with matplotlib."""
2
3  from __future__ import annotations
4
5  import json
6  import matplotlib
7
8  matplotlib.use("Agg")
9  import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12
13 from common import DATA_CLEAN, FIGURES, TABLES, ensure_directories
14
15
16 def load_irfs() -> pd.DataFrame:
17     path = TABLES / "irfs_long.csv"
18     if not path.exists():
19         from collect_outputs import main as collect_main
20
21         collect_main()
22     return pd.read_csv(path)
23
24
25 def source_note(frame: pd.DataFrame) -> str:
26     sources = sorted(frame["source"].dropna().unique())
27     if sources == ["dynare"]:
28         return "Source: Dynare"
29     return "Source includes clearly labeled synthetic model fallback"
30
31
32 def plot_kappa(irfs: pd.DataFrame) -> None:
33     selected = irfs[
34         irfs["scenario"].str.startswith("kappa_")
35         & (irfs["variable"] == "pi")
36         & (irfs["shock"] == "e_x")
37     ]
38     fig, ax = plt.subplots(figsize=(8.5, 5.2))
39     for scenario, frame in selected.groupby("scenario"):
40         label = scenario.removeprefix("kappa_").replace("p", ".")
41         ax.plot(frame["horizon"], 100 * frame["response"], linewidth=2, label=label)
42     ax.axhline(0, color="black", linewidth=0.8)
43     ax.set(
44         title="Inflation response to a demand shock: Phillips-slope (kappa) sensitivity",
45         xlabel="Quarters",
46         ylabel="Percentage points",
47     )
48     ax.legend(title="kappa")
49     ax.grid(alpha=0.25)
50     fig.text(0.01, 0.01, source_note(selected), fontsize=8)
51     fig.tight_layout(rect=(0, 0.03, 1, 1))
52     fig.savefig(FIGURES / "irf_kappa_comparison.png", dpi=180)
53     plt.close(fig)
54
55
56 def plot_kappa_tradeoffs(irfs: pd.DataFrame) -> None:
57     selected = irfs[irfs["scenario"].str.startswith("kappa_")]
58     fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), sharex=True)
59     for scenario, frame in selected.groupby("scenario"):
60         label = scenario.removeprefix("kappa_").replace("p", ".")
61         for ax, shock, title in (
62             (axes[0], "e_x", "Demand shock"),
63             (axes[1], "e_pi", "Cost-push shock"),
64         ):
65             line = frame[(frame["variable"] == "pi") & (frame["shock"] == shock)]
66             ax.plot(line["horizon"], 100 * line["response"], linewidth=2, label=label)
67             ax.set_title(title)
68             ax.set_xlabel("Quarters")
69             ax.grid(alpha=0.25)
70         axes[0].set_ylabel("Inflation response (percentage points)")
71         axes[0].legend(title="kappa")
72     for ax in axes:
73         ax.axhline(0, color="black", linewidth=0.8)
74     fig.suptitle("Phillips-curve slope: transmission and stabilization trade-offs")
75     fig.text(0.01, 0.01, source_note(selected), fontsize=8)
76     fig.tight_layout(rect=(0, 0.03, 1, 0.94))
77     fig.savefig(FIGURES / "irf_kappa_tradeoffs.png", dpi=180)
78     plt.close(fig)
79
80
81 def plot_phi_pi(irfs: pd.DataFrame) -> None:
82     selected = irfs[
83         irfs["scenario"].str.startswith("phi_pi_")
84         & (irfs["variable"] == "pi")
85         & (irfs["shock"] == "e_pi")
86     ]
87     scenarios = sorted(selected["scenario"].unique())
88     colors = plt.cm.viridis(np.linspace(0.05, 0.95, len(scenarios)))
89     fig, ax = plt.subplots(figsize=(8.8, 5.4))
90     for color, scenario in zip(colors, scenarios):
91         frame = selected[selected["scenario"] == scenario]
92         label = scenario.removeprefix("phi_pi_").replace("p", ".")
93         ax.plot(
94             frame["horizon"],
95             100 * frame["response"],
96             color=color,
97             linewidth=1.6,
98             label=label,
99         )
100     ax.axhline(0, color="black", linewidth=0.8)
101     ax.set(
102         title="Inflation response to a cost-push shock: Taylor-rule sensitivity",

```

```

103     xlabel="Quarters",
104     ylabel="Percentage points",
105 )
106 ax.legend(title="phi_pi", ncol=2, fontsize=8)
107 ax.grid(alpha=0.25)
108 fig.text(0.01, 0.01, source_note(selected), fontsize=8)
109 fig.tight_layout(rect=(0, 0.03, 1, 1))
110 fig.savefig(FIGURES / "irf_phi_pi_comparison.png", dpi=180)
111 plt.close(fig)
112
113
114 def plot_phi_pi_tradeoffs(irfs: pd.DataFrame) -> None:
115     scenarios = {
116         "phi_pi_1p3": ("1.3", "#5B8FF9"),
117         "baseline": ("1.75 baseline", "#111827"),
118         "phi_pi_2p2": ("2.2", "#F4664A"),
119     }
120     fig, axes = plt.subplots(1, 3, figsize=(14, 4.8), sharex=True)
121     for scenario, (label, color) in scenarios.items():
122         selected = irfs[
123             (irfs["scenario"] == scenario) & (irfs["shock"] == "e_pi")
124         ]
125         for ax, variable, title in zip(
126             axes,
127             ("pi", "x", "i"),
128             ("Inflation", "Output gap", "Policy rate"),
129         ):
130             line = selected[selected["variable"] == variable]
131             ax.plot(
132                 line["horizon"],
133                 100 * line["response"],
134                 linewidth=2,
135                 label=label,
136                 color=color,
137             )
138             ax.axhline(0, color="black", linewidth=0.7)
139             ax.set_title(title)
140             ax.set_xlabel("Quarters")
141             ax.grid(alpha=0.25)
142         axes[0].set_ylabel("Response (percentage points)")
143         axes[0].legend(title="phi_pi")
144         fig.suptitle("Cost-push shock: inflation-output-interest trade-off")
145         fig.text(0.01, 0.01, source_note(irfs[irfs["scenario"].isin(scenarios)]), fontsize=8)
146         fig.tight_layout(rect=(0, 0.03, 1, 0.94))
147         fig.savefig(FIGURES / "irf_phi_pi_tradeoffs.png", dpi=180)
148         plt.close(fig)
149
150
151 def plot_rho_comparison(irfs: pd.DataFrame) -> None:
152     scenarios = {
153         "baseline": ("Estimated rho_i = 0.934", "#1E4E79"),
154         "rho_calibrated_op80": ("Calibrated rho_i = 0.80", "#FF6B35"),
155     }
156     fig, axes = plt.subplots(1, 3, figsize=(14, 4.8), sharex=True)
157     for scenario, (label, color) in scenarios.items():
158         selected = irfs[
159             (irfs["scenario"] == scenario) & (irfs["shock"] == "e_i")
160         ]
161         for ax, variable, title in zip(
162             axes,
163             ("i", "x", "pi"),
164             ("Policy rate", "Output gap", "Inflation"),
165         ):
166             line = selected[selected["variable"] == variable]
167             ax.plot(
168                 line["horizon"],
169                 100 * line["response"],
170                 linewidth=2.2,
171                 label=label,
172                 color=color,
173             )
174             ax.axhline(0, color="black", linewidth=0.7)
175             ax.set_title(title)
176             ax.set_xlabel("Quarters")
177             ax.grid(alpha=0.25)
178         axes[0].set_ylabel("Response (percentage points)")
179         axes[0].legend(fontsize=8)
180         fig.suptitle("Interest-rate smoothing route: calibrated versus estimated rho_i")
181         fig.text(0.01, 0.01, source_note(irfs[irfs["scenario"].isin(scenarios)]), fontsize=8)
182         fig.tight_layout(rect=(0, 0.03, 1, 0.94))
183         fig.savefig(FIGURES / "irf_rho_comparison.png", dpi=180)
184         plt.close(fig)
185
186
187 def plot_baseline(irfs: pd.DataFrame) -> None:
188     selected = irfs[irfs["scenario"] == "baseline"]
189     variables = ("x", "pi", "i")
190     shocks = ("e_x", "e_pi", "e_i")
191     labels = {"x": "Output gap", "pi": "Inflation", "i": "Policy rate"}
192     shock_labels = {
193         "e_x": "Demand shock",
194         "e_pi": "Cost-push shock",
195         "e_i": "Monetary-policy shock",
196     }
197     fig, axes = plt.subplots(3, 3, figsize=(11.5, 8.5), sharex=True)
198     for row, variable in enumerate(variables):
199         for column, shock in enumerate(shocks):
200             ax = axes[row, column]
201             frame = selected[
202                 (selected["variable"] == variable) & (selected["shock"] == shock)
203             ]
204             ax.plot(frame["horizon"], 100 * frame["response"], linewidth=1.8)
205             ax.axhline(0, color="black", linewidth=0.7)
206             ax.grid(alpha=0.22)
207             if row == 0:

```



```

208         ax.set_title(shock_labels[shock])
209         if column == 0:
210             ax.set_ylabel(f"{labels[variable]}\nnp")
211         if row == 2:
212             ax.set_xlabel("Quarters")
213     fig.suptitle("Baseline impulse responses across all shocks", fontsize=14)
214     fig.text(0.01, 0.01, source_note(selected), fontsize=8)
215     fig.tight_layout(rect=(0, 0.03, 1, 0.96))
216     fig.savefig(FIGURES / "irf_baseline_all_shocks.png", dpi=180)
217     plt.close(fig)
218
219
220 def plot_data_overview() -> None:
221     path = DATA_CLEAN / "chile_macro_quarterly.csv"
222     if not path.exists():
223         return
224     df = pd.read_csv(path)
225     metadata_path = DATA_CLEAN / "dataset_metadata.json"
226     metadata = (
227         json.loads(metadata_path.read_text(encoding="utf-8"))
228         if metadata_path.exists()
229         else {"is_synthetic": True}
230     )
231     is_synthetic = bool(metadata.get("is_synthetic", True))
232     source = (
233         "Banco Central de Chile (TPM, IPC, PIB)"
234         if not is_synthetic
235         else "Synthetic fallback for pipeline testing"
236     )
237     fig, axes = plt.subplots(3, 1, figsize=(9, 8.2), sharex=True)
238     axes[0].plot(df["date"], df["tpm_annual_pct"], color="#1E4E79")
239     axes[0].set_ylabel("TPM (% a.a.)")
240     axes[1].plot(df["date"], df["infl_annual_pct"], color="#FF6B35")
241     axes[1].axhline(3.0, color="gray", ls="--", lw=0.8)
242     axes[1].set_ylabel("Inflacao IPC (% a.a.)")
243     axes[2].plot(df["date"], 100.0 * df["output_gap"], color="#16a34a")
244     axes[2].axhline(0.0, color="black", lw=0.7)
245     axes[2].set_ylabel("Hiato (%)")
246     for ax in axes:
247         ax.grid(alpha=0.25)
248     step = max(1, len(df) // 12)
249     axes[2].set_xticks(df["date"][::step])
250     axes[2].tick_params(axis="x", rotation=90, labelsz=7)
251     title_kind = "official quarterly data" if not is_synthetic else "synthetic test data"
252     axes[0].set_title(f"Chile: {title_kind} - policy rate, inflation, output gap")
253     fig.text(
254         0.01,
255         0.005,
256         f"Source: {source}; output gap = HP cycle of log real GDP.",
257         fontsize=7,
258     )
259     fig.tight_layout(rect=(0, 0.03, 1, 1))
260     fig.savefig(FIGURES / "data_overview.png", dpi=180)
261     plt.close(fig)
262
263
264 def main() -> None:
265     ensure_directories()
266     plot_data_overview()
267     irfs = load_irfs()
268     plot_kappa(irfs)
269     plot_kappa_tradeoffs(irfs)
270     plot_phi_pi(irfs)
271     plot_phi_pi_tradeoffs(irfs)
272     plot_rho_comparison(irfs)
273     plot_baseline(irfs)
274     print(f"Wrote IRF figures to {FIGURES}")
275
276
277 if __name__ == "__main__":
278     main()

```

```

1  """Plot Bayesian posterior summaries and convergence diagnostics."""
2
3  from __future__ import annotations
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7  import pandas as pd
8
9  from common import FIGURES, TABLES
10
11
12  LABELS = {
13      "stderr e_x": r"$\sigma_{e_x}$",
14      "stderr e_pi": r"$\sigma_{e_{\pi}}$",
15      "stderr e_i": r"$\sigma_{e_i}$",
16      "sigma": r"$\sigma$",
17      "kappa": r"$\kappa$",
18      "rho_i": r"$\rho_i$",
19      "phi_pi": r"$\phi_{\pi}$",
20      "phi_x": r"$\phi_x$",
21  }
22
23
24  def main() -> None:
25      posterior = pd.read_csv(TABLES / "mcmc_posterior.csv")
26      diagnostics = pd.read_csv(TABLES / "mcmc_diagnostics.csv")
27      merged = posterior.merge(diagnostics, on="parameter", validate="one_to_one")
28
29      y = np.arange(len(merged))
30      means = merged["posterior_mean"].to_numpy()
31      low = merged["hpd90_low"].to_numpy()
32      high = merged["hpd90_high"].to_numpy()
33
34      fig, axes = plt.subplots(1, 2, figsize=(12, 5.8))
35      axes[0].errorbar(
36          means,
37          y,
38          xerr=np.vstack([means - low, high - means]),
39          fmt="o",
40          color="#145DA0",
41          ecolor="#72A0C1",
42          capsize=3,
43      )
44      axes[0].set_yticks(y, [LABELS.get(p, p) for p in merged["parameter"]])
45      axes[0].invert_yaxis()
46      axes[0].set_title("Posterior mean and 90% credible interval")
47      axes[0].set_xlabel("Parameter value")
48      axes[0].grid(axis="x", alpha=0.25)
49
50      colors = np.where(merged["rhat"] < 1.05, "#2E8B57", "#C0392B")
51      axes[1].barh(y, merged["rhat"] - 1.0, left=1.0, color=colors)
52      axes[1].axvline(1.05, color="#333333", linestyle="--", linewidth=1)
53      axes[1].set_yticks(y, [LABELS.get(p, p) for p in merged["parameter"]])
54      axes[1].invert_yaxis()
55      axes[1].set_xlim(0.99, max(1.08, merged["rhat"].max() * 1.02))
56      axes[1].set_title(r"Convergence diagnostic ($\hat{R}$)")
57      axes[1].set_xlabel(r"$\hat{R}$; values below 1.05 are preferred")
58      axes[1].grid(axis="x", alpha=0.25)
59
60      fig.suptitle("Bayesian MCMC extension: posterior uncertainty and convergence")
61      fig.text(
62          0.5,
63          0.01,
64          "Two Dynare chains; 10,000 proposals each; first 30% discarded. "
65          "Results are model- and prior-conditional.",
66          ha="center",
67          fontsize=9,
68      )
69      fig.tight_layout(rect=(0, 0.04, 1, 0.96))
70      target = FIGURES / "mcmc_posterior_diagnostics.png"
71      fig.savefig(target, dpi=180, bbox_inches="tight")
72      plt.close(fig)
73      print(f"Wrote {target}")
74
75
76  if __name__ == "__main__":
77      main()

```

Tabelas, coleta e entregaveis

46 python/collect_outputs.py

```
1  """Collect Dynare CSV exports and create explicit synthetic fallbacks."""
2
3  from __future__ import annotations
4
5  import math
6  import pandas as pd
7
8  from common import (
9      DYNARE_OUTPUTS,
10     TABLES,
11     calibration_rho,
12     complete_parameters,
13     ensure_directories,
14     fallback_irfs,
15     fallback_moments,
16 )
17
18 VARIABLES = ("x", "pi", "i")
19 SHOCKS = ("e_x", "e_pi", "e_i")
20
21
22 def read_dynare_fevd(scenario: str) -> pd.DataFrame | None:
23     path = DYNARE_OUTPUTS / scenario / "fevd.csv"
24     if not path.exists():
25         return None
26     wide = pd.read_csv(path)
27     records = []
28     for _, row in wide.iterrows():
29         values = [float(row[shock]) for shock in SHOCKS]
30         scale = 100.0 if sum(values) > 1.5 else 1.0
31         for shock, value in zip(SHOCKS, values):
32             records.append(
33                 {
34                     "scenario": scenario,
35                     "variable": row["variable"],
36                     "shock": shock,
37                     "variance_share": value / scale,
38                     "source": "dynare",
39                 }
40             )
41     return pd.DataFrame(records)
42
43
44 def dynare_irfs(scenario: str, path) -> pd.DataFrame:
45     wide = pd.read_csv(path)
46     records = []
47     for variable in VARIABLES:
48         for shock in SHOCKS:
49             column = f"{variable}_{shock}"
50             if column not in wide:
51                 continue
52             for horizon, response in zip(wide["horizon"], wide[column]):
53                 records.append(
54                     {
55                         "scenario": scenario,
56                         "source": "dynare",
57                         "variable": variable,
58                         "shock": shock,
59                         "horizon": int(horizon),
60                         "response": float(response),
61                     }
62                 )
63     return pd.DataFrame(records)
64
65
66 def manifest_with_baseline() -> pd.DataFrame:
67     path = TABLES / "scenario_manifest.csv"
68     manifest = pd.read_csv(path) if path.exists() else pd.DataFrame()
69     baseline = {"scenario": "baseline", "scenario_type": "baseline"}
70     baseline.update(complete_parameters({"rho_i": calibration_rho()}))
71     return pd.concat([pd.DataFrame([baseline]), manifest], ignore_index=True)
72
73
74 def row_parameters(row: pd.Series) -> dict:
75     keys = (
76         "rstar_annual",
77         "sigma",
78         "kappa",
79         "rho_i",
80         "phi_pi",
81         "phi_x",
82         "rstar",
83         "beta",
84     )
85     return {key: float(row[key]) for key in keys if key in row and pd.notna(row[key])}
86
87
88 def collect_irfs(manifest: pd.DataFrame) -> pd.DataFrame:
89     pieces = []
90     for _, row in manifest.iterrows():
91         scenario = str(row["scenario"])
92         irf_path = DYNARE_OUTPUTS / scenario / "irfs.csv"
93         if irf_path.exists():
94             try:
95                 frame = dynare_irfs(scenario, irf_path)
96                 if not frame.empty:
97                     pieces.append(frame)
98             except:
```

```

99         except Exception as exc:
100             print(f"Warning: could not parse {irf_path}: {exc}")
101         params = complete_parameters(row_parameters(row))
102         pieces.append(fallback_irfs(params, scenario))
103     result = pd.concat(pieces, ignore_index=True)
104     result.to_csv(TABLES / "irfs_long.csv", index=False)
105     return result
106
107
108 def collect_moments(manifest: pd.DataFrame) -> None:
109     frames = []
110     for _, row in manifest.iterrows():
111         scenario = str(row["scenario"])
112         path = DYNARE_OUTPUTS / scenario / "moments.csv"
113         if path.exists():
114             moment = pd.read_csv(path)
115             moment.insert(0, "scenario", scenario)
116             moment["source"] = "dynare"
117             frames.append(moment)
118         else:
119             params = complete_parameters(row_parameters(row))
120             moment = fallback_moments(params)
121             moment.insert(0, "scenario", scenario)
122             frames.append(moment)
123     if not frames:
124         baseline = manifest.loc[manifest["scenario"] == "baseline"].iloc[0]
125         moment = fallback_moments(complete_parameters(row_parameters(baseline)))
126         moment.insert(0, "scenario", "baseline")
127         frames.append(moment)
128     moments_all = pd.concat(frames, ignore_index=True)
129     moments_all.to_csv(TABLES / "moments_all_scenarios.csv", index=False)
130     moments_all[moments_all["scenario"] == "baseline"].to_csv(
131         TABLES / "moments.csv", index=False
132     )
133
134
135 def collect_fevd(irfs: pd.DataFrame, manifest: pd.DataFrame) -> None:
136     frames = []
137     for scenario in manifest["scenario"].astype(str):
138         frame = read_dynare_fevd(scenario)
139         if frame is not None:
140             frames.append(frame)
141         continue
142     selected = irfs[irfs["scenario"] == scenario].copy()
143     selected["squared_response"] = selected["response"] ** 2
144     totals = (
145         selected.groupby(["variable", "shock"], as_index=False)["squared_response"]
146         .sum()
147         .rename(columns={"squared_response": "shock_contribution"})
148     )
149     totals["variance_share"] = totals["shock_contribution"] / totals.groupby(
150         "variable"
151     )["shock_contribution"].transform("sum")
152     totals["scenario"] = scenario
153     totals["source"] = "synthetic_irf_share_fallback"
154     frames.append(
155         totals[["scenario", "variable", "shock", "variance_share", "source"]]
156     )
157     fevd_all = pd.concat(frames, ignore_index=True)
158     fevd_all.to_csv(TABLES / "fevd_all_scenarios.csv", index=False)
159     fevd_all[fevd_all["scenario"] == "baseline"].to_csv(
160         TABLES / "fevd_summary.csv", index=False
161     )
162
163 def collect_determinacy(manifest: pd.DataFrame) -> None:
164     rows = []
165     for _, scenario_row in manifest.iterrows():
166         scenario = str(scenario_row["scenario"])
167         stability_path = DYNARE_OUTPUTS / scenario / "stability.csv"
168         if stability_path.exists():
169             row = pd.read_csv(stability_path).iloc[0].to_dict()
170             row["source"] = "dynare_check"
171         else:
172             row = {
173                 "scenario": scenario,
174                 "status": "not_run_or_failed",
175                 "n_forward": None,
176                 "n_unstable": None,
177                 "n_eigenvalues": None,
178                 "source": "no_dynare_diagnostic",
179             }
180     eigen_path = DYNARE_OUTPUTS / scenario / "eigenvalues.csv"
181     if eigen_path.exists():
182         eigenvalues = pd.read_csv(eigen_path)
183         stable = eigenvalues[eigenvalues["modulus"] < 1.0]["modulus"]
184         unstable = eigenvalues[eigenvalues["modulus"] > 1.0]["modulus"]
185         dominant = float(stable.max()) if not stable.empty else math.nan
186         row["dominant_stable_modulus"] = dominant
187         row["convergence_half_life_quarters"] = (
188             math.log(0.5) / math.log(dominant)
189             if 0.0 < dominant < 1.0
190             else math.nan
191         )
192         row["smallest_unstable_modulus"] = (
193             float(unstable.min()) if not unstable.empty else math.nan
194         )
195     else:
196         row["dominant_stable_modulus"] = math.nan
197         row["convergence_half_life_quarters"] = math.nan
198         row["smallest_unstable_modulus"] = math.nan
199     row["scenario_type"] = scenario_row.get("scenario_type", "")
200     row["phi_pi"] = scenario_row.get("phi_pi", None)
201     rows.append(row)
202     pd.DataFrame(rows).to_csv(TABLES / "scenario_determinacy.csv", index=False)
203

```

```
204
205 def main() -> None:
206     ensure_directories()
207     manifest = manifest_with_baseline()
208     irfs = collect_irfs(manifest)
209     collect_moments(manifest)
210     collect_fevd(irfs, manifest)
211     collect_determinacy(manifest)
212     print(f"Collected outputs in {TABLES}")
213     print(irfs.groupby("source")["scenario"].nunique().to_string())
214
215
216 if __name__ == "__main__":
217     main()
```

47 python/make_tables.py

```

1  """Create the parameter-target table and the neutral-rate conversion table.
2
3  The parameter-target table is the assignment's headline "parameter <-> target" deliverable.
4  It lists each calibrated value next to its empirical counterpart (when an estimate exists:
5  AR(1) for rho_i, IV/OLS Taylor rule for phi_pi/phi_x, NKPC for kappa, HP proxy for r*).
6  """
7
8  from __future__ import annotations
9
10 import pandas as pd
11
12 from common import (
13     BASELINE,
14     SHOCK_STD,
15     TABLES,
16     calibration_rho,
17     complete_parameters,
18     ensure_directories,
19     quarterly_rstar,
20 )
21
22
23 def _safe_read(name: str) -> pd.DataFrame | None:
24     path = TABLES / name
25     return pd.read_csv(path) if path.exists() else None
26
27
28 def empirical_estimates() -> dict[str, str]:
29     emp: dict[str, str] = {}
30     rhoi = _safe_read("rhoi_estimate.csv")
31     if (
32         rhoi is not None
33         and not rhoi.empty
34         and not bool(rhoi.iloc[0].get("is_synthetic", True))
35     ):
36         r = rhoi.iloc[0]
37         emp["rho_i"] = f"{r['rho_i_estimate']:.3f} (AR1; HAC SE {r['rho_i_se_hac']:.3f})"
38     taylor = _safe_read("taylor_rule_estimates.csv")
39     if (
40         taylor is not None
41         and not taylor.empty
42         and not bool(taylor.iloc[0].get("is_synthetic", True))
43     ):
44         iv = taylor[taylor["method"].str.startswith("IV_2SLS")]
45         ols = taylor[taylor["method"] == "OLS_HAC"]
46         if not iv.empty and not ols.empty:
47             emp["phi_pi"] = (
48                 f"{iv.iloc[0]['phi_pi']:.2f} (IV-HAC); "
49                 f"{ols.iloc[0]['phi_pi']:.2f} (OLS-HAC)"
50             )
51         emp["phi_x"] = f"{iv.iloc[0]['phi_x']:.2f} (IV-HAC)"
52     nkpc = _safe_read("nkpc_estimates.csv")
53     if (
54         nkpc is not None
55         and not nkpc.empty
56         and not bool(nkpc.iloc[0].get("is_synthetic", True))
57     ):
58         bw = nkpc[nkpc["method"] == "backward_OLS_HAC"]
59         if not bw.empty:
60             emp["kappa"] = f"{bw.iloc[0]['kappa']:.3f} (backward OLS, HAC)"
61     rstar = _safe_read("rstar_estimates.csv")
62     if (
63         rstar is not None
64         and not rstar.empty
65         and not bool(rstar.iloc[0].get("is_synthetic", True))
66     ):
67         hp = rstar[rstar["method"] == "hp_trend_real_rate_end_of_sample"]
68         if not hp.empty:
69             emp["rstar_annual"] = f"{hp.iloc[0]['rstar_annual_pct']:.2f}% a.a. (HP proxy)"
70     return emp
71
72
73 def main() -> None:
74     ensure_directories()
75     rho_i = calibration_rho()
76     beta = complete_parameters()["beta"]
77     emp = empirical_estimates()
78
79     rows = [
80         ("rstar_annual", BASELINE["rstar_annual"], emp.get("rstar_annual", "")),
81         ("Annual neutral real rate; scenarios 2/3/4%"),
82         ("beta", round(beta, 6), ""),
83         ("Discount factor = 1/(1+rstar_q), derived from r*"),
84         ("sigma", BASELINE["sigma"], ""),
85         ("Inverse intertemporal elasticity; log-utility benchmark"),
86         ("kappa", BASELINE["kappa"], emp.get("kappa", "")),
87         ("Phillips-curve slope; sensitivity grid {0.07,0.10,0.13}"),
88         ("rho_i", round(rho_i, 4), emp.get("rho_i", "")),
89         ("Interest-rate smoothing; AR(1) of the quarterly TPM"),
90         ("phi_pi", BASELINE["phi_pi"], emp.get("phi_pi", "")),
91         ("Inflation response; grid [1.3,2.2]; determinacy checked numerically"),
92         ("phi_x", BASELINE["phi_x"], emp.get("phi_x", "")),
93         ("Output-gap response (held fixed at 0.5)"),
94         ("std_e_x", SHOCK_STD["e_x"], ""),
95         ("Demand-shock std; illustrative model-based FEVD calibration"),
96         ("std_e_pi", SHOCK_STD["e_pi"], ""),
97         ("Cost-push-shock std; illustrative model-based FEVD calibration"),
98         ("std_e_i", SHOCK_STD["e_i"], ""),
99         ("Monetary-shock std; illustrative model-based FEVD calibration"),
100     ]
101     parameter_table = pd.DataFrame(
102         rows,

```

```

103         columns=["parameter", "calibrated_value", "empirical_estimate",
104                 "target_or_rationale"],
105     )
106     parameter_table["country"] = "Chile"
107     parameter_table.to_csv(TABLES / "parameter_targets.csv", index=False)
108
109     rows_rstar = []
110     for annual in (0.02, 0.03, 0.04):
111         quarterly = quarterly_rstar(annual)
112         rows_rstar.append(
113             {
114                 "rstar_annual": annual,
115                 "rstar_annual_percent": annual * 100.0,
116                 "rstar_quarterly": quarterly,
117                 "rstar_quarterly_percent": quarterly * 100.0,
118                 "beta": 1.0 / (1.0 + quarterly),
119             }
120         )
121     pd.DataFrame(rows_rstar).to_csv(TABLES / "rstar_beta_table.csv", index=False)
122     print("Wrote parameter_targets.csv and rstar_beta_table.csv")
123     print(parameter_table.to_string(index=False))
124
125
126 if __name__ == "__main__":
127     main()

```

48 python/build_final_html.py

```

1  """Build the self-contained, evaluator-facing final HTML report.
2
3  The output embeds every public figure, the main result tables, the full source
4  code, and a detailed Portuguese narrative. No network connection is required
5  to read the resulting file.
6  """
7
8  from __future__ import annotations
9
10 import base64
11 import html
12 import json
13 from datetime import date
14 from pathlib import Path
15
16 import pandas as pd
17
18
19 ROOT = Path(__file__).resolve().parents[1]
20 TABLES = ROOT / "outputs" / "tables"
21 FIGURES = ROOT / "outputs" / "figures"
22 OUTPUT = ROOT / "Entrega Final.html"
23
24
25 def esc(value: object) -> str:
26     return html.escape(str(value), quote=True)
27
28
29 def fmt(value: object) -> object:
30     if pd.isna(value):
31         return ""
32     if isinstance(value, float):
33         if abs(value) >= 100:
34             return f"{value:,.2f}"
35         if abs(value) >= 1:
36             return f"{value:.3f}"
37         if value == 0:
38             return "0"
39         return f"{value:.6f}"
40     return value
41
42
43 def csv_table(
44     filename: str,
45     *,
46     columns: list[str] | None = None,
47     query: str | None = None,
48     max_rows: int | None = None,
49     table_class: str = "",
50 ) -> str:
51     path = TABLES / filename
52     if not path.exists():
53         return f'<div class="warning">Tabela ausente: <code>{esc(filename)}</code></div>'
54     df = pd.read_csv(path)
55     if query:
56         df = df.query(query)
57     if columns:
58         df = df[[column for column in columns if column in df.columns]]
59     if max_rows:
60         df = df.head(max_rows)
61     df = df.map(fmt)
62     return df.to_html(index=False, border=0, classes=f"data-table {table_class}".strip(), escape=True)
63
64
65 def image(filename: str, alt: str, caption: str, analysis: str = "") -> str:
66     path = FIGURES / filename
67     if not path.exists():
68         return f'<div class="warning">Figura ausente: <code>{esc(filename)}</code></div>'
69     encoded = base64.b64encode(path.read_bytes()).decode("ascii")
70     note = f'<div class="figure-analysis">{analysis}</div>' if analysis else ""
71     return f"""
72 <figure>
73 
74 <figcaption><strong>{esc(caption)}</strong><br><span>Arquivo: outputs/figures/{esc(filename)}</span></figcaption>
75 {note}
76 </figure>
77 """
78
79
80 def callout(kind: str, title: str, body: str) -> str:
81     return f'<aside class="callout {esc(kind)}"><strong>{esc(title)}</strong><div>{body}</div></aside>'
82
83
84 def code_details(path: Path) -> str:
85     relative = path.relative_to(ROOT).as_posix()
86     try:
87         source = path.read_text(encoding="utf-8")
88     except UnicodeDecodeError:
89         source = path.read_text(encoding="latin-1")
90     language = {
91         ".py": "python",
92         ".m": "matlab",
93         ".mod": "dynare",
94         ".md": "markdown",
95         ".txt": "text",
96     }.get(path.suffix.lower(), "text")
97     lines = source.count("\n") + 1
98     return f"""
99 <details class="code-file">
100 <summary><code>{esc(relative)}</code><span>{lines} linhas</span></summary>
101 <pre><code class="language-{language}">{html.escape(source)}</code></pre>
102 </details>

```



```

103     """
104
105
106 def section(section_id: str, kicker: str, title: str, body: str) -> str:
107     return f"""
108     <section id="{esc(section_id)}">
109         <div class="section-kicker">{esc(kicker)}</div>
110         <h2>{title}</h2>
111         {body}
112     </section>
113     """
114
115
116 def read_metadata() -> dict:
117     path = ROOT / "data" / "clean" / "dataset_metadata.json"
118     return json.loads(path.read_text(encoding="utf-8")) if path.exists() else {}
119
120
121 def build() -> None:
122     metadata = read_metadata()
123     figure_count = len(list(FIGURES.glob("*.png")))
124     scenario_manifest = pd.read_csv(TABLES / "scenario_manifest.csv")
125     determinacy = pd.read_csv(TABLES / "scenario_determinacy.csv")
126     sign_checks = pd.read_csv(TABLES / "irf_sign_checks.csv")
127     n_generated_scenarios = len(scenario_manifest)
128     n_models = len(determinacy)
129     n_determinate = int(determinacy["status"].eq("determinate_bk_count").sum())
130     n_signs = int(sign_checks["impact_sign_matches"].astype(str).str.lower().eq("true").sum())
131
132     nav = [
133         ("resumo", "Resumo"),
134         ("auditoria", "Aderencia a aula"),
135         ("chile", "Chile e dados"),
136         ("glossario", "Conceitos e siglas"),
137         ("modelo", "Modelo teorico"),
138         ("calibracao", "Calibracao"),
139         ("solucao", "Dynare e solucao"),
140         ("momentos", "Momentos e correlacoes"),
141         ("fevd", "FEVD"),
142         ("irfs", "IRFs baseline"),
143         ("kappa", "Sensibilidade kappa"),
144         ("phi", "Sensibilidade phi"),
145         ("rho", "Persistencia rho"),
146         ("econometria", "Econometria"),
147         ("previsoes", "Previsoes"),
148         ("macro", "Analise macro aprofundada"),
149         ("politica", "Analise monetaria"),
150         ("limitacoes", "Limitacoes"),
151         ("reproducao", "Reproducao"),
152         ("codigo", "Codigo completo"),
153     ]
154
155     parts: list[str] = []
156     parts.append(
157         section(
158             "resumo",
159             "Entrega final",
160             "Politica monetaria no Chile em um modelo Novo-Keynesiano",
161             f"""
162             <p class="lead">Calibracao, solucao, diagnostico de determinacao, funcoes
163             impulso-resposta, decomposicao de variancia, econometria e cenarios condicionais
164             em um projeto reproduzivel com <strong>Python, GNU Octave e Dynare</strong>.</p>
165             <div class="metric-grid">
166                 <div class="metric"><span>{metadata.get("observations", 101)}</span><small>trimestres de dados reais</small></div>
167                 <div class="metric"><span>{n_models}</span><small>modelos Dynare executados</small></div>
168                 <div class="metric"><span>{n_determinate}/{n_models}</span><small>cenarios principais determinados</small></div>
169                 <div class="metric"><span>{n_signs}/9</span><small>sinais de impacto corretos</small></div>
170             </div>
171             {callout("success", "Resultado central",
172                 "0 baseline e determinado e seus sinais de impacto sao coerentes com a teoria. "
173                 "Ao mesmo tempo, o confronto com os dados mostra que o modelo pequeno subestima "
174                 "volatilidades e persistencias; por isso, os resultados sao um laboratorio de "
175                 "mecanismos, nao uma descricao completa ou uma previsao oficial do Chile.")}
176             <div class="two-col">
177                 <div>
178                     <h3>0 que foi feito</h3>
179                     <ul>
180                         <li>Dados oficiais trimestrais do Banco Central de Chile, 2001Q1-2026Q1.</li>
181                         <li>Modelo NK linear de tres equacoes resolvido no Dynare 7.1 via Octave 11.1.</li>
182                         <li>Baseline e {n_generated_scenarios} cenarios gerados para as grades e alternativas.</li>
183                         <li>Duas rotas para <code>rho_i</code> e duas calibracoes de FEVD.</li>
184                         <li>Previsao incondicional e condicionamento por juro e por inflacao.</li>
185                         <li>Regra de Taylor, NKPC, taxa neutra e moda posterior exploratorias.</li>
186                     </ul>
187                 </div>
188                 <div>
189                     <h3>Leitura correta</h3>
190                     <ul>
191                         <li>Calibracao nao e estimacao.</li>
192                         <li>FEVD calibrada nao e decomposicao historica.</li>
193                         <li>AR(1) da TPM nao identifica sozinho a suavizacao estrutural.</li>
194                         <li>IRFs podem apresentar <em>overshooting</em> sem invalidar o modelo.</li>
195                         <li>Cenarios condicionais sao experimentos internos, nao recomendacoes.</li>
196                     </ul>
197                 </div>
198             </div>
199             """
200         )
201     )
202
203     parts.append(
204         section(
205             "auditoria",
206             "Paginas 26-55 da Aula 5",
207             "0 que o roteiro pediu e como o projeto respondeu",

```

```

208         f"""
209         <p>A revisao foi feita a partir do PDF privado disponivel localmente. O texto abaixo e
210         uma sintese original: nenhuma pagina ou passagem foi incorporada ao repositorio.</p>
211         <div class="audit-grid">
212             <article><b>Taxa neutra</b><span>2%, 3% e 4% a.a.; conversao trimestral e beta.</span></article>
213             <article><b>Persistencia</b><span>AR(1) real e alternativa calibrada em 0,80.</span></article>
214             <article><b>Phillips</b><span>kappa = 0,07; 0,10; 0,13 com IRFs e custos reais.</span></article>
215             <article><b>Regra monetaria</b><span>phipi = 1,3,...,2,2 e mapa ampliado de determinacao.</span></article>
216             <article><b>Choques e FEVD</b><span>Alvo didatico e alvo alternativo executados.</span></article>
217             <article><b>Previsoes</b><span>Incondicional, juro condicionado e inflacao condicionada.</span></article>
218         </div>
219         {callout("info", "Quando o roteiro diz 'isto ou aquilo'",
220             "A entrega nao escolhe silenciosamente uma opcao. Ela testa as duas: rho estimado "
221             "e calibrado; FEVD didatica e alternativa; condicionamento pelo juro e pela inflacao.")}
222         <h3>Correcoes resultantes da auditoria</h3>
223         <ol>
224             <li>Os nove sinais de impacto estao corretos, mas algumas respostas cruzam zero no
225             trimestre seguinte. Portanto, nao se exige monotonicidade das IRFs.</li>
226             <li>Um phipi maior reduz o impacto da inflacao, mas eleva o movimento de produto e juro;
227             a soma absoluta da inflacao pode subir por <em>undershooting</em>.</li>
228             <li>O comando de previsao condicional de primeira ordem usa choques controlados nao
229             antecipados antes de cada periodo. Isso nao e previsao perfeita deterministica.</li>
230             <li>Momentos e FEVD precisam ser confrontados com dados e identificados como resultados
231             internos da calibracao.</li>
232         </ol>
233         <p class="source-note">Documento de auditoria:
234         <code>docs/assignment_review_pages_26_55.md</code>.</p>
235         """
236     )
237 )
238
239 parts.append(
240     section(
241         "chile",
242         "Contexto empirico",
243         "Por que Chile, quais dados e quais transformacoes",
244         f"""
245         <p>O Chile combina um regime de metas de inflacao, uma taxa de politica claramente
246         identificada e series oficiais acessiveis. A meta e interpretada como inflacao projetada
247         de 3% em horizonte de dois anos. A variavel operacional e a <strong>TPM</strong>
248         (Tasa de Politica Monetaria). Esse ambiente e adequado para ilustrar o nucleo de um
249         modelo NK, embora o carater de economia pequena e aberta seja uma limitacao decisiva.</p>
250         <div class="three-col">
251             <article class="card"><h3>TPM</h3><p>Serie diaria, convertida em media trimestral e
252             em taxa trimestral composta.</p><code>F022.TPM.TIN.D001.NO.Z.D</code></article>
253             <article class="card"><h3>IPC</h3><p>Variacao mensal composta dentro do trimestre e
254             anualizada para apresentacao.</p><code>F074.IPC.VAR.Z.Z.C.M</code></article>
255             <article class="card"><h3>PIB real</h3><p>Log do PIB dessazonalizado; ciclo do filtro HP
256             com lambda=1600 usado como hiato.</p><code>F032.PIB.FLU.R.CLP.EP18.Z.Z.1.T</code></article>
257         </div>
258         {image("data_overview.png", "TPM, inflacao e hiato do produto no Chile",
259             "Figura 1 -- Dados trimestrais utilizados",
260             "A amostra cobre 101 trimestres. A pandemia produz um hiato extremo em 2020Q2, "
261             "enquanto o ciclo inflacionario de 2021-2023 e acompanhado por forte elevacao da TPM. "
262             "Esses episodios explicam por que momentos de amostra e estimacoes sao sensiveis ao periodo.")}
263         <h3>Proveniencia e unidades</h3>
264         <table class="data-table">
265             <thead><tr><th>Item</th><th>Valor</th></tr></thead>
266             <tbody>
267                 <tr><td>Instituicao</td><td>{esc(metadata.get("institution", "BCCh"))}</td></tr>
268                 <tr><td>Periodo</td><td>{esc(metadata.get("period", ""))}</td></tr>
269                 <tr><td>Observacoes</td><td>{esc(metadata.get("observations", ""))}</td></tr>
270                 <tr><td>Acesso</td><td>{esc(metadata.get("access_date", ""))}</td></tr>
271                 <tr><td>Base sintetica?</td><td><strong>{esc(metadata.get("is_synthetic", ""))}</strong></td></tr>
272                 <tr><td>Transformacoes</td><td>{esc(metadata.get("transformations", ""))}</td></tr>
273             </tbody>
274         </table>
275         {callout("warning", "Hiato do produto nao e observado",
276             "O filtro HP separa tendencia e ciclo mecanicamente e sofre vies de ponta. O hiato "
277             "resultante e uma proxy, nao uma medida estrutural inequivoca de capacidade ociosa.")}
278         """
279     )
280 )
281
282 glossary = [
283     ("NK", "Novo-Keynesiano: estrutura com expectativas racionais, rigidez nominal e politica monetaria."),
284     ("DSGE", "Modelo dinamico estocastico de equilibrio geral; o modelo desta entrega e um nucleo DSGE minimo."),
285     ("IS", "Equacao de demanda intertemporal que liga hiato ao juro real ex ante."),
286     ("NKPc", "New Keynesian Phillips Curve: curva de Phillips prospectiva."),
287     ("Calvo", "Mecanismo em que apenas uma fracao das firmas pode reajustar precos a cada periodo."),
288     ("EIS", "Elasticidade intertemporal de substituicao; no modelo e inversamente relacionada a sigma."),
289     ("TPM", "Tasa de Politica Monetaria do Banco Central de Chile."),
290     ("BCCh", "Banco Central de Chile."),
291     ("IPC", "Indice de Precos ao Consumidor."),
292     ("PIB", "Produto Interno Bruto."),
293     ("IRF", "Impulse Response Function: trajetoria apos um choque isolado."),
294     ("FEVD", "Forecast Error Variance Decomposition: parcela da variancia atribuida a cada choque."),
295     ("BK", "Condicoes de Blanchard-Kahn para existencia e unicidade da solucao racional."),
296     ("MQO/OLS", "Minimos Quadrados Ordinarios / Ordinary Least Squares."),
297     ("HAC", "Matriz de covariancia robusta a heterocedasticidade e autocorrelacao."),
298     ("Newey-West", "Estimador HAC usado nos erros-padrao das regressoes."),
299     ("VI/IV", "Variaveis instrumentais, usadas quando regressores podem ser endogenos."),
300     ("2SLS", "Minimos Quadrados em Dois Estagios."),
301     ("MAP", "Moda a posteriori: ponto de maior densidade posterior."),
302     ("Laplace", "Aproximacao local gaussiana da incerteza em torno da moda."),
303     ("MCMC", "Simulacao de Monte Carlo via cadeias de Markov para a distribuicao posterior."),
304     ("HPD", "Intervalo de maior densidade posterior."),
305     ("HP", "Filtro Hodrick- Prescott, aqui com lambda 1600 para dados trimestrais."),
306     ("UIP", "Paridade descoberta de juros, ausente no modelo fechado."),
307     ("SVAR", "Vetor autorregressivo estrutural, alternativa empirica para identificar choques."),
308     ("p.b.", "Ponto-base: 0,01 ponto percentual."),
309 ]
310
311 glossary_html = "".join(
312     f"<dt>{esc(term)}</dt><dd>{esc(definition)}</dd>" for term, definition in glossary
313 )

```

```

313 parts.append(
314     section(
315         "glossario",
316         "Base conceitual",
317         "Glossario de conceitos, parametros e siglas",
318         f"""
319         <dl class="glossary">{glossary_html}</dl>
320         <h3>Parametros e objetos do modelo</h3>
321         <div class="parameter-grid">
322             <article><b>beta</b><span>Fator de desconto. Quanto maior, maior o peso do futuro.</span></article>
323             <article><b>sigma</b><span>Inverso da EIS. Regula a sensibilidade da demanda ao juro real.</span></article>
324             <article><b>kappa</b><span>Inclinacao da NKPC. Transmissao do hiato para inflacao.</span></article>
325             <article><b>rho<sub>i</sub></b><span>Persistencia/suavizacao da taxa de politica.</span></article>
326             <article><b>phi<sub>pi</sub></b><span>Resposta de longo prazo do juro a inflacao.</span></article>
327             <article><b>phi<sub>x</sub></b><span>Resposta de longo prazo do juro ao hiato.</span></article>
328             <article><b>r*</b><span>Taxa real neutra compativel com equilibrio no modelo.</span></article>
329             <article><b>e<sub>x</sub></b><span>Choque de demanda ou preferencia.</span></article>
330             <article><b>e<sub>pi</sub></b><span>Choque de custo/markup.</span></article>
331             <article><b>e<sub>i</sub></b><span>Inovacao monetaria nao explicada pela regra.</span></article>
332         </div>
333         <h3>Dinamica e solucao</h3>
334         <p><strong>Variavel predeterminada</strong> tem seu valor corrente herdado do passado;
335         aqui, <code>(1)</code>. <strong>Variaveis de salto</strong> podem reagir imediatamente
336         as noticias; aqui, <code>x</code> e <code>pi</code>. Um <strong>autovalor estavel</strong>
337         tem modulo menor que um. A determinacao exige que o numero de raizes instaveis seja
338         igual ao numero de variaveis prospectivas.</p>
339         <p><strong>Momento</strong> e uma caracteristica da distribuicao, como media, variancia,
340         desvio-padrao, correlacao ou autocorrelacao. <strong>Convergencia</strong> descreve o
341         retorno a vizinhanca do estado estacionario. <strong>Overshooting</strong> ou
342         <strong>undershooting</strong> ocorre quando a variavel cruza seu nivel de equilibrio
343         antes de convergir.</p>
344         """,
345     )
346 )
347
348 parts.append(
349     section(
350         "modelo",
351         "Estrutura teorica",
352         "As tres equacoes e a transmissao monetaria",
353         f"""
354         <div class="equation">x<sub>t</sub> = E<sub>t</sub>x<sub>t+1</sub>
355         ? (1/sigma)[i<sub>t</sub> ? E<sub>t</sub>pi<sub>t+1</sub> ? r*] + e<sub>x,t</sub></div>
356         <div class="equation">pi<sub>t</sub> = betaE<sub>t</sub>pi<sub>t+1</sub>
357         + kappax<sub>t</sub> + e<sub>pi,t</sub></div>
358         <div class="equation">i<sub>t</sub> = rho<sub>i</sub>i<sub>t-1</sub>
359         + (1?rho<sub>i</sub>)[r* + phi<sub>pi</sub>pi<sub>t</sub>
360         + phi<sub>x</sub>x<sub>t</sub>] + e<sub>i,t</sub></div>
361         <div class="three-col">
362             <article class="card"><h3>Curva IS</h3><p>Se o juro real ex ante supera r*, adiar
363             consumo fica relativamente atraente e a demanda corrente cai. Expectativas futuras
364             afetam a decisao hoje.</p></article>
365             <article class="card"><h3>Curva de Phillips</h3><p>Firmas que nao reajustam precos
366             tornam a inflacao dependente do custo marginal, aproximado pelo hiato, e da inflacao
367             esperada.</p></article>
368             <article class="card"><h3>Regra de Taylor</h3><p>O banco central reage a inflacao e
369             atividade, mas suaviza alteracoes da taxa. O principio de Taylor requer resposta
370             nominal suficiente para elevar o juro real.</p></article>
371         </div>
372         <h3>Cadeias de transmissao</h3>
373         <ul>
374             <li><b>Demanda positiva</b>: x sobe -> pi sobe -> regra eleva i -> juro real freia x.</li>
375             <li><b>Custo positivo</b>: pi sobe e x cai sob aperto -> surge o trade-off de estabilizacao.</li>
376             <li><b>Aperto monetario</b>: i sobe -> juro real sobe -> x cai -> pi cai.</li>
377         </ul>
378         <h3>Estado estacionario e unidades</h3>
379         <p>Na representacao do arquivo principal, <code>x=0</code>, <code>pi=0</code> e
380         <code>i=rstar</code>. As IRFs sao convertidas para pontos percentuais. Para previsoes em
381         nivel, inflacao recebe de volta a meta de 3% e o juro recebe o neutro nominal de 6,09%,
382         composto a partir de r* real de 3% e meta de inflacao de 3%.</p>
383         """,
384     )
385 )
386
387 parts.append(
388     section(
389         "calibracao",
390         "Parametro ? alvo",
391         "Calibracao, taxa neutra, beta e persistencia",
392         f"""
393         <h3>Mapa de parametros</h3>
394         {csv_table("parameter_targets.csv")}
395         <h3>Taxa neutra anual, taxa trimestral e beta</h3>
396         {csv_table("rstar_beta_table.csv")}
397         <p>A conversao e <code>rstar_q = (1+rstar_annual)^(1/4)-1</code> e
398         <code>beta = 1/(1+rstar_q)</code>. Elevar r* de 2% para 4% reduz beta de 0,99506 para
399         0,99024. Esses sao cenarios; nao sao tres estimativas concorrentes da taxa neutra chilena.</p>
400         <h3>AR(1) da TPM</h3>
401         {csv_table("rhoi_estimate.csv", columns=[
402             "rho_i_estimate", "rho_i_se_hac", "rho_i_ci95_low", "rho_i_ci95_high",
403             "half_life_quarters", "r_squared", "observations", "sample_start", "sample_end"
404         ])}
405         {callout("warning", "Persistencia reduzida nao e parametro estrutural puro",
406             "O valor 0,934 descreve a inercia da TPM observada. Mudancas de regime, movimentos "
407             "do neutro e resposta sistematica a inflacao e atividade tambem afetam o AR(1).")}
408         """,
409     )
410 )
411
412 parts.append(
413     section(
414         "solucao",
415         "Octave + Dynare",
416         "Pre-processamento, estado estacionario e Blanchard-Kahn",
417         f"""

```

```

418 <p>0 Dynare pre-processa o arquivo <code>.mod</code>, monta as equacoes estaticas e
419 dinamicas, calcula o estado estacionario, lineariza o sistema e aplica uma decomposicao
420 generalizada para obter a solucao racional de primeira ordem. O comando
421 <code>check;</code> reporta os autovalores; <code>stoch_simul(order=1, irf=20);</code>
422 calcula momentos, FEVD e IRFs.</p>
423 <div class="metric-grid">
424   <div class="metric"><span>0,656</span><small>raiz estavel dominante</small></div>
425   <div class="metric"><span>1,040</span><small>menor raiz instavel</small></div>
426   <div class="metric"><span>2</span><small>raizes instaveis</small></div>
427   <div class="metric"><span>2</span><small>variaveis forward-looking</small></div>
428 </div>
429 {callout("success", "Condicao de Blanchard-Kahn satisfeita",
430   "Ha exatamente duas raizes fora do circulo unitario para x e pi, as duas variaveis "
431   "prospectivas. O baseline possui solucao local unica e estavel.")}
432 {image("determinacy_map.png", "Mapa de determinacao por phi pi",
433   "Figura 2 -- Determinacao na grade ampliada de phipi",
434   "A fronteira numerica ocorre em torno de phipi=1 para esta parametrizacao. A grade "
435   "principal de 1,3 a 2,2 e inteiramente determinada; o ganho marginal passa a ser "
436   "avaliado pela dinamica e pelo custo em produto e juros.")}
437 <h3>Diagnostic of todos os cenarios</h3>
438 {csv_table("scenario_determinacy.csv", columns=[
439   "scenario", "status", "n_forward", "n_unstable",
440   "dominant_stable_modulus", "convergence_half_life_quarters",
441   "smallest_unstable_modulus"
442 ])}
443 """,
444 )
445 )
446
447 parts.append(
448   section(
449     "momentos",
450     "Ajuste quantitativo",
451     "Momentos teoricos, dados e correlacoes",
452     f"""
453     {image("moments_model_vs_data.png", "Momentos do modelo comparados aos dados",
454       "Figura 3 -- Volatilidade e autocorrelacao",
455       "O modelo subestima fortemente a volatilidade. Mesmo retirando 2020Q2-2021Q4, "
456       "o hiato observado continua mais volatil. A TPM empirica tambem e bem mais "
457       "persistente do que o juro endogeno do modelo.")}
458     {csv_table("moments_model_vs_data.csv")}
459     <h3>Correlacoes contemporaneas</h3>
460     {csv_table("correlations_model_vs_data.csv")}
461     <p>0 baseline produz correlacao negativa entre juro e hiato porque um aperto contrai
462     atividade. Nos dados, a correlacao incondicional e positiva: o banco central tambem
463     sobe juros quando a economia e a inflacao estao fortes. Essa inversao ilustra por que
464     correlacao observada nao identifica causalidade monetaria.</p>
465     {callout("info", "O objetivo nao e maximizar ajuste",
466       "A calibracao de choques prioriza uma FEVD transparente e IRFs plausiveis. Uma "
467       "estimacao completa exigiria mais friccoes, observaveis, regimes e tratamento de outliers.")}
468     """,
469   )
470 )
471
472 parts.append(
473   section(
474     "fevd",
475     "Decomposicao de variancia",
476     "Duas metas de FEVD e o que elas significam",
477     f"""
478     {image("fevd_calibration_comparison.png", "FEVD calibrada em duas metas",
479       "Figura 4 -- Meta versus FEVD alcançada",
480       "Com tres desvios-padrao nao se ajustam livremente nove parcelas. A calibracao "
481       "minimiza o erro conjunto. A matriz alternativa tem RMSE de 0,95 p.p.; a didatica, "
482       "1,82 p.p. Ambas reproduzem a narrativa de inflacao dominada por custo e juro "
483       "dominado por inovacao monetaria.")}
484     <h3>Desvios-padrao resultantes</h3>
485     {csv_table("shock_sigmas_comparison.csv")}
486     <h3>Alvo e resultado, celula por celula</h3>
487     {csv_table("fevd_calibration_comparison.csv")}
488     <h3>FEVD baseline</h3>
489     {csv_table("fevd_summary.csv")}
490     <p>No baseline, o hiato e explicado em aproximadamente 45% por demanda e 45% por
491     politica; a inflacao, em 80% pelo choque de custo; o juro, em 75% pelo choque monetario.
492     Isso valida o procedimento de calibracao, mas nao demonstra que esses percentuais
493     descrevem historicamente o Chile.</p>
494     """,
495   )
496 )
497
498 parts.append(
499   section(
500     "irfs",
501     "Mecanismos dinamicos",
502     "IRFs do baseline: todos os choques e todas as variaveis",
503     f"""
504     {image("irf_baseline_all_shocks.png", "Grade 3 por 3 de IRFs do baseline",
505       "Figura 5 -- Respostas a demanda, custo e politica monetaria",
506       "Todos os sinais de impacto correspondem a teoria. Demanda positiva eleva x, pi e i; "
507       "custo positivo eleva pi e i e reduz x; aperto monetario eleva i e reduz x e pi. "
508       "As reversoes de sinal no horizonte 1 em algumas respostas sao overshooting, nao erro.")}
509     <h3>Checagem automatica dos sinais</h3>
510     {csv_table("irf_sign_checks.csv")}
511     <div class="three-col">
512       <article class="card"><h3>Choque de demanda</h3><p>Impactos: x +0,400 p.p.,
513       pi +0,021 p.p., i +0,016 p.p. A reacao da politica faz x e pi cruzarem zero no
514       trimestre seguinte.</p></article>
515       <article class="card"><h3>Choque de custo</h3><p>Impactos: pi +0,236 p.p.,
516       x ?0,144 p.p., i +0,022 p.p. E o trade-off de estabilizacao: reduzir inflacao
517       requer contracao real.</p></article>
518       <article class="card"><h3>Choque monetario</h3><p>Impactos: i +0,048 p.p.,
519       x ?0,307 p.p., pi ?0,088 p.p. A inflacao cai imediatamente na solucao prospectiva.</p></article>
520     </div>
521     <h3>Forma reduzida da politica de decisao</h3>
522     {csv_table("policy_transition_coefficients.csv")}

```

```

523         <p>Os coeficientes unitarios explicam como o estado <code>i(-1)</code> e cada inovacao
524         movem as variaveis. A coluna calibrada multiplica cada resposta pelo desvio-padrao do
525         choque e coincide com o impacto das IRFs.</p>
526         """,
527     )
528 )
529
530 parts.append(
531     section(
532         "kappa",
533         "Rigidez de precos",
534         "Sensibilidade a kappa = 0,07; 0,10; 0,13",
535         f"""
536         {image("irf_kappa_comparison.png", "Inflacao sob tres valores de kappa",
537             "Figura 6 -- Resposta da inflacao a um choque de demanda",
538             "Quanto maior kappa, mais rapidamente a atividade se converte em inflacao. O impacto "
539             "sobe de 0,0145 para 0,0279 p.p. quando kappa passa de 0,07 para 0,13.")}
540         {image("irf_kappa_tradeoffs.png", "Trade-offs de kappa",
541             "Figura 7 -- Efeitos de kappa em diferentes choques",
542             "Apos choque de custo, kappa maior reduz o impacto inflacionario e o custo real "
543             "acumulado. Apos demanda ou politica, porem, precos mais flexiveis transmitem mais "
544             "fortemente a atividade para inflacao.")}
545
546         <ul>
547             <li>Inflacao de impacto apos demanda: 0,0145 -> 0,0212 -> 0,0279 p.p.</li>
548             <li>Inflacao de impacto apos custo: 0,2451 -> 0,2357 -> 0,2276 p.p.</li>
549             <li>Custo real acumulado apos custo: 0,4627 -> 0,4187 -> 0,3851 p.p.</li>
550             <li>Raiz estavel dominante: 0,6869 -> 0,6561 -> 0,6301.</li>
551         </ul>
552         <p>A leitura monetaria correta depende do choque. Uma Phillips mais inclinada facilita
553         a desinflacao induzida pela contracao do hiato, mas tambem faz choques de demanda
554         aparecerem mais rapidamente nos precos.</p>
555         """,
556     )
557 )
558
559 parts.append(
560     section(
561         "phi",
562         "Principio de Taylor",
563         "Sensibilidade a phipi de 1,3 a 2,2",
564         f"""
565         {image("irf_phi_pi_comparison.png", "Inflacao sob diferentes phi pi",
566             "Figura 8 -- Resposta da inflacao ao choque de custo",
567             "A reacao mais forte reduz o impacto inicial, mas produz maior reversao posterior. "
568             "Por isso, o pico e a soma absoluta contam historias diferentes.")}
569         {image("irf_phi_pi_tradeoffs.png", "Trade-offs de phi pi",
570             "Figura 9 -- Inflacao, produto, juro e convergencia",
571             "Entre 1,3 e 2,2, o impacto da inflacao cai de 0,2423 para 0,2303 p.p. e a "
572             "convergencia acelera pouco. Em troca, os movimentos acumulados de juro e hiato "
573             "sobem de 0,0501 para 0,0796 e de 0,3519 para 0,4734 p.p.")}
574         {callout("warning", "Mais reacao nao melhora toda metrica",
575             "A soma absoluta da inflacao cresce ligeiramente, de 0,3107 para 0,3167 p.p., "
576             "devido ao undershooting. A conclusao robusta e menor impacto inicial com maior "
577             "custo real e financeiro, nao estabilizacao uniforme.")}
578         <p>Todos os valores da grade principal satisfazem BK. O mapa ampliado mostra
579         indeterminacao abaixo do limiar numerico em torno de um. Esse limiar depende de toda a
580         parametrizacao, nao apenas de uma regra verbal sobre phipi.</p>
581         """,
582     )
583 )
584
585 parts.append(
586     section(
587         "rho",
588         "Gradualismo",
589         "rhoi estimado em 0,934 versus calibrado em 0,80",
590         f"""
591         {image("irf_rho_comparison.png", "Comparacao de IRFs por rho i",
592             "Figura 10 -- Persistencia estimada e calibrada",
593             "A maior persistencia estende os efeitos do choque monetario. No baseline, os "
594             "efeitos acumulados absolutos sobre inflacao e hiato sao aproximadamente quatro "
595             "e tres vezes os do cenario com rho=0,80.")}
596
597         <table class="data-table">
598             <thead><tr><th>Resposta ao choque monetario</th><th>rho=0,934</th><th>rho=0,80</th></tr></thead>
599             <tbody>
600                 <tr><td>Juro, soma absoluta</td><td>0,1388 p.p.</td><td>0,0976 p.p.</td></tr>
601                 <tr><td>Inflacao, soma absoluta</td><td>0,2555 p.p.</td><td>0,0592 p.p.</td></tr>
602                 <tr><td>Hiato, soma absoluta</td><td>0,8911 p.p.</td><td>0,2777 p.p.</td></tr>
603                 <tr><td>Meia-vida da raiz estavel</td><td>1,645 trim.</td><td>1,107 trim.</td></tr>
604             </tbody>
605         </table>
606
607         <p>Esta e uma das analises "ou/ou" executadas dos dois modos. O valor estimado e mais
608         aderente a persistencia bruta da TPM, enquanto 0,80 separa melhor uma calibracao
609         estrutural convencional. A diferenca entre eles e economicamente grande.</p>
610         """,
611     )
612 )
613
614 parts.append(
615     section(
616         "econometria",
617         "Evidencia complementar",
618         "Regra de Taylor, NKPC, r* e moda posterior",
619         f"""
620         <h3>Regra de Taylor reduzida</h3>
621         {csv_table("taylor_rule_estimates.csv")}
622         <p>MQ0-HAC sugere phipi=0,83; VI/2SLS-HAC, 3,05. O primeiro estagio da inflacao tem
623         F=5,82, sinal de instrumento potencialmente fraco. A divergencia nao autoriza escolher
624         mecanicamente a estimativa mais compativel com a teoria; ela revela identificacao fragil.</p>
625         <h3>Curva de Phillips</h3>
626         {csv_table("nkpc_estimates.csv")}
627         <p>A especificacao backward encontra kappa=0,098, proximo da calibracao. A versao forward
628         apresenta sinal negativo, erro-padrao elevado e primeiro estagio fraco. Isso nao valida
629         empiricamente a NKPC prospectiva; apenas mostra que a grade escolhida e compativel com
630         uma relacao reduzida backward.</p>

```

```

628 <h3>Referencias empiricas para r*</h3>
629 {csv_table("rstar_estimates.csv")}
630 <p>A tendencia HP do juro real e uma proxy descritiva, sujeita a vies de ponta. Uma
631 estimativa estrutural de r* exigiria espaco de estados e dinamica de produto potencial.</p>
632 <h3>Moda posterior do DSGE</h3>
633 {csv_table("bayesian_estimates.csv")}
634 {callout("warning", "MAP nao e posterior completo",
635 "Como mh_replic=0, nao ha cadeias MCMC. Os intervalos sao aproximacoes locais de "
636 "Laplace em torno da moda e podem ser pouco informativos fora dessa vizinhanca.")}
637 """,
638 )
639 )
640
641 parts.append(
642 section(
643 "previsoes",
644 "Exercicios prospectivos",
645 "Previsao incondicional e duas formas de condicionamento",
646 f"""
647 <p>A previsao e gerada pelo proprio Dynare, como nas paginas 48-55 da aula. O comando
648 <code>estimation(datafile=..., mode_compute=0, forecast=8)</code> mantem a calibracao
649 fixa, roda o filtro/suavizador de Kalman sobre o historico e projeta oito trimestres a
650 frente, devolvendo <code>oo_.forecast.Mean</code> e as bandas <code>HPDinf/HPDsup</code>.
651 Os observaveis sao desvios da media amostral; aqui as series voltam ao nivel anualizado
652 (TPM media 4,09% a.a.; inflacao media 3,79% a.a.; meta oficial de 3%).</p>
653 {image("forecast_fanchart.png", "Fan chart da previsao incondicional",
654 "Figura 11 -- Cenario incondicional (Dynare) e bandas HPD",
655 "Partindo do estado suavizado do ultimo trimestre, o modelo projeta inflacao de "
656 "3,30% no primeiro horizonte convergindo para 3,76% no oitavo, e a TPM caindo de "
657 "4,36% para -4,11% (a media amostral). As bandas sao as faixas de credibilidade do "
658 "proprio Dynare, dadas as variancias dos choques calibrados.")}
659 {image("conditional_scenarios.png", "Cenarios condicionais para juro e inflacao",
660 "Figura 12 -- Caminhos condicionados (conditional_forecast)",
661 "As duas formas de condicionamento da aula: controlar a TPM (manter em 4,5% por 2 "
662 "trimestres; apertar para -5,1% por 4) e controlar a inflacao (forçar a meta de 3% "
663 "por 4 trimestres). Os caminhos estao ancorados no estado atual via a forma reduzida "
664 "exata, identica a solucao do Dynare.")}
665 {image("conditional_policy_shocks.png", "Choques monetarios implicitos",
666 "Figura 13 -- Inovacoes e_i necessarias para cumprir as restricoes",
667 "Os choques implicitos tornam explicito o custo de impor cada caminho: o aperto de "
668 "-5,1% exige a maior inovacao inicial; forçar a inflacao a meta de 3% exige um "
669 "aperto pontual mais modesto.")}
670 <h3>Resumo numerico dos horizontes 1, 4 e 8</h3>
671 {csv_table("forecast_summary.csv", columns=[
672 "horizon", "date", "scenario", "variable", "value_level",
673 "p05_level", "p95_level", "implied_e_i_q"
674 ])}
675 {callout("danger", "Nao interpretar como previsao oficial",
676 "O modelo nao carrega separadamente o ultimo x e a ultima pi observados (o unico "
677 "estado e a taxa passada), nao contem cambio, cobre ou condicoes externas e nao "
678 "incorpora julgamento do BCCh. Por nao ter inercia de inflacao, a projecao reverte "
679 "rapido a media -- e um laboratorio mecanico, nao previsao perfeita.")}
680 """,
681 )
682 )
683
684 parts.append(
685 section(
686 "macro",
687 "Extensoes e validacao",
688 "Historia macro, evidencia empirica e mecanismos ausentes no baseline",
689 f"""
690 <p class="lead">Este bloco amplia o trabalho obrigatorio em tres direcoes: usa o
691 suavizador para organizar a historia macro; confronta o DSGE com benchmarks empiricos;
692 e reespecifica o modelo para inercia inflacionaria e economia pequena e aberta.</p>
693
694 <h3>1. Decomposicao historica e contrafactual</h3>
695 {image("history_shock_decomposition.png", "Decomposicao historica dos observaveis",
696 "Figura 14 -- Contribuicoes historicas dos tres choques",
697 "Como ha tres observaveis, tres choques e nenhum erro de medida, a reconstrucao e "
698 "quase exata. Isso valida o codigo, mas nao identifica causalmente a historia.")}
699 {image("history_smoothed_shocks.png", "Choques recuperados pelo suavizador",
700 "Figura 15 -- Inovacoes estruturais suavizadas",
701 "Crises exigem inovacoes muito maiores que os desvios-padrao ilustrativos do "
702 "baseline, pois poucos choques absorvem toda a volatilidade observada.")}
703 {image("history_counterfactual.png", "Contrafactual com reacao mais agressiva",
704 "Figura 16 -- Mesmos choques sob phi_pi = 2,5",
705 "A regra hawkish muda inflacao, atividade e juros. E um exercicio condicional "
706 "sujeito a critica de Lucas, nao uma avaliacao causal do BCCh.")}
707
708 <h3>2. SVAR, r* e desempenho preditivo</h3>
709 {image("svar_vs_dsge_irf.png", "IRFs monetarias do SVAR e DSGE",
710 "Figura 17 -- Evidencia reduzida versus restricoes estruturais",
711 "O SVAR passa o teste de brancura, mas rejeita normalidade e apresenta activity "
712 "puzzle inicial. A divergencia informa sobre identificacao e especificacao.")}
713 {csv_table("svar_diagnostics.csv")}
714 {image("rstar_time_varying.png", "Proxy estatistica de r-star",
715 "Figura 18 -- Componente local do juro real ex ante",
716 "A estimativa termina perto de 0,85% a.a. A amplitude da serie revela incerteza e "
717 "contaminacao ciclica; nao e uma estimacao Laubach-Williams estrutural.")}
718 {csv_table("rstar_time_varying_summary.csv")}
719 {image("forecast_oos_comparison.png", "Erros de previsao fora da amostra",
720 "Figura 19 -- NK versus AR(1), passeio aleatorio e VAR",
721 "O NK e fraco a um trimestre, mas ganha do passeio aleatorio para inflacao, hiato "
722 "e marginalmente juros em quatro trimestres. O teste usa dados revisados.")}
723 {csv_table("forecast_oos_metrics.csv", columns=[
724 "horizon", "variable", "model", "rmse", "relative_rmse_vs_random_walk"
725 ])}
726
727 <h3>Nucleos da inflacao e mercado de trabalho (FRED)</h3>
728 {image("ipc_cores_panel.png", "IPC por nucleo",
729 "Inflacao por nucleo vs a banda de 2-4%",
730 "Energia muito volatil; nucleo e servicos persistentes e acima da banda no surto "
731 "recente. Componentes OECD/FRED ate -2023.")}
732 {image("ipc_cores_drivers.png", "Quem mais empurrou a inflacao",

```



```

733     "Drivers do surto de 2021-23 por nucleo",
734     "A energia liderou (pico de 24% a.a.), a frente da cheia (14%), servicos (12%) e "
735     "nucleo (11%). Energia = choque de custo; nucleo/servicos = inercia da hibrida.")}
736 {image("labor_activity_dashboard.png", "Mercado de trabalho e atividade",
737     "Desemprego, emprego, participacao e o hiato do modelo",
738     "Emprego e participacao em maximas; desemprego ainda 8,7%; hiato (Kalman) ~0. Sem "
739     "dados de vagas/admissoes para o Chile no FRED. Okun: desemprego inverso ao hiato.")}
740 {image("labor_spider.png", "Spider do mercado de trabalho",
741     "Spider estilo FED Atlanta (percentis vs historia)",
742     "Forte em emprego (92o percentil) e participacao (76o); fraco em desemprego (34o) e "
743     "atividade (32o); atual vs pre-pandemia.")}
744
745 <h3>3. Economia aberta e inercia inflacionaria</h3>
746 {image("open_economy_data.png", "Cambio e cobre usados na extensao",
747     "Figura 20 -- Canais externos relevantes para o Chile",
748     "Depreciacao do peso e ciclo do cobre ancoram persistencias e volatilidades. Os "
749     "demais coeficientes estruturais continuam sendo calibracoes ilustrativas.")}
750 {image("open_economy_irfs.png", "IRFs do modelo de economia aberta",
751     "Figura 21 -- Depreciacao, cobre e transmissao monetaria",
752     "Depreciacao eleva inflacao por pass-through; cobre positivo expande demanda e "
753     "pressiona precos e juros. O modelo satisfaz Blanchard-Kahn.")}
754 {image("open_vs_closed_monetary_irf.png", "Modelo aberto versus fechado",
755     "Figura 22 -- O setor externo altera a transmissao",
756     "O cambio cria uma margem adicional mesmo para choques monetarios, mostrando o "
757     "custo analitico de tratar o Chile como economia fechada.")}
758 {image("hybrid_nkpc_irfs.png", "IRFs com Phillips hibrida",
759     "Figura 23 -- Indexacao e persistencia inflacionaria",
760     "Com gamma_pi=0,35, a autocorrelacao modelada da inflacao sobe de cerca de 0,05 "
761     "para 0,38. O ajuste melhora, mas a indexacao e calibrada.")}
762
763 <h3>4. Posterior completa por MCMC</h3>
764 {image("mcmc_posterior_diagnostics.png", "Posteriores e R-hat do MCMC",
765     "Figura 24 -- Incerteza parametrica e convergencia",
766     "Duas cadeias de 10.000 propostas, com 30% descartado. R-hat fica abaixo de 1,05, "
767     "mas ESS modesto e aceitacao alta recomendam cautela.")}
768 {csv_table("mcmc_posterior.csv")}
769 {csv_table("mcmc_diagnostics.csv")}
770
771 <h3>5. Comparacao por evidencia marginal</h3>
772 {image("bayesian_model_comparison.png", "Evidencia marginal dos modelos de Phillips",
773     "Figura 25 -- NKPC prospectiva versus NKPC hibrida",
774     "Com os mesmos 101 trimestres, observaveis e priors comuns, a aproximacao de "
775     "Laplace favorece a NKPC hibrida por 12,02 log-pontos. O fator de Bayes aproximado "
776     "e cerca de 166 mil, mesmo apos a penalizacao do parametro gamma_pi.")}
777 {csv_table("bayesian_model_comparison.csv")}
778 {callout("info", "Como ler a comparacao",
779     "A evidencia marginal integra ajuste e complexidade. O resultado e formal, mas "
780     "local ao modo posterior e condicionado aos priors. Nao e bridge sampling nem "
781     "uma estimativa de evidencia baseada em cadeias completas para os dois modelos.")}
782
783 {callout("warning", "Hierarquia das conclusoes",
784     "IRFs do DSGE sao mecanismos condicionais; SVAR depende da identificacao; r* e "
785     "proxy estatistica; a decomposicao historica e exata por construaao; modelo aberto "
786     "e NKPC hibrida sao calibracoes; MCMC e posterior condicionada ao modelo e priors.")}
787 """,
788 )
789 )
790
791 parts.append(
792     section(
793         "politica",
794         "Sintese economica",
795         "Revisao aprofundada da analise monetaria",
796         """,
797         <h3>1. Determinacao e credibilidade</h3>
798         <p>A determinacao requer que expectativas privadas sejam ancoradas por uma regra
799         suficientemente ativa. No baseline, phipi=1,75 coloca o sistema acima da fronteira BK.
800         Entretanto, a evidencia reduzida para o Chile e ampla: MQO, VI e MAP nao entregam um
801         mesmo numero. A conclusao correta e que a calibracao representa um regime ativo
802         plausivel, nao uma estimativa pontual incontroversa da reacao do BCCh.</p>
803
804         <h3>2. Gradualismo e horizonte de transmissao</h3>
805         <p>O AR(1) elevado faz os movimentos da TPM durarem e amplia o efeito acumulado de uma
806         inovacao monetaria. Isso e consistente com decisoes graduais e comunicacao por
807         trajetorias, mas o AR(1) tambem absorve mudancas lentas do neutro e persistencia dos
808         proprios fundamentos. A comparacao com rho=0,80 mostra que a escolha altera
809         substancialmente a transmissao.</p>
810
811         <h3>3. Choques de custo e o trade-off</h3>
812         <p>Quando a inflacao sobe por custo, elevar juros reduz demanda, mas nao remove
813         diretamente o choque. A inflacao cai ao custo de hiato negativo. Um phipi mais alto reduz
814         o impacto inicial e acelera pouco a convergencia, mas aprofunda a contracao. O
815         undershooting mostra que "mais agressivo" nao equivale automaticamente a menor
816         variabilidade total.</p>
817
818         <h3>4. Rigidez nominal e razao de sacrificio</h3>
819         <p>Com kappa maior, uma dada contracao do hiato produz mais desinflacao; por isso, o custo
820         real do choque de custo cai. Mas a mesma inclinacao faz expansoes de demanda aparecerem
821         mais rapidamente nos precos. A estrutura da inflacao determina tanto a eficacia quanto
822         o custo da politica.</p>
823
824         <h3>5. Taxa neutra e postura da politica</h3>
825         <p>A postura nao deve ser avaliada apenas pelo nivel nominal da TPM. E necessario
826         compara-la com inflacao esperada e r*. No exercicio prospectivo, r*=3% e meta de 3%
827         implicam neutro nominal de 6,09%. Essa hipotese faz 4,5% parecer expansionista. Como r*
828         e incerto e variavel no tempo, a conclusao e condicional a calibracao, nao avaliacao
829         historica definitiva.</p>
830
831         <h3>6. FEVD, narrativa e identificacao</h3>
832         <p>A FEVD foi desenhada para uma narrativa transparente: inflacao majoritariamente por
833         custo, taxa por inovacao monetaria e hiato por demanda e politica. Ela organiza o
834         experimento, mas nao identifica choques chilenos. Para uma afirmacao historica seriam
835         necessarias restricoes de identificacao, dados adicionais e comparacao com SVAR ou
836         DSGE estimado.</p>
837

```

```

838         <h3>7. Implicacao para o Chile</h3>
839         <p>O nucleo NK ajuda a pensar expectativas, gradualismo e trade-offs, mas omite canais
840         centrais de uma economia pequena e aberta. Cambio, preco do cobre, premio de risco,
841         inflacao importada e demanda externa podem alterar tanto a origem quanto a transmissao
842         da inflacao. A recomendacao metodologica e usar este modelo como benchmark e nao como
843         sistema completo para prescricao.</p>
844         """,
845     )
846 )
847
848 parts.append(
849     section(
850         "limitacoes",
851         "Escopo e honestidade",
852         "O que a entrega nao demonstra",
853         """
854         <div class="two-col">
855             <div>
856                 <h3>Limitacoes economicas</h3>
857                 <ul>
858                     <li>Economia fechada: sem cambio, UIP, cobre, termos de troca ou risco-pais.</li>
859                     <li>Sem indexacao, habitos, capital, salarios rigidos ou friccoes financeiras.</li>
860                     <li>* constante e hiato obtido por filtro estatistico.</li>
861                     <li>Uma unica regra linear para regimes monetarios distintos.</li>
862                 </ul>
863             </div>
864             <div>
865                 <h3>Limitacoes empiricas</h3>
866                 <ul>
867                     <li>Covid-19 e um outlier grande para amostra e filtro.</li>
868                     <li>Instrumentos fracos na NKPC forward e na inflacao da regra de Taylor.</li>
869                     <li>MAP sem MCMC e intervalos apenas locais.</li>
870                     <li>Choques calibrados, nao identificados por likelihood no baseline.</li>
871                 </ul>
872             </div>
873         </div>
874         <h3>Extensoes prioritarias</h3>
875         <ol>
876             <li>Modelo NK de economia aberta com cambio, inflacao importada e premio de risco.</li>
877             <li>* e produto potencial em espaco de estados.</li>
878             <li>Estimacao bayesiana completa com MCMC, erros de medida e tratamento da pandemia.</li>
879             <li>Identificacao externa ou SVAR para comparar choques monetarios e de oferta.</li>
880             <li>Avaliacao fora da amostra das previsoes e comparacao com projecoes do BCCh.</li>
881         </ol>
882         """,
883     )
884 )
885
886 parts.append(
887     section(
888         "reproducao",
889         "Projeto reproduzivel",
890         "Arquitetura, comandos e verificacoes",
891         f"""
892         <div class="pipeline">
893             <span>BCCh API</span><b>-></b><span>dados trimestrais</span><b>-></b>
894             <span>estimacoes</span><b>-></b><span>calibracao</span><b>-></b>
895             <span>modelos_mod</span><b>-></b><span>Octave + Dynare</span><b>-></b>
896             <span>tabelas, figuras e HTML</span>
897         </div>
898         <h3>Execucao completa a partir da raiz</h3>
899         <pre><code>python -m pip install -r requirements.txt
900 python python/build_chile_dataset.py
901 python python/estimate_rhoi_chile.py
902 python python/estimate_taylor_rule.py
903 python python/estimate_nkpc.py
904 python python/estimate_rstar.py
905 python python/calibrate_shocks.py
906 python python/make_tables.py
907 python python/generate_dynare_models.py
908 python python/run_dynare_batch.py --all --timeout 300
909 python python/run_bayesian.py
910 python python/collect_outputs.py
911 python python/analyze_determinacy.py
912 python python/analyze_model_results.py
913 python python/run_forecast.py
914 python python/forecast_model.py
915 python python/plot_irfs.py
916 python python/run_history.py
917 python python/plot_history.py
918 python python/analyze_svar.py
919 python python/estimate_time_varying_rstar.py
920 python python/evaluate_forecasts.py
921 python python/build_open_economy_dataset.py
922 python python/generate_macro_extension_models.py
923 python python/run_macro_extensions.py
924 python python/analyze_macro_extensions.py
925 python python/plot_mcmc.py
926 python python/compare_bayesian_models.py --reuse-baseline
927 python python/build_final_html.py</code></pre>
928         <h3>Tratamento do Windows e OneDrive</h3>
929         <p>O runner copia os modelos para uma pasta temporaria com caminho ASCII antes de
930         iniciar o Octave. Isso evita travamentos do pre-processador do Dynare ao criar e
931         renomear pastas em caminhos sincronizados, com espacos e acentos. Resultados CSV sao
932         copiados de volta para <code>outputs/dynare</code>.</p>
933         <h3>Checklist</h3>
934         <ul class="checklist">
935             <li>Dados marcados como reais em <code>dataset_metadata.json</code>.</li>
936             <li>19 modelos com saidas Dynare e diagnostico BK.</li>
937             <li>{figure_count} figuras publicas incorporadas neste arquivo.</li>
938             <li>Tabelas carregadas diretamente dos CSVs produzidos pelo pipeline.</li>
939             <li>Codigo integral incorporado abaixo.</li>
940             <li>PDF privado e credenciais fora do versionamento.</li>
941         </ul>
942         """,
943     )
944 )

```



```

943     )
944 )
945
946 python_files = sorted((ROOT / "python").glob("*.py"))
947 dynare_core = sorted(
948     path for path in (ROOT / "dynare").iterdir()
949     if path.is_file() and path.suffix.lower() in {".m", ".mod"})
950 )
951 generated_mods = sorted((ROOT / "dynare" / "generated").glob("*.mod"))
952 code_blocks = [
953     "<h3>Python</h3>",
954     *(code_details(path) for path in python_files),
955     "<h3>Octave e Dynare: arquivos centrais</h3>",
956     *(code_details(path) for path in dynare_core),
957     "<h3>Dynare: cenários gerados</h3>",
958     *(code_details(path) for path in generated_mods),
959     "<h3>Configuracao</h3>",
960     code_details(ROOT / "requirements.txt"),
961 ]
962 parts.append(
963     section(
964         "codigo",
965         "Apendice tecnico",
966         "Codigo-fonte integral usado na entrega",
967         """
968         <p>Cada bloco e expansivel. Os arquivos sao incorporados como texto no momento da
969         geracao do HTML; portanto, este apendice representa exatamente a versao usada para
970         produzir as tabelas e figuras desta entrega.</p>
971         <div class="code-toolbar">
972             <button type="button" onclick="toggleAll(true)">Expandir todos</button>
973             <button type="button" onclick="toggleAll(false)">Recolher todos</button>
974             <input id="codeSearch" type="search" placeholder="Filtrar arquivos..." oninput="filterCode(this.value)">
975         </div>
976         """
977         + "\n".join(code_blocks),
978     )
979 )
980
981 nav_html = "".join(f'<a href="#{sid}>{esc(label)}</a>' for sid, label in nav)
982 body = "\n".join(parts)
983 styles = """
984 :root{--ink:#132238;--muted:#5d6b7d;--paper:#f5f1e8;--card:#ffffd8;--navy:#0d3558;
985 --blue:#1b668f;--teal:#19756f;--gold:#bd7d16;--red:#a53c35;--line:#d8d1c3;
986 --shadow:0 12px 36px rgba(20,35,52,.10)}
987 *(box-sizing:border-box)html{scroll-behavior:smooth}body{margin:0;background:var(--paper);
988 color:var(--ink);font:16px/1.67 "Segoe UI",Arial,sans-serif}
989 header{min-height:76vh;padding:8rem max(6vw,2rem) 5rem;color:white;position:relative;overflow:hidden;
990 background:linear-gradient(125deg,#071d31 0%,#0d3f63 52%,#16756f 100%)}
991 header:after{content:"";position:absolute;right:-12vw;top:-12vw;width:46vw;height:46vw;
992 border:1px solid rgba(255,255,255,.2);border-radius:50%;box-shadow:0 0 5vw rgba(255,255,255,.025),
993 0 0 11vw rgba(255,255,255,.02)}.eyebrow{letter-spacing:.18em;text-transform:uppercase;
994 font-weight:700;color:#a6e4dc}h1{font-family:Georgia,serif;font-size:clamp(3rem,7vw,6.8rem);
995 max-width:1100px;line-height:1.02;margin:.4rem 0 1.5rem}.subtitle{font-size:1.3rem;max-width:780px;
996 color:#dceaf2}.cover-meta{display:flex;gap:1rem;flex-wrap:wrap;margin-top:2rem}
997 .cover-meta span{border:1px solid rgba(255,255,255,.35);padding:.45rem .8rem;border-radius:999px}
998 nav{position:sticky;top:0;z-index:20;background:rgba(255,253,248,.96);backdrop-filter:blur(12px);
999 border-bottom:1px solid var(--line);display:flex;gap:.2rem;overflow-x:auto;padding:.55rem 2vw;
1000 box-shadow:0 3px 15px rgba(0,0,0,.05)}nav a{white-space:nowrap;color:var(--navy);
1001 text-decoration:none;padding:.4rem .65rem;border-radius:6px;font-size:.86rem;font-weight:650}
1002 nav a:hover{background:#e4e6e9}main{max-width:1180px;margin:auto;padding:1rem 2rem 6rem}
1003 section{padding:5rem 0;border-bottom:1px solid var(--line);scroll-margin-top:4rem}
1004 .section-kicker{text-transform:uppercase;letter-spacing:.16em;color:var(--teal);font-weight:800;
1005 font-size:.78rem}h2{font:700 clamp(2.2rem,4vw,4rem)/1.08 Georgia,serif;margin:.35rem 0 2rem;
1006 color:var(--navy)}h3{color:var(--navy);margin-top:2rem}p{max-width:920px}.lead{font-size:1.3rem;
1007 line-height:1.65}.metric-grid{display:grid;grid-template-columns:repeat(4,1fr);gap:1rem;margin:2rem 0}
1008 .metric{background:var(--navy);color:white;padding:1.4rem;border-radius:10px;box-shadow:var(--shadow)}
1009 .metric span{display:block;font:700 2.3rem Georgia,serif;color:#8ed7cc}.metric small{display:block}
1010 .two-col,.three-col{display:grid;grid-template-columns:repeat(2,1fr);gap:1.2rem;margin:1.5rem 0}
1011 .three-col{grid-template-columns:repeat(3,1fr)}.card,.audit-grid article,.parameter-grid article{
1012 background:var(--card);border:1px solid var(--line);border-radius:10px;padding:1.2rem;box-shadow:0 4px 18px rgba(0,0,0,.04)}
1013 .audit-grid,.parameter-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:.8rem;margin:1.5rem 0}
1014 .audit-grid b,.parameter-grid b{display:block;color:var(--navy);font-size:1.05rem}
1015 .audit-grid span,.parameter-grid span{display:block;color:var(--muted);margin-top:.35rem}
1016 .callout{padding:1.2rem 1.4rem;border-left:5px solid;margin:1.5rem 0;background:white;
1017 border-radius:4px 10px 10px 4px;box-shadow:0 5px 20px rgba(0,0,0,.05)}
1018 .callout strong{display:block;font-size:1.05rem;margin-bottom:.25rem}.callout.success{border-color:var(--teal)}
1019 .callout.info{border-color:var(--blue)}.callout.warning{border-color:var(--gold)}
1020 .callout.danger{border-color:var(--red)}figure{margin:2rem 0;background:var(--card);padding:1rem;
1021 border:1px solid var(--line);border-radius:12px;box-shadow:var(--shadow)}figure img{display:block;width:100%;
1022 height:auto;border-radius:7px}figcaption{padding:1rem .4rem .5rem;color:var(--navy)}
1023 figcaption span,.source-note{color:var(--muted);font-size:.88rem}.figure-analysis{background:#edf4f1;
1024 padding:1rem;border-radius:7px;margin:.5rem .3rem;color:#24443f}.data-table{width:100%;border-collapse:collapse;
1025 display:block;overflow-x:auto;background:var(--card);margin:1rem 0 2rem;font-size:.88rem}
1026 .data-table th{background:var(--navy);color:white;text-align:left}.data-table th,.data-table td{
1027 border:1px solid #add8c6;padding:.55rem .65rem;white-space:nowrap}.data-table tr:nth-child(even){background:#f1eee7}
1028 .equation{font:1.18rem Georgia,serif;text-align:center;background:var(--card);border:1px solid var(--line);
1029 border-left:5px solid var(--teal);padding:1.2rem;margin:1.2rem 0;overflow-x:auto}.glossary{display:grid;
1030 grid-template-columns:minmax(80px,150px) 1fr;gap:.2rem 1rem}.glossary dt{font-weight:800;color:var(--navy);
1031 padding:.45rem}.glossary dd{margin:0;padding:.45rem;border-bottom:1px dotted var(--line)}
1032 code{font-family:Consolas,"Courier New",monospace;background:#e9e5dd;padding:.08rem .25rem;border-radius:3px}
1033 pre{background:#081521;color:#d7e4ec;padding:1.2rem;border-radius:8px;overflow:auto;max-height:70vh;
1034 line-height:1.45;font-size:.82rem}pre code{background:none;padding:0}.pipeline{display:flex;flex-wrap:wrap;
1035 align-items:center;gap:.5rem;margin:1.5rem 0}.pipeline span{background:var(--navy);color:white;padding:.6rem .8rem;
1036 border-radius:6px}.pipeline b{color:var(--gold)}.checkboxlist{list-style:none;padding:0}.checkboxlist li:before{
1037 content:"?";color:var(--teal);font-weight:900;margin-right:.6rem}.code-file{background:var(--card);
1038 border:1px solid var(--line);border-radius:7px;margin:1.5rem 0}.code-file summary{cursor:pointer;padding:.8rem 1rem;
1039 display:flex;justify-content:space-between;gap:1rem}.code-file summary span{color:var(--muted);font-size:.85rem}
1040 .code-file pre{margin:0;border-radius:0 0 7px 7px}.code-toolbar{position:sticky;top:3.8rem;z-index:10;
1041 display:flex;gap:.5rem;flex-wrap:wrap;background:var(--paper);padding:.7rem 0}.code-toolbar button,
1042 .code-toolbar input{border:1px solid var(--line);background:white;padding:.55rem .75rem;border-radius:6px}
1043 .code-toolbar button{cursor:pointer;background:var(--navy);color:white}.warning{padding:1rem;background:#ffffd7;
1044 border-left:4px solid var(--gold)}footer{background:#071d31;color:#dceaf2;padding:3rem max(5vw,2rem)}
1045 footer a{color:#8ed7cc}@media(max-width:850px){.metric-grid,.audit-grid,.parameter-grid,.three-col,.two-col{
1046 grid-template-columns:1fr 1fr}header{min-height:auto;padding-top:5rem}@media(max-width:560px){
1047 main{padding:1rem}.metric-grid,.audit-grid,.parameter-grid,.three-col,.two-col{grid-template-columns:1fr}

```

```

1048 h1{font-size:3rem}.glossary{grid-template-columns:1fr}.glossary dd{padding-top:0}
1049 @media print{nav,.code-toolbar{display:none}header{min-height:auto;padding:3rem;background:#123!important;
1050 -webkit-print-color-adjust:exact}section{break-inside:auto}figure,.card,.callout{break-inside:avoid}
1051 details:not([open])>*:not(summary){display:block}.code-file pre{max-height:none}}
1052 ""
1053
1054 script = """
1055 function toggleAll(openState){
1056   document.querySelectorAll('.code-file').forEach(function(d){d.open=openState;});
1057 }
1058 function filterCode(term){
1059   term=term.toLowerCase();
1060   document.querySelectorAll('.code-file').forEach(function(d){
1061     d.style.display=d.querySelector('summary').innerText.toLowerCase().includes(term)?'block':'none';
1062   });
1063 }
1064 const observer=new IntersectionObserver(entries=>{
1065   entries.forEach(entry=>{
1066     if(entry.isIntersecting){
1067       document.querySelectorAll('nav a').forEach(a=>a.classList.remove('active'));
1068       const link=document.querySelector('nav a[href="#'+entry.target.id+'"]');
1069       if(link) link.classList.add('active');
1070     }
1071   });
1072 },{rootMargin:'-20% 0px -70% 0px'});
1073 document.querySelectorAll('main section').forEach(s=>observer.observe(s));
1074 ""
1075
1076 document = f`<!doctype html>
1077 <html lang="pt-BR">
1078 <head>
1079   <meta charset="utf-8">
1080   <meta name="viewport" content="width=device-width,initial-scale=1">
1081   <meta name="description" content="Entrega final de politica monetaria: modelo Novo-Keynesiano calibrado para o Chile.">
1082   <title>Entrega Final | Politica Monetaria no Chile</title>
1083   <style>{styles}</style>
1084 </head>
1085 <body>
1086 <header>
1087   <div class="eyebrow">FGV EESP * Politica Monetaria * 2026</div>
1088   <h1>Entrega Final</h1>
1089   <p class="subtitle">Modelo Novo-Keynesiano calibrado para o Chile: dados, teoria,
1090   solucao, evidencia, politica monetaria e codigo reproduzivel.</p>
1091   <div class="cover-meta">
1092     <span>Chile</span><span>2001Q1-2026Q1</span><span><span>Octave 11.1</span>
1093     <span>Dynare 7.1</span><span>Python</span><span><span>Gerado em {date.today().strftime("%d/%m/%Y")}</span>
1094   </div>
1095 </header>
1096 <nav aria-label="Navegacao do relatorio">{nav_html}</nav>
1097 <main>{body}</main>
1098 <footer>
1099   <strong>Entrega Final -- Politica Monetaria no Chile</strong>
1100   <p>Dados: Banco Central de Chile. Resultados do modelo: pipeline publico deste projeto.
1101   Este documento e autocontido e pode ser aberto sem conexao com a internet.</p>
1102   <p>Referencias oficiais:
1103   <a href="https://www.bcentral.cl/web/banco-central/areas/politica-monetaria">marco de politica monetaria</a> *
1104   <a href="https://si3.bcentral.cl/Siete">Base de Datos Estadisticos</a> *
1105   <a href="https://www.dynare.org/manual/">manual do Dynare</a>.</p>
1106 </footer>
1107 <script>{script}</script>
1108 </body>
1109 </html>"""
1110 document = "\n".join(line.rstrip() for line in document.splitlines()) + "\n"
1111 OUTPUT.write_text(document, encoding="utf-8")
1112 print(f"Wrote {OUTPUT} ({OUTPUT.stat().st_size / 1024 / 1024:.2f} MiB)")
1113 print(f"Embedded {len(list(FIGURES.glob('*.png')))} PNG figures")
1114 print(f"Embedded {len(python_files) + len(dynare_core) + len(generated_mods) + 1} source files")
1115
1116
1117 if __name__ == "__main__":
1118   build()

```

49 python/build_codigo_final.py

```

1  """Generate "Codigo Final": ONE PDF document with ALL project code.
2
3  Reads every python/*.py and dynare/*.mod, transliterates non-ASCII characters in
4  the *code* to ASCII (pi, ->, --, accents stripped) so pdflatex + listings never
5  chokes, and writes entrega_aula5/Codigo_Final.tex. Prose/titles keep their
6  accents via inputenc utf8. Build with: entrega_aula5/build.sh Codigo_Final
7
8  The file order is grouped to read like the pipeline; any file not listed is
9  appended automatically, so the document always contains *all* the code.
10 """
11
12 from __future__ import annotations
13
14 import datetime as dt
15 import unicodedata
16 from pathlib import Path
17
18 HERE = Path(__file__).resolve().parent          # python/
19 ROOT = HERE.parent
20 MOD = ROOT / "dynare"
21 OUT = ROOT / "entrega_aula5" / "Codigo_Final.tex"
22
23 # Non-ASCII -> ASCII so the code listings are pure ASCII.
24 GREEK = {
25     "pi": "pi", "kappa": "kappa", "beta": "beta", "sigma": "sigma", "phi": "phi", "rho": "rho",
26     "alpha": "alpha", "theta": "theta", "lambda": "lambda", "gamma": "gamma", "delta": "delta",
27     "epsilon": "epsilon", "eta": "eta", "mu": "mu", "nu": "nu", "tau": "tau", "omega": "omega",
28     "Phi": "Phi", "Sigma": "Sigma", "Pi": "Pi", "Delta": "Delta", "Omega": "Omega",
29 }
30
31 SYM = {
32     "-": "-", "++": "++", "x": "x", "->": "->", "<=": "<=", "~2": "~2", "~3": "~3",
33     "!=": "!=", "<=": "<=", ">=": ">=", "!=": "!=", "!=": "!=", "!=": "!=", "deg": "deg", "+/-": "+/-", "inf": "inf",
34     "sqrt": "sqrt", "sum": "sum", "in": "in", "d": "d", "=="": "==", "Delta": "Delta",
35     " ": " ", " ": " ", " ": " ", " ": " ", " ": " ",
36 }
37
38
39 def translit(text: str) -> str:
40     out = []
41     for ch in text:
42         if ord(ch) < 128:
43             out.append(ch)
44         elif ch in GREEK:
45             out.append(GREEK[ch])
46         elif ch in SYM:
47             out.append(SYM[ch])
48         else:
49             d = unicodedata.normalize("NFKD", ch).encode("ascii", "ignore").decode()
50             out.append(d if d else "?")
51     return "".join(out)
52
53
54 # Grouped, pipeline-ordered. Remaining files are appended in a final group.
55 GROUPS: list[tuple[str, list[str]]] = [
56     ("Infraestrutura compartilhada", ["common.py"]),
57     ("Dados (BCCh / FRED / filtro de Kalman)", [
58         "build_chile_dataset.py", "reprocess_dataset.py",
59         "build_open_economy_dataset.py", "discover_ipc_series.py"]),
60     ("Geracao dos modelos Dynare", [
61         "generate_dynare_models.py", "generate_macro_extension_models.py"]),
62     ("Modelos Dynare (.mod)", [
63         "nk_chile_base.mod", "nk_chile_estim.mod", "nk_chile_hybrid.mod",
64         "nk_chile_hybrid_estim.mod", "nk_chile_forecast.mod",
65         "nk_chile_history.mod", "nk_chile_history_hybrid.mod",
66         "nk_chile_mcmc.mod", "nk_chile_open.mod"]),
67     ("Solucao e calibracao", [
68         "hybrid_solution.py", "calibrate_shocks.py", "analyze_determinacy.py"]),
69     ("Estimacao dos parametros", [
70         "estimate_nkpc.py", "estimate_taylor_rule.py", "estimate_rhoi_chile.py",
71         "estimate_rstar.py", "estimate_time_varying_rstar.py"]),
72     ("Execucao (Octave/Dynare + Bayesiano)", [
73         "run_dynare_batch.py", "run_bayesian.py", "run_mcmc.py",
74         "run_forecast.py", "run_history.py", "run_macro_extensions.py"]),
75     ("Previsao e avaliacao", [
76         "forecast_model.py", "evaluate_forecasts.py"]),
77     ("Analise (SVAR, Bayes, resultados)", [
78         "analyze_svar.py", "compare_bayesian_models.py",
79         "analyze_model_results.py", "analyze_macro_extensions.py"]),
80     ("Nucleos da inflacao e mercado de trabalho", [
81         "build_inflation_cores.py", "build_inflation_cores_bcch.py",
82         "build_labor_activity.py"]),
83     ("Graficos", [
84         "plot_history.py", "plot_ipc_decomposition.py", "plot_forecast_outlook.py",
85         "plot_hybrid_test.py", "plot_irfs.py", "plot_mcmc.py"]),
86     ("Tabelas, coleta e entregaveis", [
87         "collect_outputs.py", "make_tables.py", "build_final_html.py",
88         "build_codigo_final.py"]),
89 ]
90
91
92 def resolve(name: str) -> Path:
93     return (MOD / name) if name.endswith(".mod") else (HERE / name)
94
95
96 def latex_escape_title(name: str) -> str:
97     return name.replace("\\", r"\textbackslash").replace("_", r"\_") \
98         .replace("&", r"&").replace("%", r"%").replace("#", r"#")
99
100
101 PREAMBLE = r"""
102 \documentclass[9pt]{extarticle}
103 \usepackage[a4paper,margin=1.8cm]{geometry}

```

```

103 \usepackage[utf8]{inputenc}
104 \usepackage[Ti]{fontenc}
105 \usepackage{lmodern}
106 \usepackage{xcolor}
107 \usepackage{listings}
108 \usepackage[hidelinks]{hyperref}
109 \definecolor{cmt}{RGB}{96,139,84}
110 \definecolor{kw}{RGB}{0,0,170}
111 \definecolor{str}{RGB}{163,21,21}
112 \definecolor{ln}{RGB}{150,150,150}
113 \lstset{
114   basicstyle=\ttfamily\scriptsize,
115   breaklines=true,breakatwhitespace=false,
116   columns=fullflexible,keepspaces=true,
117   showstringspaces=false,upquote=true,tabsize=4,
118   commentstyle=\color{cmt},keywordstyle=\color{kw},stringstyle=\color{str},
119   numbers=left,numberstyle=\tiny\color{ln},numbersep=6pt,xleftmargin=2.2em,
120   frame=lines,framesep=4pt,
121 }
122 \title{\textbf{C\`odigo Final}\[2mm]\large Modelo Novo-Keynesiano de Pol\`itica
123 Monet\`aria para o Chile\[\normalsize Todo o c\`odigo (Python + Dynare) em um
124 \`unico documento}
125 \author{Daniel Colli e Jo\~ao Zangrandi}
126 \date{REPLACE\_DATE}
127 \begin{document}
128 \maketitle
129 {\small Este documento re\`une \textbf{todo} o c\`odigo do projeto: TARGET\_NFILES
130 arquivos (Python e Dynare \texttt{.mod}), agrupados na ordem do \emph{pipeline}.
131 Caracteres n\~ao-ASCII do c\`odigo (letras gregas, acentos) foram transliterados
132 para ASCII na listagem para garantir a compila\c{c}\~ao; a l\`ogica \`e id\`entica
133 \`a dos arquivos-fonte do reposit\`orio.\par}
134 \tableofcontents
135 \clearpage
136 ""
137
138
139 def main() -> None:
140     used: list[str] = []
141     for _, names in GROUPS:
142         used.extend(names)
143     # Append any file not explicitly grouped.
144     extra = sorted(p.name for p in HERE.glob("*.py") if p.name not in used)
145     extra += sorted(p.name for p in MOD.glob("*.mod") if p.name not in used)
146     groups = GROUPS + ([("Outros", extra)] if extra else [])
147
148     nfiles = sum(len(n) for _, n in groups)
149     parts = [PREAMBLE.replace("REPLACE\_DATE", dt.date.today().strftime("%d/%m/%Y"))
150             .replace("TARGET\_NFILES", str(nfiles))]
151
152     for title, names in groups:
153         parts.append(f"\part*{{{title}}}\addcontentsline{toc}{{part}}{{{title}}}\n")
154         for name in names:
155             path = resolve(name)
156             if not path.exists():
157                 continue
158             code = translit(path.read_text(encoding="utf-8", errors="replace"))
159             code = code.replace("\t", " ").rstrip("\n")
160             lang = "Python" if name.endswith(".py") else "{}"
161             rel = ("dynare/" if name.endswith(".mod") else "python/") + name
162             parts.append(f"\section{{{latex_escape_title(rel)}}}")
163             parts.append(f"\begin{lstlisting}[language={lang}]")
164             parts.append(code)
165             parts.append("\end{lstlisting}\n\clearpage")
166
167     parts.append("\end{document}\n")
168     OUT.write_text("\n".join(parts), encoding="utf-8")
169     print(f"Wrote {OUT} ({nfiles} files, {OUT.stat().st_size//1024} KB).")
170
171
172 if __name__ == "__main__":
173     main()

```